GEK-65426A

DRIVE SYSTEM INSTRUCTIONS

Directo-Matic® PLUS

CONTROL BLOCK LANGUAGE REFERENCE MANUAL

General Electric Company
Drive Systems Department
Salem, Virginia U.S.A.

GENERAL ELECTRIC

(GEK-65426A)

ISSUE DATE: April 1980
REVISION DATE: June 1982

The information herein does not purport to cover all details or variations in equipment nor to provide for every possible contingency to be met in connection with installation, operation, and maintenance. Should further information be desired or should particular problems arise which are not covered sufficiently for the purchaser's purposes, the matter should be referred to General Electric Company, Drive Systems Department, USA.

TABLE OF CONTENTS

SECTION		PAGE
1	INTRODUCTION TO MANUAL	1-1
2	INTRODUCTION TO CONTROL BLOCK LANGUAGE	2-1
3	ELEMENTS OF CONTROL BLOCK LANGUAGE	3-1
3-1	SEGMENTS	3-1
3-2	Continuous Segments	3-5
3-3	Initialize Segments	3-10
3-4	Interrupt Segments	3-10
3-5	Procedural Segments	3-11
3-6	CONTROL BLOCKS	3-14
3-7	Relay Ladder Blocks	3-14
3-8	Logic Blocks	3-15
3-9	Regulating Blocks	3-16
3-10	Pseudo and Special Control Blocks	3-17
4	CONTROL BLOCK REFERENCE SHEETS	4-1
4-1	Control Block Descriptiong Summary in Control Block Order	

CONTROL BLOCK NAME	OP CODE (HEX)	OP CODE (OCTAL)	BLOCK DESCRIPTION	PAGE NO.
AABS	8B80	105600	Analog Absolute Value	4-6
ACIN	8B81	105601	Analog Clamped Integrator	4-8
ACLP	8B82	105602	Analog Clamp	4-10
ACMP	8B83	105603	Analog Compare	4-12
ACNV	8B84	105604	Analog Conversion	4-14
ADIF	8B85	105605	Analog Differentiator	4-16
ADTL	8B86	105606	Analog Dead Time List	4-18
ADVD	8B87	105607	Analog Divide	4-20
AFGS	8B88	105610	Analog Function Generator (Simple)	4-21
AFGT	8B89	105611	Analog Function Generator (Tabular)	4-23
AGAI	8B8A	105612	Analog Gain	4-25
AHLD	8B8B	105613	Analog Hold	4-28
AICB	8B8C	105614	Analog Input Converter	4-30
ALLG	8B8D	105615	Analog Lead-Lag	4-34
AMAX	8B8E	105616	Analog Maximum	4-36
AMIN	8B8F	105617	Analog Minimum	4-38
AMPY	8B90	105620	Analog Multiply	4-40
ANSW	8B91	105621	Analog Switch	4-42
AOCB	8B92	105622	Analog Output Converter	4-44
APIN	8B93	105623	Analog Integrator with Lead	4-48
ASQR	8B94	105624	Analog Square Root	4-50
ASUM	8B95	105625	Analog Summer	4-52
AVRG	8B96	105626	Analog Average	4-54
CADD	8B20	105440	Count Add	4-56

CONTROL BLOCK NAME	OP CODE (HEX)	OP CODE (OCTAL)	BLOCK DESCRIPTION	PAGE NO.
CCMP	8B21	105441	Count Comparator	4-58
CCNV	8B22	105442	Count Conversion	4-60
CCTL	8B23	105443	Count to Logic	4-62
CICB	8B24	105444	Count Input Control	4-64
CLTC	8B25	105445	Logic to Count	4-66
CNTR	8B26	105446	Count Counter	4-68
COCB	8B27	105447	Count Output Control	4-70
CPAK	8B2C	105454	Count Pack Word	4-72
CSHR	8B28	105450	Count Shift Register	4-74
CSUB	8B29	105451	Count Subtract	4-76
CTMR	8B2A	105452	Count Timer	4-78
KCTR	8B51	105521	Counter	4-80
KDLY	8B45	105505	Time Delay	4-84
KHLD	8B4A	105512	Hold Logic	4-86
KLCH	8B42	105502	Latched Relay	4-88
KPLS	8B44	105504	Pulsed Relay	4-90
KRLY	8B49	105511	Relay Ladder	4-92
LAND	8B47	105507	Logic AND	4-94
LANO	8347	101507	Logic AND Output	4-96
LCMP	8B4D	105500	Logic Complement	4-98
LFFD	8B41	105501	Logic D Type Flip-Flop	4-100
LFSR	8B42	105502	Logic Set-Reset Flip-Flop	4-102
LGIC	8B49	105502 or 105511	Logic Boolean Operator	4-104
LORI	8B48	105510	Logic OR	4-108
LORO	8348	101510	Logic OR Output (Inclusive)	4-110
LSHT	8B44	105504	Logic Single Shot	4-112
LTDL	8B45	105505	Logic Time Delay	4-114
LXIT	8B4A	105512	Logic Exit	4-116
PACLP	8A60	105140	Clamp	4-118
PACTV	8A70	105160 or 105172	Activate Segment	4-120
PAFGT	8A61	105141	Function Generator (Tabular)	4-124
PCALC	8A62	105142	Calculator	4-126
PCALL	8A76	105166	Call Submodule	4-128
PCONV	8A63	105143	Conversion	4-130
PDELY	8A71	105161 or 105173	Delay Segment	4-132
PDOTL	8A7D	105175 or 105176	Do Until True	4-136
PDOWL	8A7F	105177	Do While True	4-138
PFIND	8A64	105144	Table Search	4-140
PFOFX	8A65	105145 or 105627	Function of Input	4-142
PLCHK	8A69	105151	Limit Check	4-144
PLGIC	8A77	105167	Logical Operator	4-146
PMAIN	8A72	105162	Beginning of Main Module	4-148
PMEND	8A73	105163	End of Main Module	4-150
PMOVE	8A66	105146	Move Variable	4-152

CONTROL BLOCK NAME	OP CODE (HEX)	OP CODE (OCTAL)	BLOCK DESCRIPTION	PAGE NO.
PRETN	8A78	105170	Return From Submodule	4-154
PSUBM	8A79	105171	Beginning of Submodule	4-156
PSUSP	8A74	105164	Suspend Segment	4-158
PTERM	8A75	105165	Terminate Segment	4-160
PTEST	8A6A	105152	Compare Values	4-162
PTRKO	8A67	105147	Track Open	4-164
PTRKR	8A68	105150	Track Ring	4-167
PTXTI	8A6E	105156 or 105157	Text Input	4-168
PTXTO	8A6B	105153	Text Output	4-172
PTXTS	8A6D	105155	Text Select	4-178
DNST	None	-	Constant Assignment	4-180
CTRF	8B50	105520	Continuous Transfer	4-182
DATV	None	-	Array Assignment	4-186
DIDB	None	-	Data Link Interrupt	4-190
INZO	834B	101515	Initialize Outputs	4-192
INZS	8B4B	105510	Initialize Data	4-194
INZV	8B4F	105517	Initialize Array	4-196
PDATV	None	-	Array Assignment	4-198
RBEG	8B2D	105435	Begin Reusable Sequence	4-200
RCALL	8A2F	105057	Call Reusable Sequence	4-202
REND	8B2E	105456	End Reusable Sequence	4-204

4-2 Control Block Description Summary
In Hexadecimal Op Code Order

OP CODE (HEX)	OP CODE (OCTAL)	CONTROL BLOCK NAME	BLOCK DESCRIPTION	PAGE No.
8347	101507	LANO	Logic AND Output	4-96
8348	101510	LORO	Logic OR Output (Inclusive)	4-110
834B	101515	INZO	Initialize Outputs	4-192
8A2F	105057	RCALL	Call Reusable Sequence	4-202
8A60	105140	PACLP	Clamp	4-118
8A61	105141	PAFGT	Function Generator (Tabular)	4-124
8A62	105142	PCALC	Calculator	4-126
8A63	105143	PCONV	Conversion	4-130
8A64	105144	PFIND	Table Search	4-140
8A65	105145 or 105627	PFOFX	Function of Input	4-142
8A66	105146	PMOVE	Move Variable	4-152
8A67	105147	PTRKO	Track Open	4-164
8A68	105150	PTRKR	Track Ring	4-167
8A69	105151	PLCHK	Limit Check	4-144
8A6A	105152	PTEST	Compare Values	4-162
8A6B	105153	PTXTO	Text Output	4-172
8A6D	105155	PTXTS	Text Select	4-178
8A6E	105156 or 105157	PTXTI	Text Input	4-168
8A70	105160 or 105172	PACTV	Activate Segment	4-120

CONTROL BLOCK NAME	OP CODE (HEX)	OP CODE (OCTAL)	BLOCK DESCRIPTION	PAGE NO.
8A72	105162	PMAIN	Beginning of Main Module	4-148
8A73	105163	PMEND	End of Main Module	4-150
8A74	105164	PSUSP	Suspend Segment	4-158
8A75	105165	PTERM	Terminate Segment	4-160
8A76	105166	PCALL	Call Submodule	4-128
8A77	105167	PLGIC	Logical Operator	4-146
8A78	105170	PRETN	Return From Submodule	4-154
8A79	105171	PSUBM	Beginning of Submodule	4-156
8A7D	105175			
	or			
	105176	PDOTL	Do Until True	4-136
8F7F	105177	PDOWL	Do While True	4-138
8B20	105440	CADD	Count Add	4-56
8B21	105441	CCMP	Count Comparator	4-58
8B22	105442	CCNV	Count Conversion	4-60
8B23	105443	CCTL	Count to Logic	4-62
8B24	105444	CICB	Count Input Control	4-64
8B25	105445	CLTC	Logic to Count	4-66
8B26	105446	CNTR	Count Counter	4-68
8B27	105447	COCB	Count Output Control	4-70
8B28	105450	CSHR	Count Shift Register	4-74
8B29	105451	CSUB	Count Subtract	4-76
8B2A	105452	CTMR	Count Timer	4-78
8B2C	105454	CPAK	Count Pack Word	4-72
8B2D	105435	RBEG	Begin Reusable Sequence	4-200
8B2E	105456	REND	End Reusable Sequence	4-204
8B41	105501	LFFD	Logic D Type Flip-Flop	4-100
8B42	105502	LFSR		
		and	Logic set-Reset Flip-Flop	4-102
		KLCH	Latched Relay	4-88
8B44	105504	LSHT		
		and	Logic Single Shot	4-112
		KPLS	Pulsed Relay	4-90
8B45	105505	LTDL		
		and	Logic Time Delay	4-114
		KDLY	Time Delay	4-84
8B47	105507	LAND	Logic AND	4-94
8B48	105510	LORI	Logic OR	4-108
8B49	105503	LGIC		
		and	Logic Boolean Operator	4-104
	105511	KRLY	Relay Ladder	4-92
8B4A	105512	LXIT		
		and	Logic Exit	4-116
		KHLD	Hold Logic	4-86
8B4B	105510	INZS	Initialize Data	4-194
8B4D	105500	LCMP	Logic Complement	4-98
8B4F	105517	INZV	Initialize Array	4-196
8B50	105520	CTRF	Continuous Transfer	4-182
8B51	105521	KCTR	Counter	4-80
8B80	105600	AABS	Analog Absolute Value	4-6
8B81	105601	ACIN	Analog Clamped Integrator	4-8
8B82	105602	ACLP	Analog Clamp	4-10

CONTROL BLOCK NAME	OP CODE (HEX)	OP CODE (OCTAL)	BLOCK DESCRIPTION	PAGE NO.
8B83	105603	ACMP	Analog Compare	4-12
8B84	105604	ACNV	Analog Conversion	4-14
8B85	105605	ADIF	Analog Differentiator	4-16
8B86	105606	ADTL	Analog Dead Time List	4-18
8B87	105607	ADVD	Analog Divide	4-20
8B88	105610	AFGS	Analog Function Generator (Simple)	4-22
8B89	105611	AFGT	Analog Function Generator (Tabular)	4-24
8B8A	105612	AGAI	Analog Gain	4-26
8B8B	105613	AHLD	Analog Hold	4-28
8B8C	105614	AICB	Analog Input Converter	4-30
8B8D	105615	ALLG	Analog Lead-Lag	4-34
8B8E	105616	AMAX	Analog Maximum	4-36
8B8F	105617	AMIN	Analog Minimum	4-38
8B90	105620	AMPY	Analog Multiply	4-40
8B91	105621	ANSW	Analog Switch	4-42
8B92	105622	AOCB	Analog Output Converter	4-44
8B93	105623	APIN	Analog Integrator with Lead	4-48
8B94	105624	ASQR	Analog Square Root	4-50
8B95	105625	ASUM	Analog Summer	4-52
8B96	105626	AVRG	Analog Average	4-54
None	-	CNST	Constant Assignment	4-180
None	-	DATV	Array Assignment	4-186
None	-	PDATV	Array Assignment	4-198
None	-	DIDB	Data Link Interrupt	4-190

SECTION 5 APPENDICES 4-206

A	CBL Performance Data	4-206
B	Reusable Sequence	4-212
C	CBL Macro Instruction Formats and Addressing	4-215
D	CBL Function Flow Charts	4-226

LIST OF ILLUSTRATIONS

Figure No.	Title	Page No.
3-1	Segment Identification	3-3

LIST OF TABLES

Table No.	Title	Page No.
4-1	Control Block Signal Description	4-3
4-2	Macro Descriptors	4-4

SECTION 1

INTRODUCTION TO MANUAL

1-1 Purpose of Manual. This publication provides a repository of Control Block Language (CBL) material which can be referred to on a random basis. The front part of the manual contains an introductory description of the language along with rules for its use. A detailed definition of all control blocks and their characteristics is given. A group of block reference sheets is given which describes the operating parameters of each control block.

1-2 Audience Level. The presented material is addressed to experienced electricians or technicians primarily in hardware applications.

SECTION 2

INTRODUCTION TO CONTROL BLOCK LANGUAGE

Control block language (CBL) is a graphic means of entering functional control programs into the DIRECTO-MATIC ® Plus Programmable Master Controller (PMC). The CBL system shows the operator how the system is functioning and how system malfunctions can be diagnosed. System functioning can be modified off line without interrupting an on-going process. Thereafter, a modified system can be implemented through proper on-line operation.

Tuneup and adjustment procedures are enhanced by use of the CBL system. All levels of process control complexity are within the grasp of CBL whether it is used for a simple sequencing scheme or for complex control systems such as automatic gauge control (AGC).

Throughout this manual, the abbreviation CSH is used in reference to Control Signal Highway. The CSH is the external signal I/O subsystem described in detail in GE publication GEK-65411.

This document contains Proprietary Information of General Electric Company, U.S.A., is furnished by the Company to its customer solely for purposes of assisting that customer in the installation, testing and/or maintenance of the equipment described herein and shall not be reproduced in whole or in part nor shall the contents hereof be disclosed to any third party without the written approval of General Electric Company, Drive Systems Department.

SECTION 3

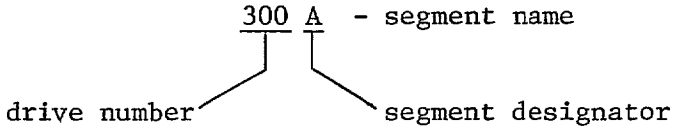
ELEMENTS OF CONTROL BLOCK LANGUAGE

3-1 SEGMENTS

Before describing the form of control blocks it is necessary to discuss the discrete entities that the controller recognizes and will cause to be executed. These programs or portions of executable machine code are known as segments. Each segment has its individual name. Each segment is associated with a given drive number. The name of the segment consists of a maximum of three digits plus an alphabetic character.

Example: 300A
251B
251C, etc.

The segment name is composed of two quantities: the drive number and segment designator as follows:



This is referred to as segment A in drive 300 or by its full name: segment 300A.

Each segment is comprised of a number of blocks, each of which performs a single calculation or function. These control blocks and their use are the primary concern of the majority of this manual. A segment identification block is shown in Figure 3-1.

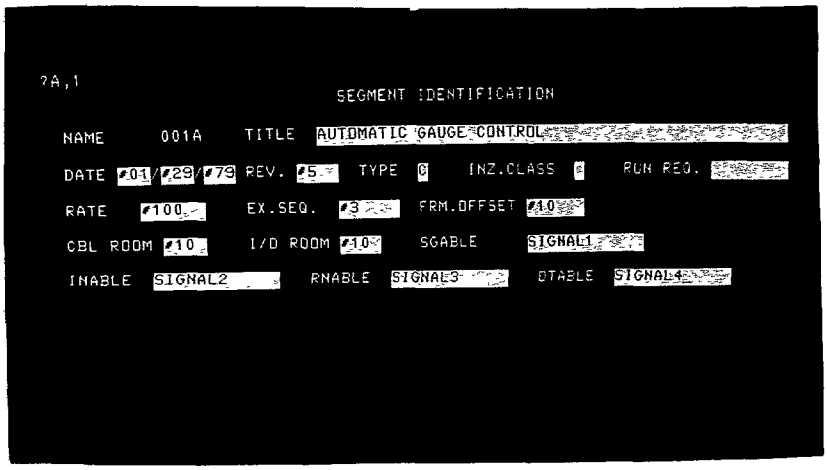
Within the control block language, four types of segments are defined.
They are:

Continuous
Initialize
Interrupt
Procedural

The interrupt, initialize and continuous segments are similar, differing only in the rate at which they are executed. Interrupt segments run in response to external signals. Initialize segments

run only once when the PMC is initialized. Continuous segments run repeatedly at a fixed repetition rate which is in multiples of 10 milliseconds. These segment types are intended for use in applications involving analog or digital control.

The procedural segments provide a structured language for defining iterative processes and algorithmic functions.



(Typical values shown in inverse video fields)

FORMS *	VARIABLE TYPE	HEADINGS	DESCRIPTION
SPECIAL	TEXT	TITLE	Alpha-numeric description of segment
C/C/C	COUNT	DATE	Current revision month, day, year
C	COUNT	REV.	Numeric Revision level **
OC		TYPE***	P-Procedural, C-Continuous I-Interrupt, Z-Initialize
OC	-	INZ CLASS	Initialize segment type for initialize segments only. H - hot start C - cold start
SN,N,S,G,C	LOGIC	RUN REQ.	Name of run request logical variable for procedure segment only.
C	COUNT	RATE	Numeric segment repetition rate in multiples of the time frame rate.
C	COUNT		Numeric segment priority number.
C	COUNT	EX.SEQ. FRAME OFFSET	Numeric segment execution time offset in multiples of time frame rate.
C	COUNT	CBL ROOM	Control block extra space (%)
C	COUNT	I/O ROOM	Percentage spare I/O control table room.
SN,N,G,C	LOGIC	SGABLE INABLE RNABLE OCTABLE } ***	Enable Flags (bits)

* Refer to Table 4-1 for form codes.
** If the revision level is set to #99 the segment will not be translated. This enables the user to specify which segments are (are not) to be translated for testing in a PMC.
*** Mandatory input.

Figure 3-1. Segment Identification

Within the segments of CBL, six different types of control blocks exist. They are:

Relay ladder blocks
Logic blocks
Regulating blocks
Procedural blocks
Pseudo blocks
Special blocks

The relay ladder blocks perform logic operations normally associated with sequencing. Pictorially, they are represented as strings of contacts.

The logic blocks perform the logic operations normally associated with true/false variables. They are pictorially represented as AND gates or flip-flops.

The regulating blocks perform the digital and analog functions associated with counters, shift registers, summing junctions and integrators.

Procedural blocks provide the ability to define algorithmic functions in a flow chart language. The program control functions provided are based on the highly structured forms of: DO-WHILE; IF-THEN and SEQUENCE, which embrace a single entry-single exit philosophy.

The pseudo blocks enable the user to define required constants and data arrays for use by the control segments.

The special blocks are used to perform those functions that do not conform to the previously defined types. Examples are data re-initialization and early exit program control.

The pseudo and special block types are permitted to be used in any segment type mixed with any other block type.

The CBL signal types and characteristics are as follows:

LOGICAL - True or False
COUNT - Numeric counts ranging from -32768 to 32767 (Integers)
ANALOG - High resolution values ranging from $\pm$ 1.7 x 10^{-38} to $\pm$ 1.7 x 10^{38} with a 6-digit resolution (floating point)
TEXT - 8 character (ASCII characters) groups

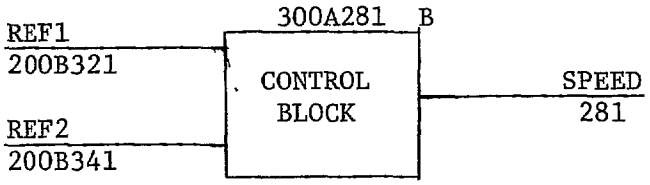
Most block output and input signals will be one of the above types. Other control block parameters include external I/O addresses and functional descriptors such as comparison or conversion codes.

The allowable forms of signals are as follows and will be detailed in a later section when appropriate. They are:

Signal Number (identification of block output signal)
Signal Name (optional functional name for above)
Scalar (single value for procedure block)
Vector (subscripted value from array block)
Constant (fixed value block input)
Global (pre-defined system value)

3-2 Continuous Segments

The PMC executes functions in the form of segments. Continuous segments are run on a repetitive basis and are comprised of a series of blocks, each of which performs a single identifiable function. Each block in the regulating and sequencing (logic category) may have several inputs to it but it can have only one output. This output variable will carry a drive, segment and block identification number and it will also be allowed to carry an alphabetic name of up to seven characters. Therefore, we could have a block defined as follows:



In the example above, the output of the block has a signal number 300A281 (combining drive, segment and block identification number). The output also has an optional signal name, SPEED.

From the header line, displayed with each block on the CRT, it can be seen that:

The block is in drive 300
It is in segment A
It is block 281
It will be drawn on sheet 300AB of the system elementary

Since there is always a one-to-one correspondence between outputs and blocks, the block iteself will be identified by the output identification number, i.e., 300A281. This is known as the signal number. Therefore, the generalized block identification scheme is:

DDD S NNN

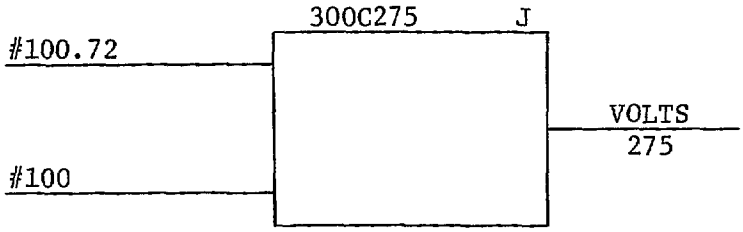
Where: DDD - Drive number (001-999)
S - Segment descriptor (A-Z, not I or O)
NNN - Block number (001-999)

The user needs only to supply the block number since the current drive number and segment descriptor will be automatically appended to achieve the final signal number.

In a continuous segment, the blocks are executed in the numerical sequence of their block numbers; hence, when assigning block numbers, enough room should be left to allow for future addition.

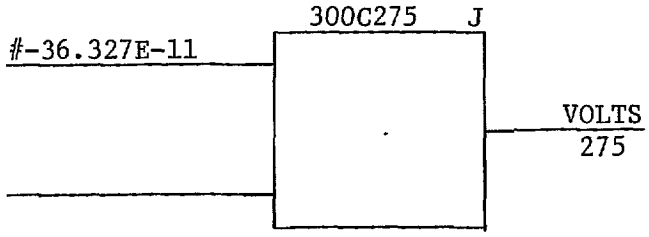
The inputs to regulating or logic blocks can be of several different types. First of all, we can introduce a constant value at the input to a block, for instance, in the case where a bias is required. All constants are preceded by a special symbol, #, in order to characterize them. They will be interpreted as analog or count values, as required by the control block input.

For example:



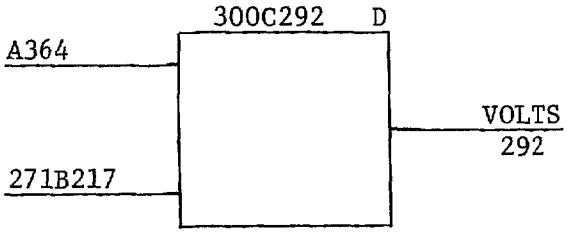
In the first case, an analog value of 100.72 was required whereas the second literal is a count value since it is an integer. The total field width for entering inputs is twelve characters and since this would be exceeded by very large or by very small analog numbers, it is permissible to enter them in scientific notation.

For example:



This gives five places of accuracy in the analog value. If the number was a very large one, the sign on the exponent may be eliminated which will imply a positive value. In this case, one more significant digit could be added.

Signal numbers can also be used as inputs to a block. The user must enter at least the segment descriptor and block number or the entire signal number. In the former case, the current drive number could be implied. For instance:



There is one further kind of signal naming convention which differs from what has been discussed so far. That is the case of an external signal. In the case of an input or output signal (i.e., one corresponding to an actual hardware address), the seven character name will be the same as its CSH hardware point address. For example, we might have the following block:



Where:
- C - Channel (J, K or L)
- S - Station number (0-F)
- GG - Group number (00-7F)
- P - Point number (0-F for individual bits and blank for the whole group. Add G if more than one bit and less than 16 are used.)

Notice that the input name is seven characters in length as allowed in all other symbolic signal names. The numbers within the name define the control hardware address of the signal and employ hexadecimal notation in some of the fields.

These internal hardware status variables will generally appear at the input to an analog or count input conversion block or at the output of an analog or count output conversion block. However, they may appear directly at the input of a control block if no conversion is necessary. This would occur fairly frequently in the case of the logic type blocks where only a single bit in an input group is used as a logic variable.

All of the signals within a segment, whether they are inputs from the hardware, outputs to hardware or internal intermediate results, will have space allocated for them and will be manipulated according to the type of variable.

Some mention has already been made of the alphabetic name which can be assigned to a signal. If a name is chosen such that the number of characters is five or greater, then this name shall be deemed a Global signal which can only refer to a single location in the PMC memory.

They are, therefore, unique to any one PMC. For example, the names:

SPEED
VOLTS
CURRENT

The names are global in nature and reference made to them from any segment will affect a single signal location. If, however, a name is chosen such that the number of characters is four or less, then this name will be regarded as a local variable within a drive. Thus, the names:

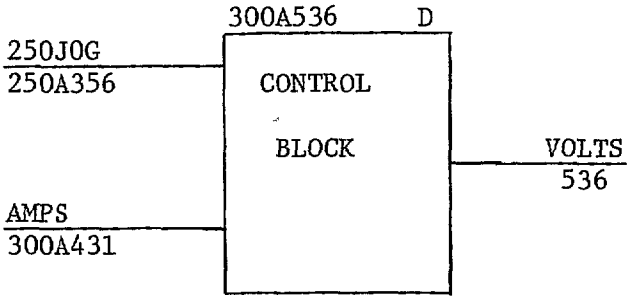
AMPS
REF1
JOG

are local to a drive and can be used in another drive to create other signal locations. This feat is accomplished by the fact that when a name of four or less characters is used, the drive number is implied to be a prefix to the name. In the above example, these names would actually be:

300AMPS
200REF1
281JOJ

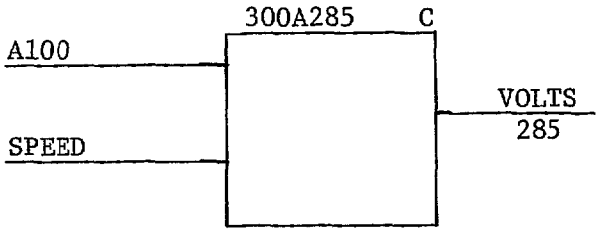
The drive number, however, will be appended by the system to keep references straight. This means that we can use these names in various segments as long as they are in different drives. We cannot use the name AMPS in both segments A and B of drive 300 and expect these to be different signals but we could do this by using it again in a different drive.

If an individual wishes to reference a variable from a different drive which has a name of four characters or less, then the drive number would have to be appended to the name. For example, if we were operating in Drive 300, we could have a control block as follows:



The name 250J0G required the drive number prefix to distinguish it from a variable named J0G which may exist in Drive 300.

A signal name must contain all alphabetic characters or at least two initial alphabetic characters if numerics are used in it. This is necessary to prevent confusion between a signal name and a signal number. For example:



The A100 refers to a signal number of 300A100 (i.e., implied drive prefix).

If A100 were an allowable signal name, we could not distinguish between it and the output of the block with that signal number. Hence, if numbers are used in a signal name, then at least two initial alphabetic characters are required. (i.e., AA100 would be acceptable).

3-3 Initialize Segments

The initialize segment is executed only when the PMC has been initialized or there has been a power failure. PMC initialization occurs when the mode switch is changed from OFF-LINE to STBY. These initialize segments are executed only once, after which they are disabled from being scheduled for execution. The user should use these segments to initialize data and outputs in the event that a controller is initialized or when there has been a power failure. For this reason, these segments normally contain only initialize blocks (i.e., INZS or INZV) and output blocks (i.e., COCB and AOCB) but can contain any of the regulating, sequencing or pseudo blocks if so desired.

The user should assign the highest priority levels in the entire system to these initialize segments. In this way, they will be the first to execute when the PMC resumes processing. In this way, the user may set internal variables and outputs to a known condition before resuming normal execution. These segments have been further subdivided into two classes as follows:

(1) Hot start initialize segment

(2) Cold start initialize segment

The hot start initialze segments are the only ones scheduled for execution after a power failure having a duration less than a user-defined threshold. For a power failure greater than the threshold or a controller initialize, then all initialize segments are scheduled for execution (i.e., this is a cold start).

3-4 Interrupt Segments

There is an interrupt segment comprised of regulating and sequencing control blocks. It will only run in response to a request which must be transmitted by the control signal highway. If such a request is made to the PMC, this segment will take precedence over all others including the continuous segments to be executed within that time frame. The PMC will cause the interrupt segment to be the next one executed. This segment may cause some procedural segments to be scheduled but they will not begin until all of the continuous segments are complete.

Interrupt segments are created in the same manner as continuous segments, but do not have a defined repetitive rate or priority. Any of the blocks permitted in a continuous segment may be used.

3-5 Procedural Segments

The procedural segments may contain only the procedural block type and optionally the pseudo or special blocks. Pictorially, the blocks appear as flow chart elements with single-entry/single-exit flow lines. Procedural blocks are functionally grouped as follows:

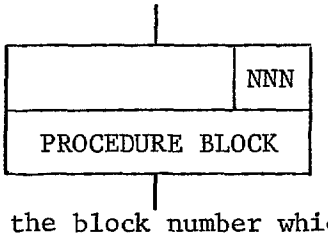
Function block

Decision block

Repetition block

In the procedural type segments, the blocks are named exactly the same as they are in the regulating or sequencing segments. That is, there is a unique drive/segment/block identification number for each procedural control block. However, the outputs or resulting actions of the procedural blocks no longer correspond to a certain variable or signal name.

For example:



Where: NNN is the block number which has an implied prefix of the drive and segment descriptor.

Procedural blocks perform the function of doing more complex mathematical calculations and table or list manipulations. Each procedural block no longer corresponds to a control function which might have been implemented in hardware, as was the case with the regulating blocks. Here, the user can initiate other segments and he is able to delay or suspend the execution of a segment. Iterative type calculations can be performed, conditional branching can take place and generally more complex problems can be solved.

Procedural segments are not executed on a fixed cyclic basis and will not receive the priority attention that continuous segments receive from the control executive. For these reasons, it would be unwise to perform any time-critical calculations in a procedural segment.

Typical applications of procedural segments are:

Mill set-up calculations
Furnace temperature control setpoint calculations
Model adaption calculations
Logging functions
Primary data input (PDI)

The structured format of the procedural CBL is based on the well defined constructs of the linear sequence, if-then-else, and do-while. Implementing functions with procedural CBL requires the same top-down programming and modularization techniques that are features of the structured programming discipline.

Defined sub-modules begin and end with the PSUBM and PRETN blocks. Program logic control is implemented by selectively invoking sub-modules using the PTEST, PDOWL, PDOTL or PCALL types of blocks. Major segment starting and ending points are defined by the PMAIN and PMEND blocks.

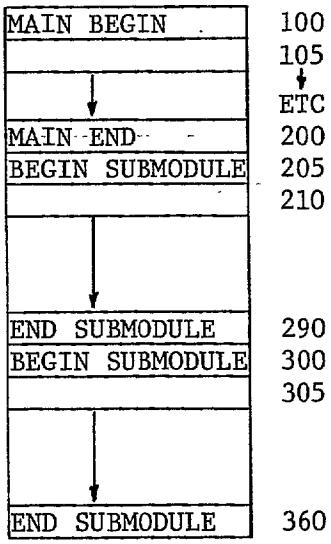
The signal forms of procedure blocks are different from continuous blocks. Function results of these blocks are classes as scalar or vector when they are single item or array item, respectively. Input data is the same except no CSH addresses are allowed.

Vectors with variable subscripts are allowed but the vector name must not exceed the allowable 7 characters. The sub-scripts may be constants or count signals (scalar or signal name) where the count signal name must be two alphabetic characters.

The value within the count signal must never exceed 99 since this is the maximum array size. If two vectors are subscripted, with variable subscripts within the same block, the subscripts must be identical.

A procedural segment must be composed of a main module and optionally several sub-modules which are called as required from the main module. Execution within a procedural module will be done sequentially according to block numbers and all block numbers for sub-modules must be greater than those employed in the main program. If any blocks are to be added using a maintenance function, these must be block numbers available for the addition.

For example, it is permissible to add a block number 152 between blocks numbered 150 and 155; however, no block can be inserted between blocks numbered 171 and 172. For this reason, the block numbers should be chosen so that insertions can be made. A typical procedural segment may appear as follows:



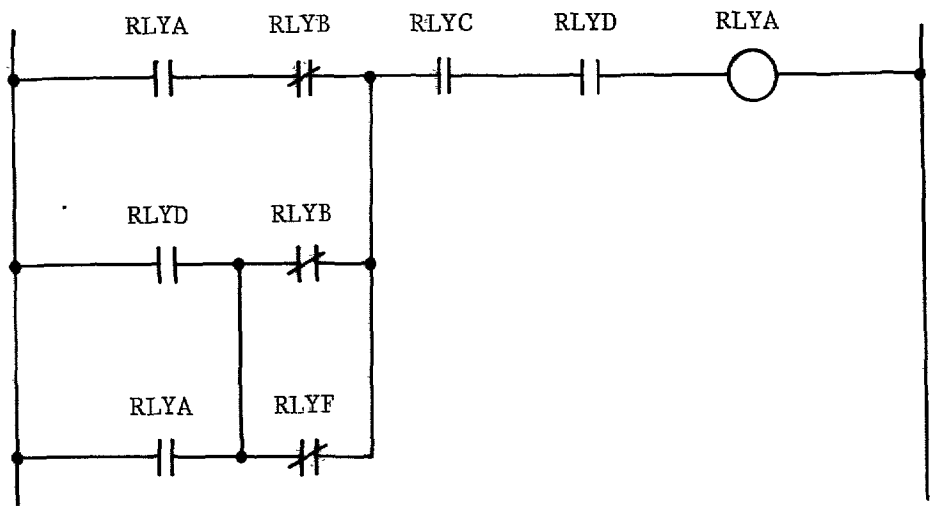
3-6 CONTROL BLOCKS

3-7 Relay Ladder Blocks

Within a segment, sequencing blocks may consist of logic blocks, logic and regulating blocks, relay ladder blocks, special blocks or pseudo blocks. Relay ladder blocks emulate actual hardware relays. Sequencing functions can be accomplished using either a relay blocks or a logic block. The block actually used is the programmer's option and inclination. Some advantages of the relay ladder block are as follows:

(1) More inputs per blocks
(2) More easily understood functionally
(3) Can be displayed on PMC's IMM console
(4) Easier to modify

The following diagram illustrates the format of one type of relay ladder block:

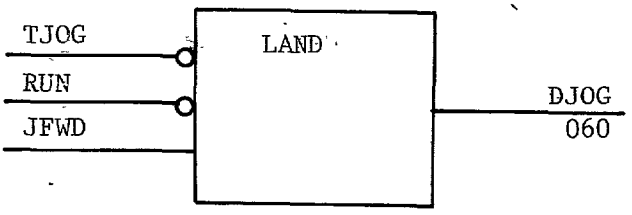


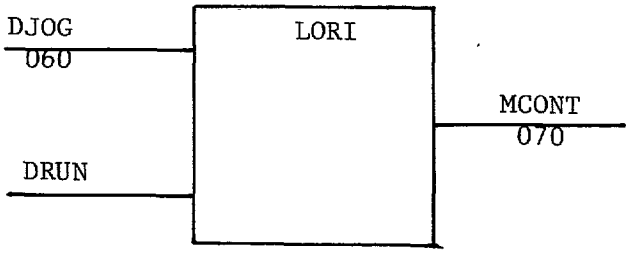
3-8 Logic Blocks

The logic blocks are characterized as those whose inputs and outputs consist of true/false logic signals. Some logic blocks have an invert option on inputs to provide an inverse sense signal to the block if desired. The most used logic blocks are the logical and (LAND), the logical or inclusive (LORI) and the logical Boolean operator (LGIC). Within a segment, sequencing blocks may consist of logic blocks, logic and regulating blocks, relay ladder blocks, special blocks or pseudo blocks. Sequencing functions can be accomplished using either a relay block or logic block. The block actually used is the programmer's option and inclination. Some advantages of a logic block are as follows:

(1) Inputs may be inverted
(2) Simulates solid state logic gate technology
(3) Some logic blocks have a "disable execution" capability.

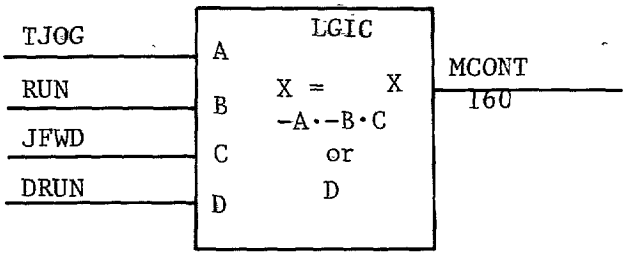
The following examples illustrate logic block usage.





The logical signal output of block number 060 is true when TJOG = FALSE and RUN = FALSE and JFWD = TRUE. Block number 070 is processed after 060 and its output is true when DJOG = TRUE or DRUN = TRUE. The small circle (o) adjacent to the inputs of LAND indicate that the input signal sense is to be inverted before being used by the block.

For applications where logic manipulators are preferably expressed as Boolean equations, the Logic Calculator block (LGIC) is provided. The following example illustrates its use.



This is the same function depicted in the LAND/LORI example. Only one block is required but has the disadvantage of not providing the intermediate signal DJOG.

3-9 Regulating Blocks

Within the regulating type of control blocks, there are two sub-groups known as Analog and Count.

The primary distinction between the signals in these sub-groups is that analog blocks generally have analog signals as inputs and outputs, whereas count blocks have count signals as inputs and outputs.

Analog and count blocks may be mixed freely within a segment along with logic blocks. For this reason, some attention must be given to compatibility of signal types. Output signals must be of the same type as the input signal of the block to which they are connected.

Since the analog and count signals are stored in different numeric formats within the PMC (see Appendix B), some method must be provided to convert analog to count and count to analog. The analog converter (ACNV) produces analog from several different formats. The count converter (CCNV) performs a similar function for count signals.

Provision is also made with CBL blocks to convert input data from any of the commonly used formats in the regulating hardware (part of CSH) to either analog or count signals as required in the PMC.

For example, inputs could be represented externally as follows:

16 Bit Binary
4 Digit BCD
12 Bit Binary (A/D Cards)

In all cases, the input could be converted to either a count signal or an analog signal.

Similarly, outputs can be represented externally as follows:

16 Bit Binary
4 Digit BCD
12 Bit Binary (D/A Cards)

Conversion from internal format, either analog or count, to the required external format is performed by the AOCB and COCB blocks.

3-10 Pseudo and Special Control Blocks

Pseudo blocks are used to define arrays and constants. They do not occupy any space in the PMC as control block macros. They only set up information at translation time. They are not executed by the PMC.

Special blocks perform functions that are not of the sequencing, regulating, procedural or pseudo type. These blocks initialize data, transfer CSH data, control program flow and define reusable sequences. Special blocks have control block macros and are executed like regulating/sequencing blocks in the PMC.

Both the pseudo and special types of control blocks can be used in any type of segment, including procedural segments.

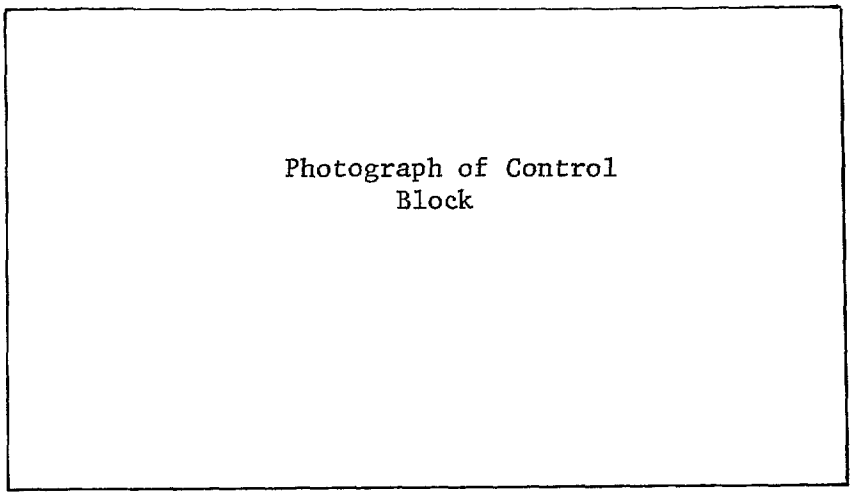
NOTES

SECTION 4

CONTROL BLOCK REFERENCE SHEETS

4-1 CONTROL BLOCK DESCRIPTION SUMMARY

The format of the control block reference sheets presented in this section is as follows:



1. FUNCTION
2. PARAMETERS
3. SPECIAL RESTRICTIONS
4. CBL MACRO INSTRUCTION FORMAT
5. ADDITIONAL INFORMATION

The photograph shows how the control block appears on the FSS CRT screen.

The function (1) usually provides a formula and a description of the control block operation.

The parameters (2) use FORM CODES (described in Table 4-1) which identify the types of signals in the INPUT, DESCRIPTOR and OUTPUT fields of the control block. The VARIABLE TYPE signal and DEFAULT conditions are also shown in the parameters. An enable variable appears on many control blocks. If the specified enable variable is not applied to the block, the block defaults to an enabled condition. If an enable variable is applied, its value (true or false) determines whether the control block is enabled or disabled.

The special restrictions (3) normally describe the DEFAULT conditions.

The CBL macro instruction format (4) is the translated object for each type of control block. This is how the control block appears as it resides in the PMC. Table 4-2 describes the macro descriptors. These descriptors summarize the signal types and names with associated symbols used in the FORMS column of the control block reference sheets.

Table 4-1. Control Block Signal Descriptors

FORM CODE	TYPE SIGNAL	EXAMPLE	DESCRIPTION
(C)	Constants (literals)	#100.7	Analog number
		#100	Count number
		#22.76E4	Analog number in scientific notation
		T or F	Logic true or false
(S)	Scalar	#SPEED	Data value
(V)	Vector	TORQUE(32)	Element of data array
		TORQUE(IX)	
		TORQUE(#IJ)	
(N)	Signal names	REF12	Name with numeric (0 thru 9) and alphabetic characters (A thru Z).
		VOLTS	Name with alphabetic characters only.
		AMP	Local variable name
		281AMP	Specifying drive on local variable
(A)	Address (CSH)	/J34F/6	CSH hardware address signal name
(SN)	Signal numbers	300A275	Full signal name
		C365	Signal name with implied drive
(G)	Globals	&T	System defined value of logical true
		&PI	System defined value of PI
(OC)	Operator Code	=>	Equal to or greater than,
		=<	Equal to or less than,
		+	Or
		*	And
(BN)	Block Number	#286	Range 001 to 999
(AN)	Array Name	TABLE	Non-subscripted array name defined by DATV or INZV or PDATV.

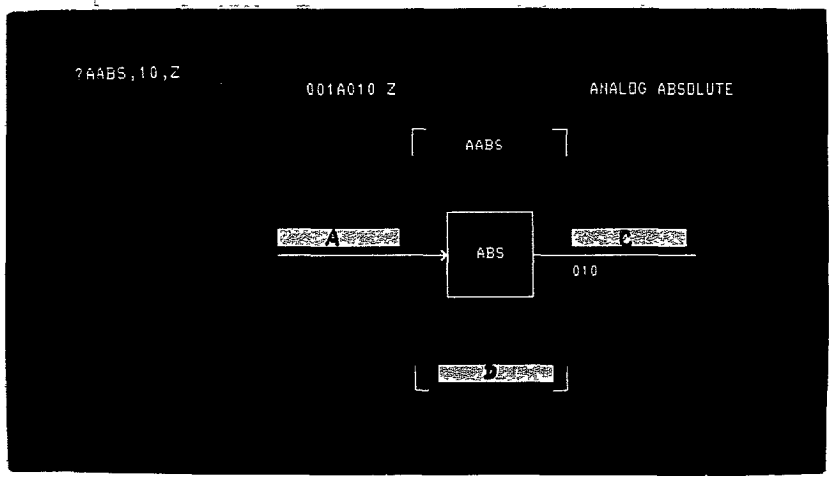
NOTES

Table 4-2. Macro Descriptors

DESCRIPTOR	MEANING
M	Bit indicating that the control block macro has been modified in the PMC. (1 bit)
BLOCK SIZE	Number of words in the macro (5 bits).
BLOCK SEQUENCE NUMBER	The number of the block as it was added to the control system. e.g., block 100A010 indicates sequence number 10. (10 bits).
OPERATION CODE	Unique fixed number for each type of control block.
POINTER TO ...	Address in memory.
NEXT CBL OP. CODE	Preceded by "POINTER TO" indicates the addresses of the next control block to be executed in the PMC.
OP	Operation code that defines the rules for combining two logical variables. (4 bits).

AABS

ANALOG ABSOLUTE VALUE



1. FUNCTION:

$C = \sqrt{A^2}$ if D is true (logic 1) or not entered

Analog output C is the absolute value of the analog input A.

2. PARAMETERS:

IDENTIFICATION	FORMS	VARIABLE TYPES	DEFAULT
INPUT			
A	SN, N, S, V, C, G	ANALOG	①
DESCRIPTOR			
D	SN, N, S, V, C, G, A	LOGIC	ENABLED ②
OUTPUT			
C	SN, N	ANALOG	-

3. SPECIAL RESTRICTIONS:

① Required input

② Optional input

4. CBL MACRO INSTRUCTION FORMAT:

Type ANALOG Op. Code 8B80 Name ABSOLUTE

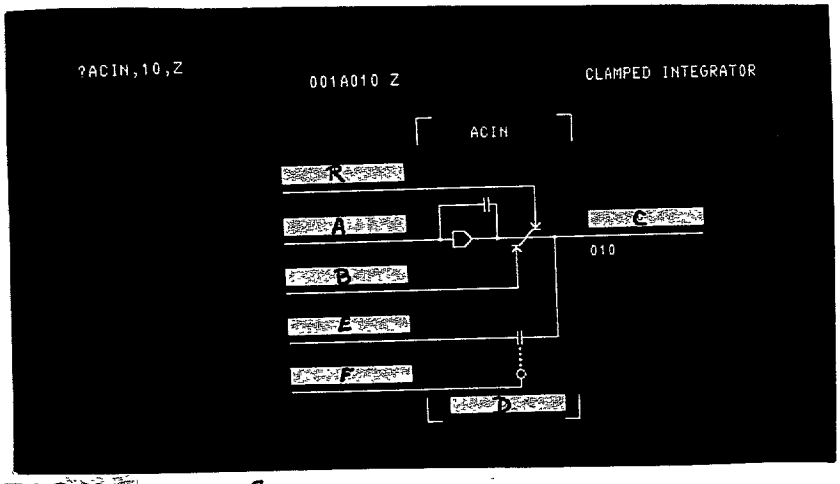
POS			
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER
	OPERATION CODE		
	POINTER TO NEXT CBL OP. CODE		
1	POINTER TO INPUT DATA "A"		
-2	POINTER TO OUTPUT DATA "C"		

5. ADDITIONAL INFORMATION:

None

ACIN

ANALOG CLAMPED INTEGRATOR



1. FUNCTION:

C = $\int$A dt if F false

C RANGE: R ≥ C ≥ B

OR

C = E if F true

Analog output C is the time integral of analog input A limited to R ≥ C ≥ B if F is false; otherwise, if F true, C = E (initialization value).

2. PARAMETERS:

IDENTIFICATION	FORMS	VARIABLE TYPE	DEFAULT
INPUT			
A	(INPUT) SN,N,S,V,C	ANALOG	①
B	(LOW LIMIT) SN,N,S,V,C,G	ANALOG	-1.7E38
E	(INITIAL VALUE) SN,N,S,V,C,G	ANALOG	0.0
F	(INITIALIZE BIT) SN,N,S,V,C,G	LOGIC	FALSE
R	(HIGH LIMIT) SN,N,S,V,C,G	ANALOG	1.7E38
DESCRIPTOR			
D	SN,N,S,V,C,G,A	LOGIC	ENABLE ②
OUTPUT			
C	SN,N	ANALOG	

3. SPECIAL RESTRICTIONS:

① Required input

② Optional input

4. CBL MACRO INSTRUCTION FORMAT:

Type ANALOG Op. Code 8B81 Name CLAMPED INTEGRATOR

POS			
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER
	OPERATION CODE		
	POINTER TO NEXT CBL OP. CODE		
1	POINTER TO FIRST TIME INDICATOR		*
2	POINTER TO INITIALIZE LOGICAL F		
3	POINTER TO INITIAL VALUE E		
4	POINTER TO INPUT DATA A		
5	POINTER TO SEGMENT INTERVAL		**
6	POINTER TO INTERMEDIATE VALUE		***
7	POINTER TO HIGH CLAMP R		
8	POINTER TO LOW CLAMP B		
9	POINTER TO OUTPUT VALUE C		

* First time indicator is a count variable in the segment status block (SSB).

** Segment interval is an analog variable in segment status block which contains the segment executation rate in seconds.

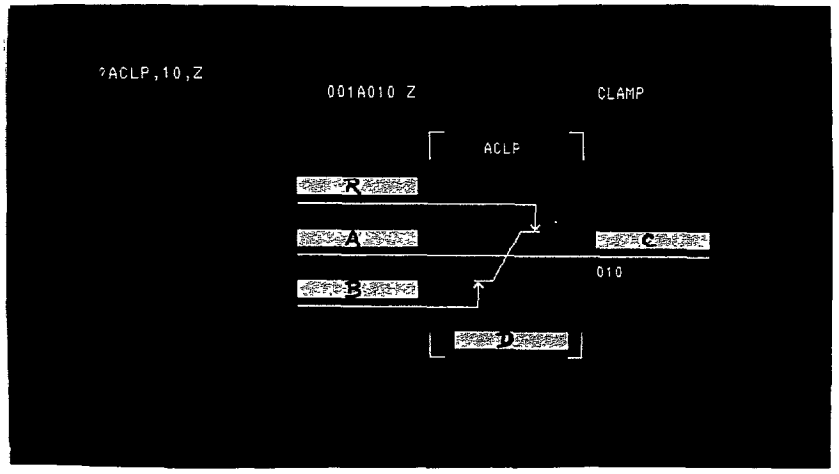
*** Intermediate value is an analog variable allocated by the translator.

5. ADDITIONAL INFORMATION:

A flow chart appears in Appendix D.

ACLP

ANALOG CLAMP



1. FUNCTION:

C = A Within limits
R ≥ C ≥ B

Analog output C is equal to analog input A within limits B to R (analog values). Otherwise C = R when A > R and C = B when A < B. All provided D is true (1). Otherwise, output C is frozen at last analog value if D is false (0).

2. PARAMETERS:

IDENTIFICATION	FORMS	VARIABLE TYPE	DEFAULT
INPUT			
A	(INPUT) SN,N,S,V,C,G	ANALOG	①
B	(LOW LIMIT) SN,N,S,C,G	ANALOG	-1.7E38
R	(HIGH LIMIT) SN,N,S,C,G	ANALOG	1.7E38
DESCRIPTOR			
D	SN,N,S,V,C,G,A	LOGIC	ENABLED ②
OUTPUT			
C	SN,S	ANALOG	-

3. SPECIAL RESTRICTIONS:

High limit must be greater than low limit

① Required input

② Optional input

4. CBL MACRO INSTRUCTION FORMAT:

Type ANALOG Op. Code 8B82 Name CLAMP

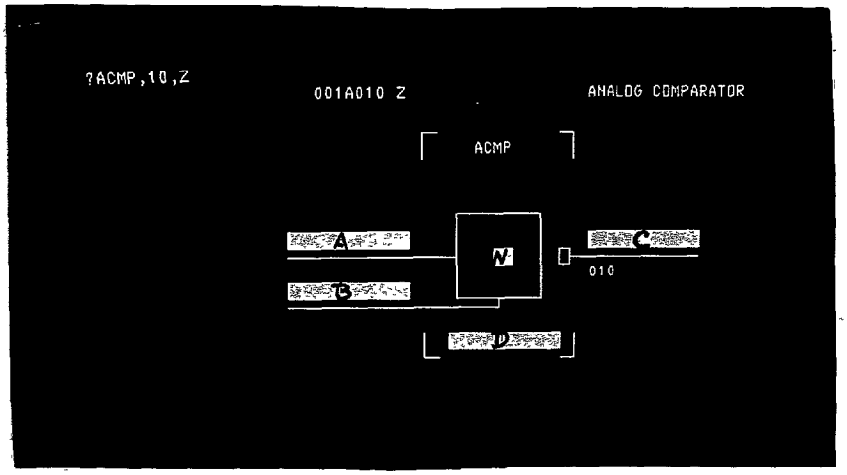
POS	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER
	OPERATION CODE		
	POINTER TO NEXT CBL OP. CODE		
1	POINTER TO INPUT VALUE A		
2	POINTER TO HIGH CLAMP R		
3	POINTER TO LOW CLAMP B		
4	POINTER TO OUTPUT VALUE C		

5. ADDITIONAL INFORMATION:

A flow chart appears in Appendix D.

ACMP

ANALOG COMPARE



1. FUNCTION:

A :: B If true, C = true
Otherwise, C = false

Cases 1) = (equal to)
2)< >(≠) (not equal to)
3)< (less than)
4) >(greater than)
5)>= (greater than or equal to)
6)<= (less than or equal to)

2 characters wide define "N"

If comparison (1 or 6 cases) of A and B signals is true (and D is enabled), C = true. Otherwise (if D is enabled) C = false. A and B are both analog signals. If D is false, C is frozen.

2. PARAMETERS:

IDENTIFICATION	FORMS	VARIABLE TYPE	DEFAULT
INPUT			
A	SN,N,S,V,C,G	ANALOG	①
B	SN,N,S,V,C,G	ANALOG	①
DESCRIPTOR			
D	SN,N,S,V,C,G,A	LOGIC	ENABLED ②
N (COMPARATOR)	OC	-	①
OUTPUT			
C	SN,N,A	LOGIC	-

3. SPECIAL RESTRICTIONS:

Use "=" operator with caution. Exact match between inputs A and B is required for true output.

① Required input

② Optional input

4. CBL MACRO INSTRUCTION FORMAT:

Type ANALOG Op. Code 8B83 Name ANALOG COMPARATOR

POS	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER	
	OPERATION CODE			
	POINTER TO NEXT CBL OP. CODE			
1	0	0	0 CODE	*
2	POINTER TO INPUT VALUE A			
3	POINTER TO INPUT VALUE B			
4	POINTER TO OUTPUT LOGICAL C			

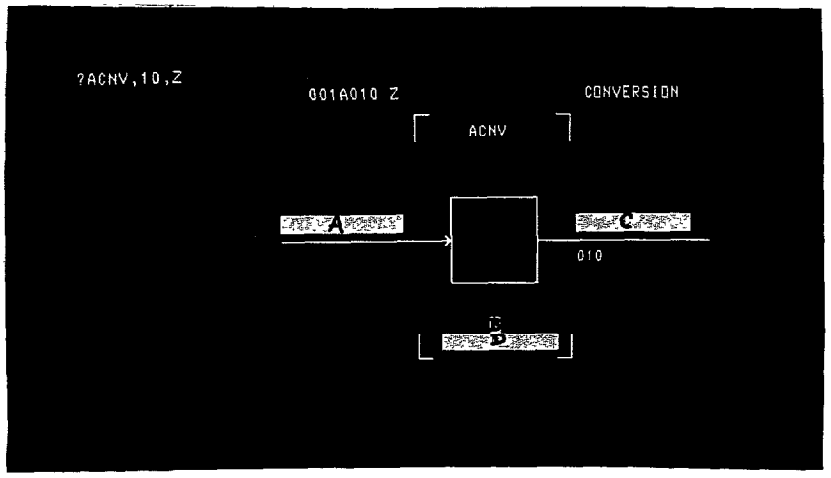
* Code:
0 → Equal
1 → Not equal
2 → Less than
3 → Greater than
4 → Greater than or equal
5 → Less than or equal

5. ADDITIONAL INFORMATION:

None

ACNV

ANALOG CONVERSION



1. FUNCTION:

Input count value or 4 decade B.C.D. value A (both 16 bits) is converted to analog value C if if D flag is true; otherwise, C is frozen to its last enabled or initialized value.

CODE B: 0:C = A (Count) 16 bit (2's complement)
1:C = A (4 decade B.C.D)

2. PARAMETERS:

IDENTIFICATION	FORMS	VARIABLE TYPE	DEFAULT
INPUT			
A	SN,N,S,V,C,G	COUNT	①
DESCRIPTOR			
B (Conversion Code)	OC	-	①
D	SN,N,S,V,C,G	LOGIC	ENABLED ②
OUTPUT			
C	SN,N	ANALOG	-

3. SPECIAL RESTRICTIONS:

Output value is limited to 0000.0 through 9999.0 (BCD) and -32768 to +32767 for count.

① Required input

② Optional input

ACNV

4. CBL MACRO INSTRUCTION FORMAT:

Type ANALOG Op. Code 8B84 Name ANALOG CONVERSION

POS					
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER		
	OPERATION CODE				
	POINTER TO NEXT CBL OP. CODE				
1	0	0	0	CODE	*
2	POINTER TO INPUT DATA A				
3	POINTER TO OUTPUT VALUE C				

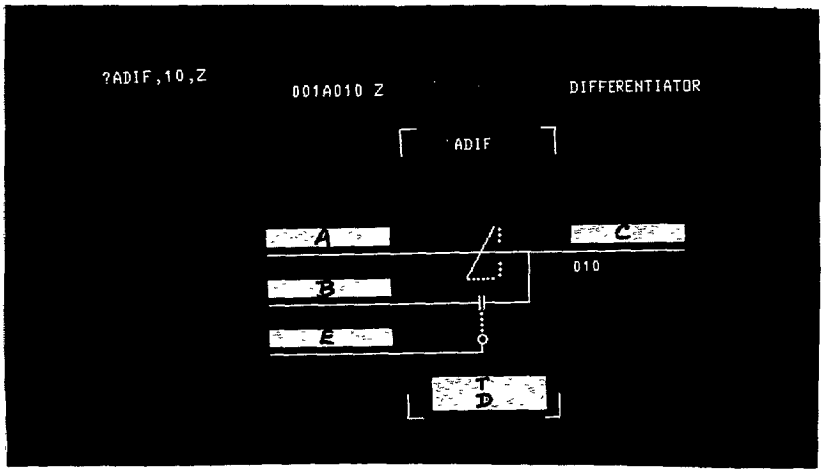
* Code: 0 ⟶ Count to analog
1 ⟶ BCD to analog

5. ADDITIONAL INFORMATION:

None

ADIF

ANALOG DIFFERENTIATOR



1. FUNCTION:

If D is true { For E false, $C = \frac{da}{dt}$; For E true, C = B (initial value)

If D is false, C = frozen at last enabled value.

The analog output C is the time derivative of analog input A unless E is true, in which case output C is clamped to initial value B. T is filter or lag time constant in seconds.

2. PARAMETERS:

IDENTIFICATION INPUT	FORMS	VARIABLE TYPE	DEFAULT
A	(INPUT) SN,N,S,V,C,G	ANALOG	①
B	(INITIAL VALUE) SN,N,S,V,C,G	ANALOG	0.0
E	(INITIALIZE FLAG) SN,N,S,V,C,G	LOGIC	FALSE
DESCRIPTOR			
D	SN,N,S,V,C,G,A	LOGIC	ENABLED ②
T	(LAG TIME CONSTANT) C	ANALOG	0.0
OUTPUT			
C	(OUTPUT) SN,N	ANALOG	-

3. SPECIAL RESTRICTIONS:

① Required input

② Optional input

4. CBL MACRO INSTRUCTION FORMAT:

Type ANALOG Op. Code 8B85 Name DIFFERENTIATOR

POS	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER	
	OPERATION CODE			
	POINTER TO NEXT CBL OP. CODE			
1	POINTER TO FIRST TIME INDICATOR			*
2	POINTER TO INITIALIZE FLAG E			
3	POINTER TO INPUT VALUE B			
4	POINTER TO INPUT VALUE A			
5	POINTER TO INTERMEDIATE VALUE			**
6	POINTER TO SEGMENT INTERVAL			***
7	POINTER TO LAG CONSTANT T			
8	POINTER TO OUTPUT C			

* First time indicator is count variable in segment status block (SSB).

** Intermediate value is analog variable allocated by the translator.

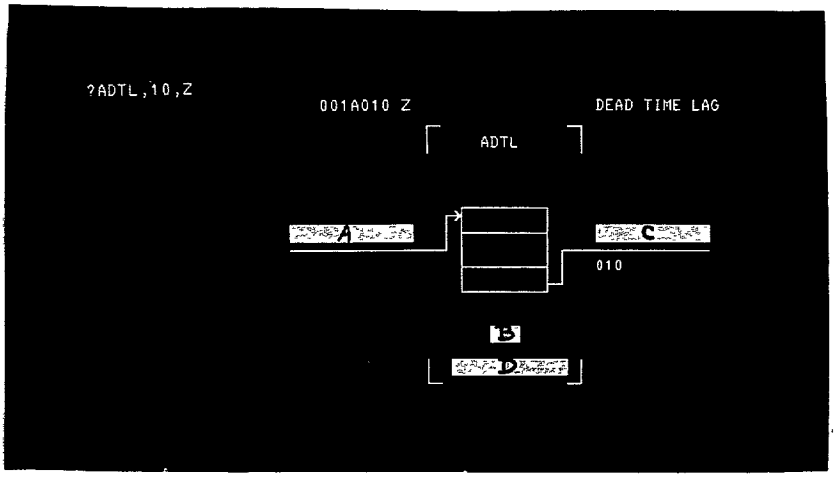
*** Segment interval is analog variable in SSB which contains seconds between start of segment.

5. ADDITIONAL INFORMATION:

A flow chart appears in Appendix D.

ADTL

ANALOG DEAD TIME LIST



1. FUNCTION:

The analog output C is taken from a "first in-first out" list (the oldest element). Count value B (max. 99) is list length.

At initialization: Entire list is set to input A value and (internal) pointers for A entries and C extractions a set up.

2. PARAMETERS:

IDENTIFICATION		VARIABLE	
INPUT	FORMS	TYPE	DEFAULT
A	SN,N,S,V,C,G	ANALOG	①
DESCRIPTOR			
B (LIST LENGTH)	C	COUNT	①
D	SN,N,S,V,C,G,A	LOGIC	ENABLED ②
OUTPUT			
C	SN,N	ANALOG	

3. SPECIAL RESTRICTIONS:

Maximum list length (B) is 99.

① Required input

② Optional input

4. CBL MACRO INSTRUCTION FORMAT:

Type ANALOG Op Code 8B86 Name DEAD TIME LIST

POS			
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER
	OPERATION CODE		
	POINTER TO NEXT CBL OP. CODE		
1	POINTER TO FIRST TIME INDICATOR		*
2	POINTER TO LIST		**
3	POINTER TO OUTPUT DATA C		
4	POINTER TO INPUT DATA A		

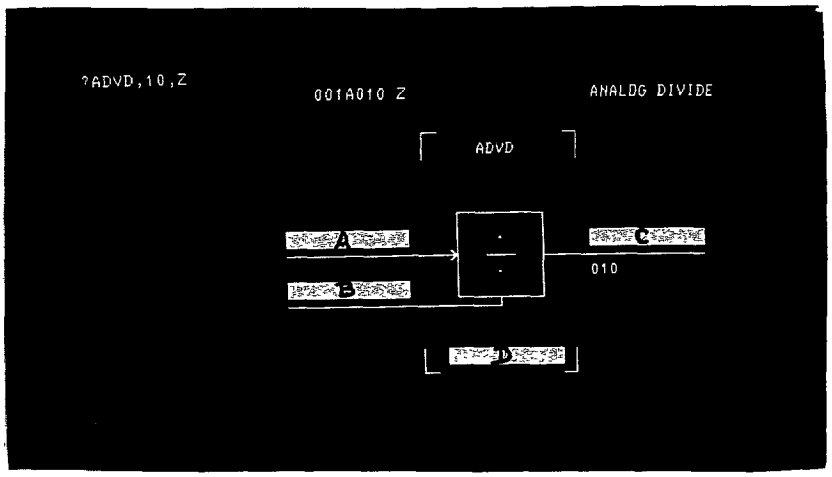
* First time indicator is a count variable in the segment status block (SSB).
** List is an intermediate array, B words long.

5. ADDITIONAL INFORMATION:

None

ADVD

ANALOG DIVIDE



1. FUNCTION:

$C = \frac{A}{B}$ if D is true; otherwise C is frozen.

Analog output C = analog input A divided by analog input B.

2. PARAMETERS:

IDENTIFICATION INPUT	FORMS	VARIABLE TYPE	DEFAULT
A	SN,N,S,V,C,G	ANALOG	①
B	SN,N,S,V,C,G	ANALOG	①
DESCRIPTOR			
D	SN,N,S,V,C,G,A	LOGIC	ENABLED ②
OUTPUT			
C	SN,N	ANALOG	-

3. SPECIAL RESTRICTIONS:

Division by zero is permitted; the result, however, is the largest possible number.

① Required input

② Optional input

4. CBL MACRO INSTRUCTION FORMAT:

Type ANALOG Op Cod 8B87 Name ANALOG DIVIDE

POS					
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER		
	OPERATION CODE				
	POINTER TO NEXT CBL OP. CODE				
1	1	4	0	0	*
2	POINTER TO INPUT A				
3	POINTER TO INPUT B				
4	POINTER TO OUTPUT C				

* Fixed set of operator codes (Used also by ASUM and PCALC.)

Operators

0 ⟶ Store result

1 ⟶ Add

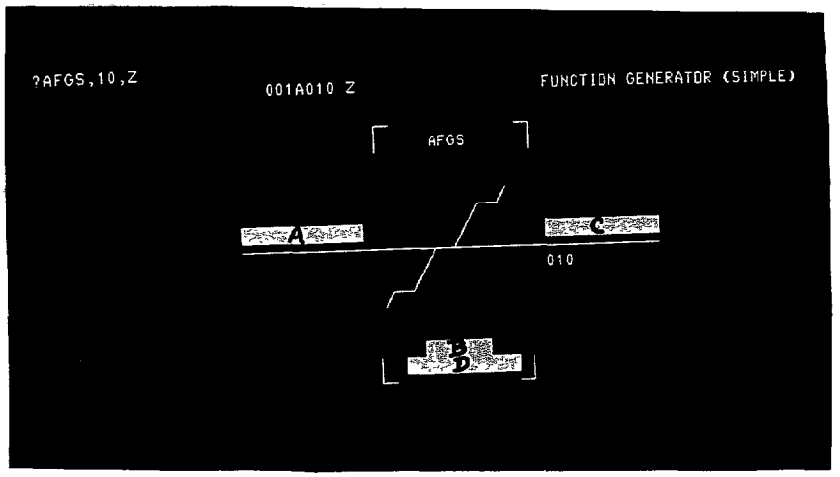
4 ⟶ Divide

5. ADDITIONAL INFORMATION:

None

AFGS

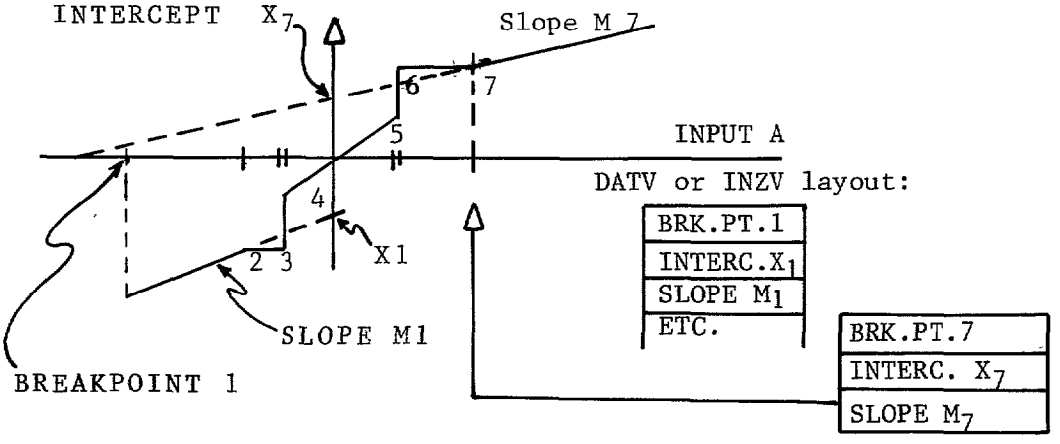
ANALOG FUNCTION GENERATOR (SIMPLE)



1. FUNCTION: C = FUNCTION (A) BREAK POINT / INTERCEPT / SLOPE

B = Name of data block table (DATV or INZV) containing intercepts and slopes

The analog output C is calculated in straight-line segment fashion from the input A using one of seven linear equations $C = X_N + M_N * INPUT$.



2. PARAMETERS:

AFGS

IDENTIFICATION INPUT	FORMS	VARIABLE TYPE	DEFAULT
A	SN,N,S,V,C,G	ANALOG	①
B	AN	ANALOG	①
D	SN,N,S,V,C,G,A	LOGIC	ENABLED ②
OUTPUTS			
C	SN,N	ANALOG	-

3. SPECIAL RESTRICTIONS:

① Required input

② Optional input

4. CBL MACRO INSTRUCTION FORMAT:

Type ANALOG Op Code 8B88 Name FUNCTION GENERATOR (SIMPLE)

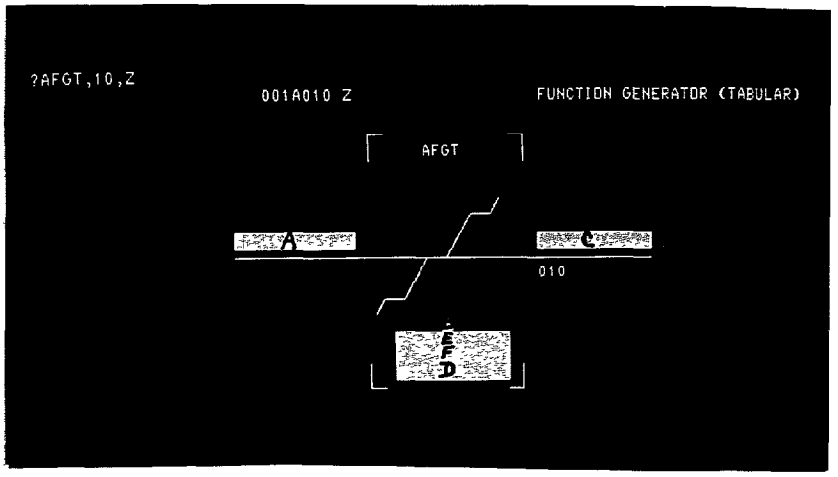
POS			
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER
	OPERATION CODE		
	POINTER TO NEXT CBL OP. CODE		
1	POINTER TO INPUT DATA A		
2	POINTER TO INPUT VECTOR B		
3	POINTER TO OUTPUT VALUE C		

5. ADDITIONAL INFORMATION:

None

AFGT

ANALOG FUNCTION GENERATOR (TABULAR)



1. FUNCTION: Two tables are used, and if B = 0: analog ouptut C is derived from a linear interpolation:

$$C = \frac{X_N - INPUT}{X_N - X_N - 1} * (Y_N - Y_N - 1) + Y_N - 1$$

Tables named by E and F are monotonically increasing in values

$N \leq 90$ 6 - DATV tables for both E & F. Each DATV can have 15 entries.

Value X_N is found in table E and C is then interpolated from corresponding positions in table F, values Y_N. If B = 1, no linear interpolation is performed:

$C = Y_N - 1$ for input

$X_N > A \geq X_N - 1$

2. PARAMETERS:

IDENTIFICATION INPUT	FORMS	VARIABLE TYPE	DEFAULT
A	SN,N,S,V,C,G	ANALOG	①
DESCRIPTOR			
E (TABLE)	AN	ANALOG	①
F (TABLE)	AN	ANALOG	①
B (CODE)	OC	-	②
D	SN,N,S,V,C,G,A	LOGIC	ENABLED ②
OUTPUTS			
C	SN,N	ANALOG	-

3. SPECIAL RESTRICTIONS:

Vectors E and F must be defined using DATV or INZV blocks and must be the same length (i.e., contain the same number of elements).

① Required input

② Limited to Ø or 1 *

③ Optional input

4. CBL MACRO INSTRUCTION FORMAT:

Type ANALOG Op Code 8B89 Name FUNCTION GENERATOR (TABULAR)

POS					
	M	BLOCK SIZE		BLOCK SEQUENCE NUMBER	
	OPERATION CODE				
	POINTER TO NEXT CBL OP. CODE				
1	POINTER TO INPUT VALUE A				
2	POINTER TO VECTOR E				
3	0	0	0	CODE B	*
4	POINTER TO VECTOR F				
5	POINTER TO OUTPUT VALUE C				

* Interpolation Code B

0 — Do linear interpretation

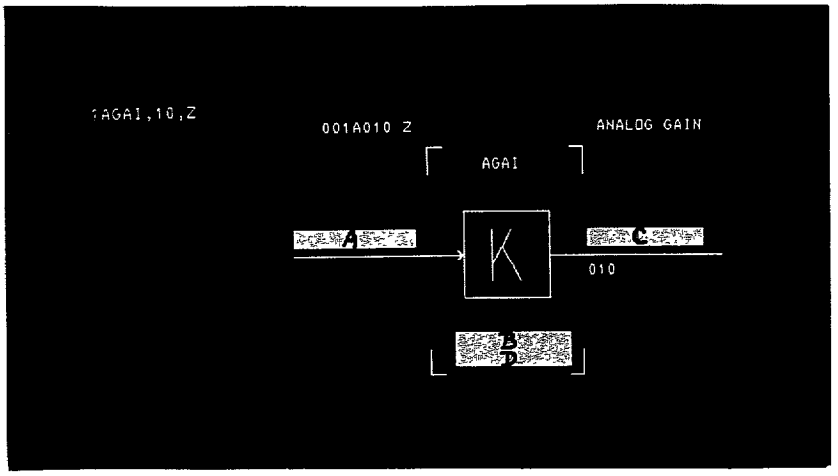
1 — Skip linear interpretation

5. ADDITIONAL INFORMATION:

None

AGAI

ANALOG GAIN



1. FUNCTION:

C = B * A

Analog output C is equal to analog input A multiplied by constant gain B. Overflow results with C = the largest possible signed number.

2. PARAMETERS:

IDENTIFICATION		FORMS	VARIABLE TYPE	DEFAULT
INPUT				
A	(INPUT)	SN,N,S,V,C,G	ANALOG	①
DESCRIPTOR				
B	(GAIN)	SN,N,S,V,C,G	ANALOG	①
D		SN,N,S,V,C,G,A	LOGIC	ENABLED ②
OUTPUT				
C	(OUTPUT)	SN,N	ANALOG	-

3. SPECIAL RESTRICTIONS:

① Required input

② Optional input

AGAI

4. CBL MACRO INSTRUCTION FORMAT:

Type ANALOG Op Code 8B8A Name ANALOG GAIN

POS					
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER		
	OPERATION CODE				
	POINTER TO NEXT CBL OP. CODE				
1	1	3	0	0	*
2	POINTER TO INPUT DATA A				
3	POINTER TO GAIN B				
4	POINTER TO OUTPUT C				

* Constant codes (used by ASUM and PCALC also).

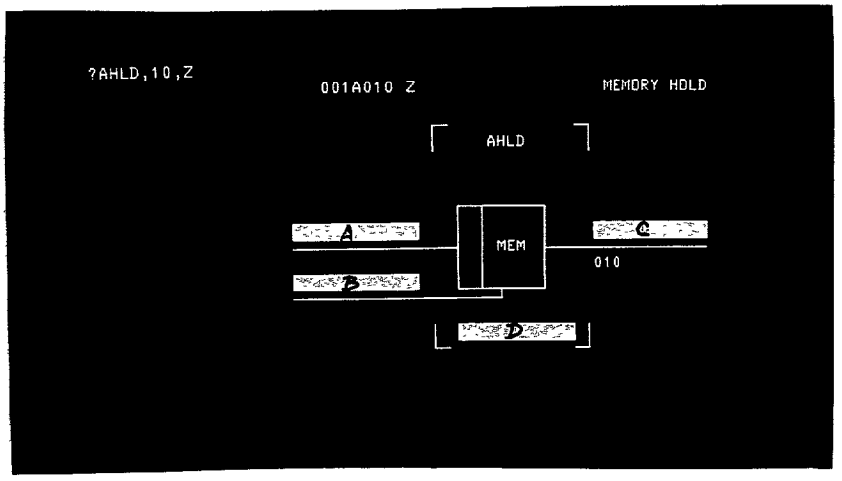
Operators
0 ⟶ store
1 ⟶ add
3 ⟶ multiply

5. ADDITIONAL INFORMATION:

None

AHLD

ANALOG HOLD



1. FUNCTION:

Analog output C is last analog input A held in memory. When B is false, C = A. When B is true, C = last A (freezes A). (This is the opposite of D action).

2. PARAMETERS:

IDENTIFICATION INPUT		FORMS	VARIABLE TYPE	DEFAULT
A		SN,N,S,V,C,G	ANALOG	①
B	(CONTROL)	SN,N,S,V,G,C	LOGIC	①
DESCRIPTOR				
D		SN,N,S,V,C,G,A	LOGIC	ENABLED ②
OUTPUTS				
C		SN,N	ANALOG	-

3. SPECIAL RESTRICTIONS:

① Required input

② Optional input

4. CBL MACRO INSTRUCTION FORMAT:

Type ANALOG Op Code 8B8B Name ANALOG HOLD

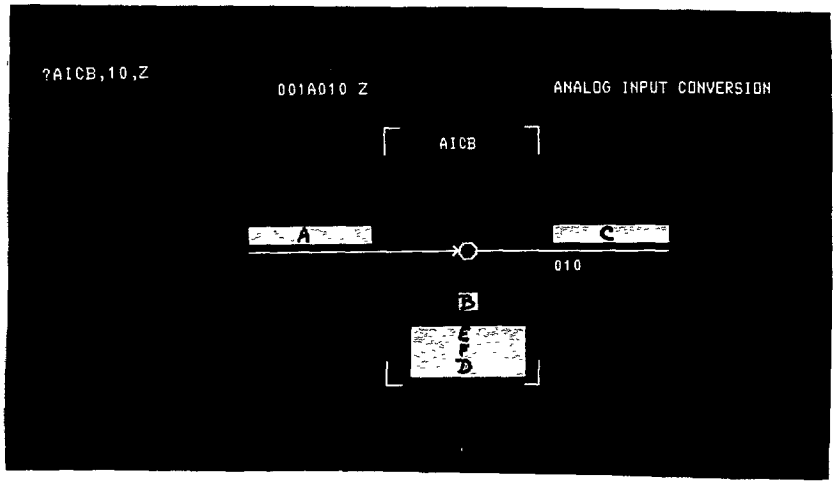
POS			
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER
	OPERATION CODE		
	POINTER TO NEXT CBL OP. CODE		
1	POINTER TO NEXT CBL OP. CODE		
2	POINTER TO CONTROL LOGICAL B		
3	POINTER TO INPUT DATA A		
4	POINTER TO OUTPUT VALUE C		

5. ADDITIONAL INFORMATION:

None

AICB

ANALOG INPUT CONVERTER



1. FUNCTION:

C = F + E * A (Converted according to code B)

This block causes PMC to perform input of CSH address A prior to execution of the segment. When AICB is executed, the input is converted according to code B, is multiplied by scale factor E and added to offset F.

Code B:
- 0 - No conversion
- 1 - 12 bit A/D
- 2 - undefined
- 3 - 16 bit binary
- 4 - 4 digit BCD
- 5 - 8 digit BCD
- 6 - 16 bit selsyn card

2. PARAMETERS:

IDENTIFICATION		VARIABLE	
INPUT	FORMS	TYPE	DEFAULT
A	A	-	①
DESCRIPTOR			
E (SCALE FACTOR)	SN,N,S,V,C,G	ANALOG	1.0
F (OFFSET)	SN,N,S,V,C,G	ANALOG	0.0
B (CONVERSION CODE)	OC	-	①
D	SN,N,S,V,C,G,A	LOGIC	ENABLED ②
OUTPUTS			
C	SN,N	ANALOG	-

3. SPECIAL RESTRICTIONS:

① Required input

② Optional input

4. CBL MACRO INSTRUCTION FORMAT:

Type ANALOG Op Code 8B8C Name ANALOG INPUT CONVERSION

POS					
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER		
	OPERATION CODE				
	POINTER TO NEXT CBL OP. CODE				
1	0	0	0	CODE B	*
2	POINTER TO INPUT DATA A				
3	POINTER TO SCALE FACTOR E				
4	POINTER TO OFFSET F				
5	POINTER TO OUTPUT DATA C				

* CODE B

0 - No conversion
1 - 12 bit A/D
2 - undefined
3 - 16 bit binary
4 - 4 digit BCD
5 - 8 digit BCD
6 - 16 bit selsyn card

5. ADDITIONAL INFORMATION:

If conversion code B is zero, no offset or scale factor is applied. The double word analog signal is input directly from two successive CSH addresses. This code is intended primarily for data transfer between PMC systems.

If conversion code B is non-zero, the following calculation is performed after conversion from the input type.

$$C = F + A \times E$$

where F is an analog constant used to bias the converted value and E is an analog scaling constant such that input A produces the desired value of C.

AICB

Code B = 1

The constants E and F are chosen to give desired C for A in the range -1.0 to 1.0 corresponding to full range of 12 bit input. In the case of F = 0, the value of E is equal to the desired value of C for 100% input. The 12 bit input is left justified in the 16 bit CSH data field.

Code B = 3

The constants E and F are chosen to give desired C for A in the range -32768 to +32767, which is full range of the 16 bit input. In the case of F = 0, the value of E is calculated as desired value of C for 100% input divided by 32767.

Code B = 4

The constants E and F are chosen to give desired C for the BCD input in range 0 to 9999. In the case of F = 0, the value of E is calculated as desired value of C divided by 9999.

Code B = 5

The constants E and F are chosen to give desired C for the BCD input in range 0 to 999999.99. In the case of F = 0, the value of E is calculated as desired value of C divided by 999999.99. The 8-digit BCD input C comes from two 16-bit CSH data fields with 9999 in the first address and 99.99 in the next address.

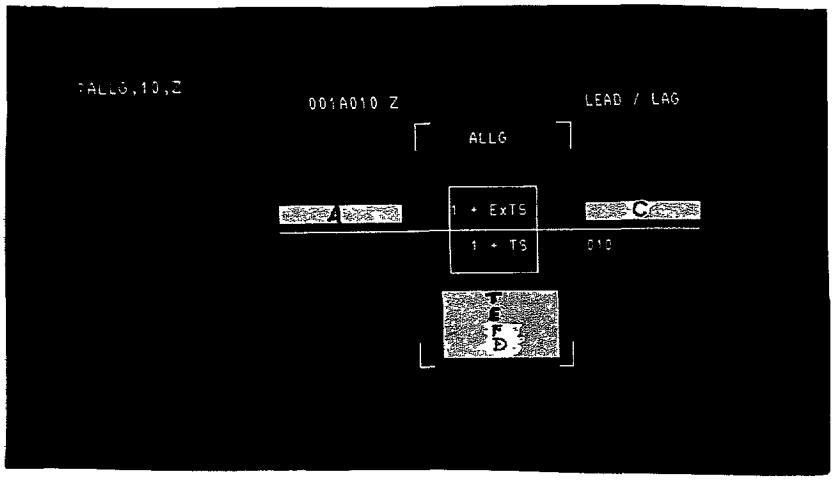
Code B = 6

The constants E and F are chosen to give desired C for A in the range 0 to 1023/1024 (.9990234) for full range of the 10 bit unsigned input which is left justified in the 16 bit CSH data field. In the case of F = 0 and E = 1024, the output C will correspond directly to the 10 bit count 0 to 1023.

NOTES

ALLG

ANALOG LEAD-LAG



1. FUNCTION:

$$O = \frac{1 + E \times TS}{1 + TS} \times A$$

Where: A is laplacian variable (domain of frequency analysis).

Analog output C results from applying a discrete approximation of a lead-lag network to analog input A. The lag time constant in seconds is T and the ratio of the lead to lag time constant is entered in E. The value of the initial output is entered in F.

2. PARAMETERS:

IDENTIFICATION	FORMS	VARIABLE TYPE	DEFAULT
INPUT			
A	SN,N,S,V,C,G	ANALOG	①
DESCRIPTORS			
T (LAG TIME CONSTANT)	SN,N,S,V,C,G	ANALOG	①
E (RATIO OF TIME CONSTANTS)	SN,N,S,V	ANALOG	0.0
F (INITIAL VALUE OF OUTPUT)	SN,N,S,V,C,G	ANALOG	0.0
D	SN,N,S,V,G,C,A	LOGIC	ENABLED ②
OUTPUT			
C	SN,N	ANALOG	-

ALLG

3. SPECIAL RESTRICTIONS:

① Required input

② Optional input

4. CBL MACRO INSTRUCTION FORMAT:

Type ANALOG Op Code 8B8D Name LEAD/LAG

POS			
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER
	OPERATION CODE		
	POINTER TO NEXT CBL OP. CODE		
1	POINTER TO FIRST TIME INDICATOR		*
2	POINTER TO INITIAL VALUE F		
3	POINTER TO INPUT DATA A		
4	POINTER TO INTERMEDIATE VALUE		**
5	POINTER TO SEGMENT INTERVAL		***
6	POINTER TO TIME CONSTANT T		
7	POINTER TO TIME CONSTANT RATIO E		
8	POINTER TO OUTPUT C		

* First time indicator is a count variable in the segment status block (SSB).

** Intermediate value is an analog variable allocated by the translator.

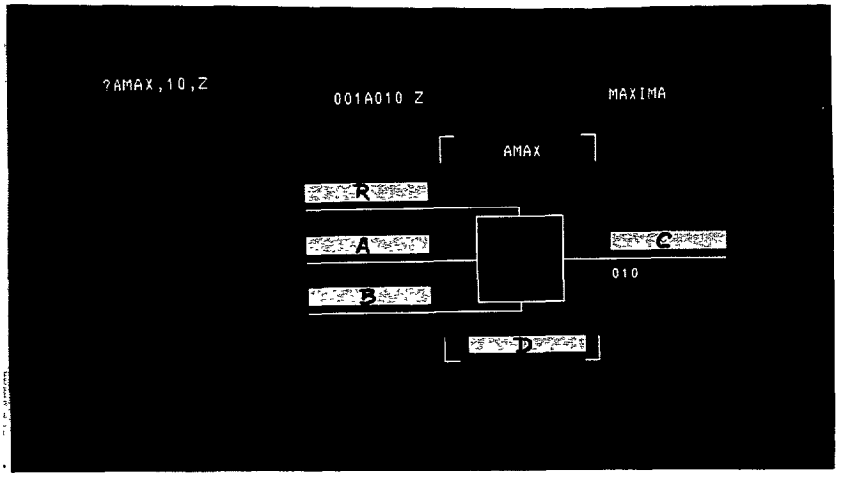
*** Segment interval is an analog variable in segment status block which contains the segments executation rate in seconds.

5. ADDITIONAL INFORMATION:

A flow chart appears in Appendix D.

AMAX

ANALOG MAXIMUM



1. FUNCTION:

C = MAX (R, A, B)

The analog output C is set equal to the input which is largest (maximum) one of three.

2. PARAMETERS:

IDENTIFICATION		VARIABLE	
INPUT	FORMS	TYPE	DEFAULT
A	SN,N,S,V,C,G	ANALOG	①
B	SN,N,S,V,C,G	ANALOG	①
R	SN,N,S,V,C,G	ANALOG	①
DESCRIPTOR			
D	SN,N,S,V,C,G,A	LOGIC	ENABLED ②
OUTPUT			
C	SN,N	ANALOG	-

3. SPECIAL RESTRICTIONS:

① Required input

② Optional input

4. CBL MACRO INSTRUCTION FORMAT:

Type ANALOG Op Code 8B8E Name MAXIMUM VALUE SELECT

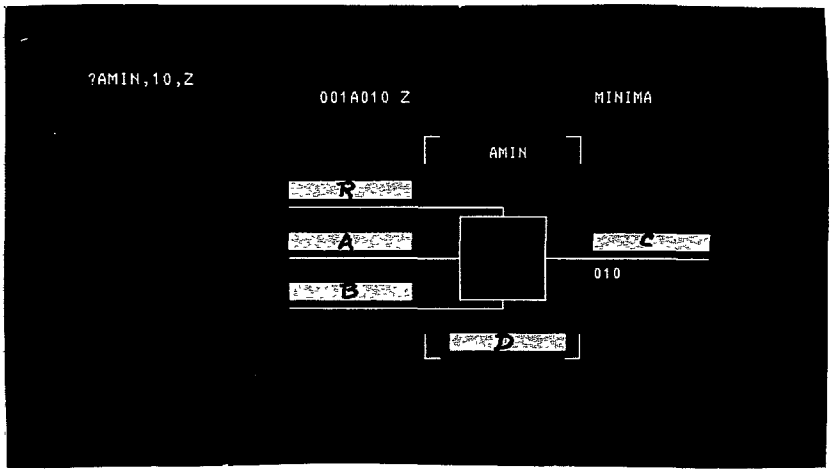
POS			
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER
	OPERATION CODE		
	POINTER TO NEXT CBL OP. CODE		
1	POINTER TO INPUT DATA R		
2	POINTER TO INPUT DATA A		
3	POINTER TO INPUT DATA B		
4	POINTER TO OUTPUT DATA C		

5. ADDITIONAL INFORMATION:

A flow chart appears in Appendix D.

AMIN

ANALOG MINIMUM



1. FUNCTION:

C = MIN (R, A, B)

The analog output C is set equal to the smallest (minimum) input R, A or B.

2. PARAMETERS:

IDENTIFICATION INPUT	FORMS	VARIABLE TYPE	DEFAULT
R	SN,N,S,V,C,G	ANALOG	①
A	SN,N,S,V,C,G	ANALOG	①
B	SN,N,S,V,C,G	ANALOG	①
DESCRIPTOR			
D (ENABLE)	SN,N,S,V,C,G,A	LOGIC	ENABLED ②
OUTPUT			
C	SN,N	ANALOG	-

3. SPECIAL RESTRICTIONS:

① Required input

② Optional input

4. CBL MACRO INSTRUCTION FORMAT:

Type ANALOG Op Code 8B8F Name MINIMUM VALUE SELECT

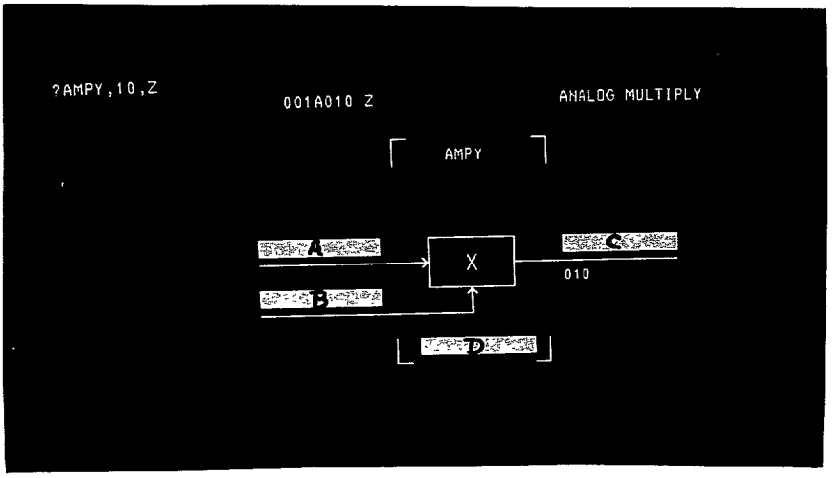
POS			
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER
	OPERATION CODE		
	POINTER TO NEXT CBL OP. CODE		
1	POINTER TO INPUT DATA R		
2	POINTER TO INPUT DATA A		
3	POINTER TO INPUT DATA B		
4	POINTER TO OUTPUT DATA C		

5. ADDITIONAL INFORMATION:

A flow chart appears in Appendix D.

AMPY

ANALOG MULTIPLY



1. FUNCTION:

C = A X B

The analog output C is set equal to the multiplication of analog inputs A and B if D is true. Overflow results with C = largest signal number.

2. PARAMETERS:

IDENTIFICATION INPUT	FORMS	VARIABLE TYPE	DEFAULT
A	SN,N,S,V,C,G	ANALOG	①
B	SN,N,S,V,C,G	ANALOG	①
DESCRIPTOR			
D	SN,N,S,V,C,G,A	LOGIC	ENABLED ②
OUTPUT			
C	SN,N	ANALOG	-

3. SPECIAL RESTRICTIONS:

① Required input

② Optional input

4. CBL MACRO INSTRUCTION FORMAT:

Type ANALOG Op Code 8B90 Name ANALOG MULTIPLY

POS					
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER		
	OPERATION CODE				
	POINTER TO NEXT CBL OP. CODE				
1	1	3	0	0	*
2	POINTER TO INPUT A				
3	POINTER TO INPUT B				
4	POINTER TO OUTPUT C				

* This is a fixed set of operator codes as used in the ASUM and PCALC blocks.

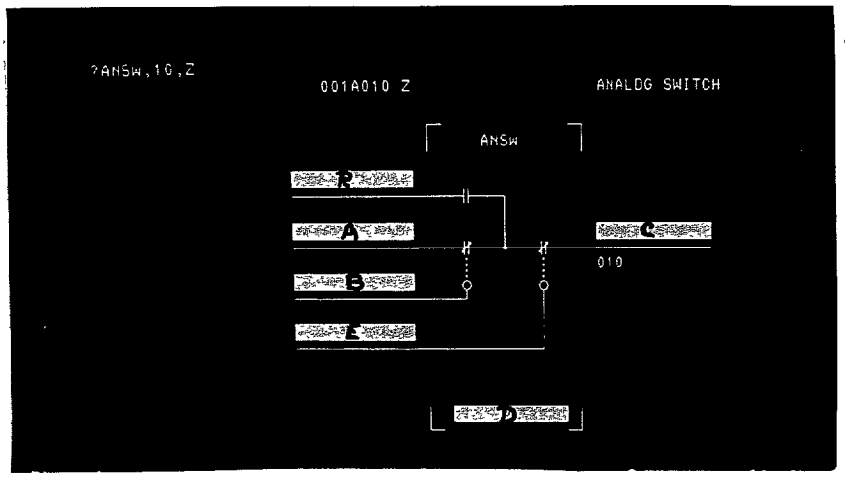
Operators
0 ⟶ Store Result
1 ⟶ Add
3 ⟶ Multiply

5. ADDITIONAL INFORMATION:

None

ANSW

ANALOG SWITCH



1. FUNCTION:

C = R, A or 0.0

The analog output C is switched as a function of inputs (logic) B and E:

B	E	C
-	T	0.0
T	F	R
F	F	A

2. PARAMETERS:

IDENTIFICATION INPUT	FORMS	VARIABLE TYPE	DEFAULT
R	SN,N,S,V,C,G	ANALOG	①
A	SN,N,S,V,C,G	ANALOG	①
B	SN,N,S,V,C,G	LOGIC	①
E	SN,N,S,V,C,G	LOGIC	①
DESCRIPTOR			
D	SN,N,S,V,C,G,A	LOGIC	ENABLED ②
OUTPUT			
C	SN,N	ANALOG	-

3. SPECIAL RESTRICTIONS:

① Required input

② Optional input

4. CBL MACRO INSTRUCTION FORMAT:

Type ANALOG Op Code 8B91 Name ANALOG SWITCH

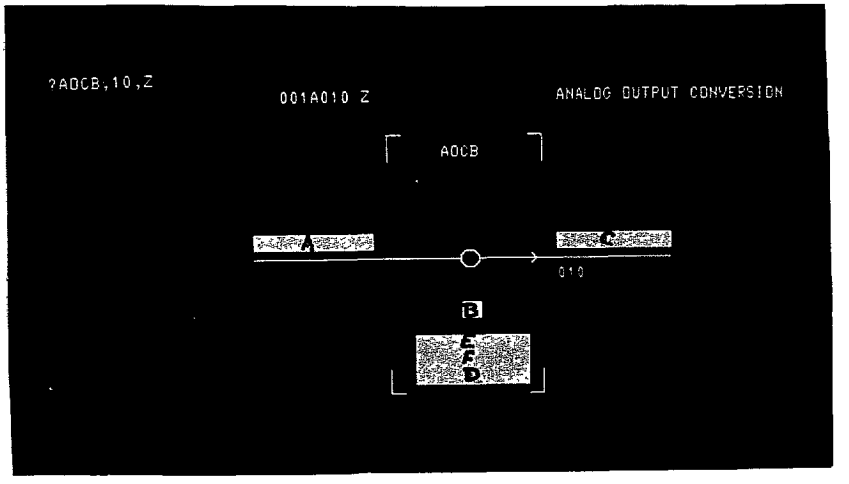
POS			
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER
	OPERATION CODE		
	POINTER TO NEXT CBL OP. CODE		
1	POINTER TO CONTROL LOGICAL E		
2	POINTER TO INPUT DATA R		
3	POINTER TO INPUT DATA A		
4	POINTER TO CONTROL LOGICAL B		
5	POINTER TO OUTPUT C		

5. ADDITIONAL INFORMATION:

None

AOCB

ANALOG OUTPUT CONVERTER



1. FUNCTION:

C = F + E X A (According to conversion code B)

Analog input A is multiplied by a scale factor E, added to offset F and the result converted according to code B, then stored in C. This output block causes the PMC to schedule the output of the data address C =/----/ after running the segment.

Code B:
- 0 - No conversion
- 1 - 12 bit D/A card
- 2 - Undefined
- 3 - 16 bit binary
- 4 - 4 digit BCD
- 5 - 8 digit BCD

If this block is not enabled, the last converted value will be output.

2. PARAMETERS:

IDENTIFICATION INPUT	FORMS	VARIABLE TYPE	DEFAULT
A	SN,N,S,V,C,G	ANALOG	①

DESCRIPTORS	FORMS	VARIABLE TYPE	DEFAULT
B (CONVERSION CODE)	OC	-	①
E (SCALE FACTOR)	SN,N,S,V,C,G	ANALOG	1.0
F (OFFSET)	SN,N,S,V,C,G	ANALOG	0.0
D	SN,N,S,V,C,G,A	LOGIC	ENABLED ②
OUTPUT			
C	A	-	-

3. SPECIAL RESTRICTIONS:

① Required input

② Optional input

4. CBL MACRO INSTRUCTION FORMAT:

Type ANALOG Op Code 8B92 Name ANALOG OUTPUT CONVERSION

POS					
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER		
	OPERATION CODE				
	POINTER TO NEXT CBL OP. CODE				
1	POINTER TO	0	0	CODE B	*
2	POINTER TO INPUT DATA A				
3	POINTER TO SCALE FACTOR E				
4	POINTER TO OFFSET F				
5	POINTER TO OUTPUT DATA C				

* Conversion Code B

0 - No conversion
1 - D/A conversion, 12 bit VDAC card
2 - Undefined
3 - 16 bit binary
4 - BCD 4 digit
5 - BCD 8 digit

AOCB

5. ADDITIONAL INFORMATION:

If conversion code B is zero, no offset or scale factor is applied. The double word analog signal is output directly to a pair of output registers. This code is intended primarily for data transfer between PMC systems.

If conversion code B is non-zero, the following calculation is performed prior to conversion to the desired output format.

$$C = F + A \times E$$

where F is an analog constant used to bus the converted value and E is an analog scaling constant such that the value of C is within the range described below for each code. The value of C is clamped to the defined range to prevent erroneous operation of output hardware.

Code B = 1

The constants E and F are chsoen for C in the range -1.0 to +1.0 to permit full range operation of the 12 bit D/A converter. In the case of F = 0, the value of E is calculated as reciprocal of maximum value of A for 100% output. The 12 bit result is left justified in the 16 bit data field of CSH.

Code B = 3

The constants E and F are chosen for C in the range -32768 to +32767 to permit full range of 16 bit parallel output. In the case of F = 0, the value of E is calculated as 32767 divided by maximum value of A for 100% output.

Code B = 4

The constants E and F are chosen for C in the range -9999 to +9999, which is maximum range of 4 digit BCD output. In the case of F = 0, the value of E is calculated as 9999 divided by maximum value of A for 100% output. The BCD output is the absolute value of C rounded to nearest integer value.

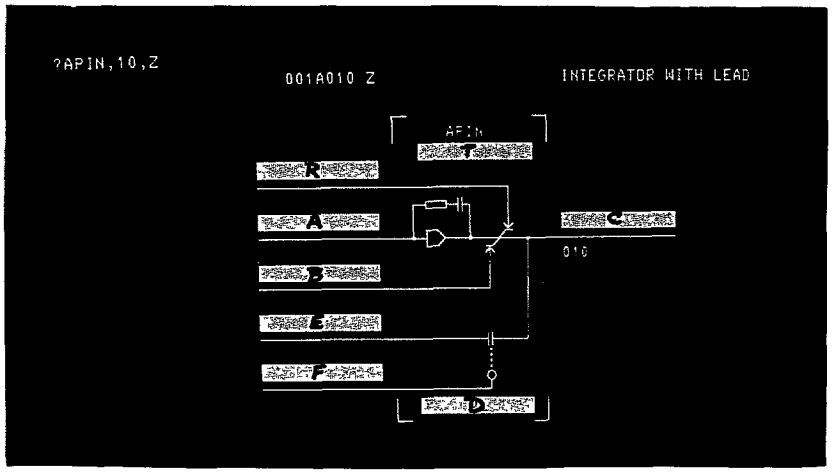
Code B = 5

The constants E and F are chosen for C in the range -999999.99 to +999999.99, which represents maximum range of 8 digit BCD. In the case of F = 0, the value of E is calculated as 999999.99 divded by maximum value of A for 100% output. The BCD output is the absolute value of C rounded to nearest .01 and output of two successive CSH addresses. The first address will contain 9999 while the next address will contain 99.99.

NOTES

APIN

ANALOG INTEGRATOR WITH LEAD



1. FUNCTION:

$C = \int A\,dt + AT$ if F false.

C RANGE: $R \geq C \geq B$

OR

C = E if F true

Analog output C is the time integral of analog input A added to a proportion of A as determined by T. C is limited to $R \geq C \geq B$.

2. PARAMETERS:

IDENTIFICATION	FORMS	VARIABLE TYPE	DEFAULT
INPUT			
R (HIGH LIMIT)	SN,N,S,V,C,G	ANALOG	1.7E38
A (INPUT)	SN,N,S,V,C,G	ANALOG	①
B (LOW LIMIT)	SN,N,S,V,C,G	ANALOG	-1.7E38
E (INITIAL VALUE)	SN,N,S,V,C,G	ANALOG	0.0
F (INITIALIZE INDICATOR)	SN,N,S,V,C,G	LOGIC	FALSE
DESCRIPTOR			
T (TIME CONSTANT)	SN,N,S,V,C,G	ANALOG	0.0
D	SN,N,S,V,C,G,A	LOGIC	ENABLED ②

OUTPUT	FORMS	VARIABLE TYPE	DEFAULT
C	SN,N	ANALOG	-

3. SPECIAL RESTRICTIONS:

① Required input

② Optional input

4. CBL MACRO INSTRUCTION FORMAT:

. Type ANALOG Op Code 8B93 Name INTEGRATOR WITH LEAD

POS		
	M | BLOCK SIZE	BLOCK SEQUENCE NUMBER
	OPERATION CODE	
	POINTER TO NEXT CBL OP. CODE	
1	POINTER TO FIRST TIME INDICATOR	*
2	POINTER TO INITIALIZE LOGICAL F	
3	POINTER TO INITITAL VALUE E	
4	POINTER TO INPUT DATA A	
5	POINTER TO INTERMEDIATE VALUE	**
6	POINTER TO SEGMENT INTERVAL	***
7	POINTER TO LEAD TIME CONSTANT T	
8	POINTER TO HIGH CLAMP R	
9	POINTER TO LOW CLAMP B	
10	POINTER TO OUTPUT C	

* First time indicator is a count variable in the segment status block (SSB).

** Intermediate value is an analog variable allocated by the translator.

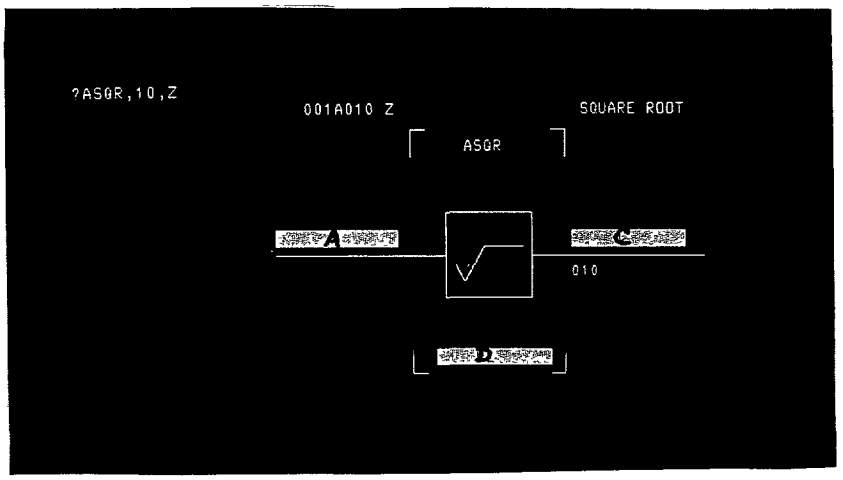
*** Segment interval is an analog variable in SSB which contains the segments executation rate in seconds.

5. ADDITIONAL INFORMATION:

A flow chart appears in Appendix D.

ASQR

ANALOG SQUARE ROOT



1. FUNCTION:

C = A if A positive

C = 0.0 if A negative

The analog output C is the square root of analog input A. C is set to 0.0 if A is negative.

2. PARAMETERS:

IDENTIFICATION	FORMS	VARIABLE TYPE	DEFAULT
INPUT			
A	SN,N,S,V,C,G	ANALOG	①
DESCRIPTOR			
D	SN,N,S,V,C,G	LOGIC	ENABLED ②
OUTPUT			
C	SN,N	ANALOG	-

3. SPECIAL RESTRICTIONS:

① Required input

② Optional input

4. CBL MACRO INSTRUCTION FORMAT:

Type ANALOG Op Code 8B94 Name SQUARE ROOT

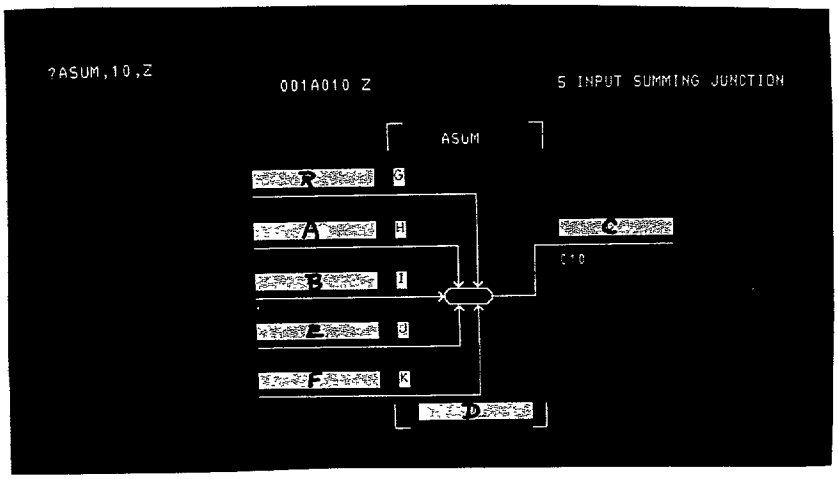
POS			
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER
	OPERATION CODE		
	POINTER TO NEXT CBL OP. CODE		
1	POINTER TO INPUT DATA A		
2	POINTER TO OUTPUT VALUE C		

5. ADDITIONAL INFORMATION:

None

ASUM

ANALOG SUMMER



1. FUNCTION:

Analog output C is the arithmetic summation of analog inputs R, A, B, E and F. Negation of these 5 inputs may be made individually with a minus (-) sign entered in G, H, I, J, K respectively. If a + or blank is used, signal is not inverted.

2. PARAMETERS:

IDENTIFICATION INPUT	FORMS	VARIABLE TYPE	DEFAULT
R	SN,N,S,V,C,G	ANALOG	①
A	SN,N,S,V,C,G	ANALOG	①
B	SN,N,S,V,C,G	ANALOG	②
E	SN,N,S,V,C,G	ANALOG	②
F	SN,N,S,V,C,G	ANALOG	②
DESCRIPTOR			
G (INVERTER)	OC	1 char. ASCII	③
H (INVERTER)	OC	1 char. ASCII	③
I (INVERTER)	OC	1 char. ASCII	③
J (INVERTER)	OC	1 char. ASCII	③
K (INVERTER)	OC	1 char. ASCII	③
D	SN,N,S,V,C,G	LOGIC	ENABLED ②
OUTPUT			
C	SN,N	ANALOG	-

3. SPECIAL RESTRICTIONS:

③ Operator must enter a + or - sign for inputs G, H, I, J and K (Default being an assumed +). In this way, any of the inputs can be negated. There is no input for B, E and F if omitted.

① Required input

② Optional input

4. CBL MACRO INSTRUCTION FORMAT:

Type ANALOG Op Code 8B95 Name SUMMING JUNCTION

POS					
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER		
	OPERATION CODE				
	POINTER TO NEXT CBL OP. CODE				
1	OPERATOR R	OPERATOR A	OPERATOR B	OPERATOR E	*
2	POINTER TO INPUT DATA R				
3	POINTER TO INPUT DATA A				
4	POINTER TO INPUT DATA B				
5	POINTER TO INPUT DATA E				
6	OPERATOR F	OPERATOR C	0	0	*
7	POINTER TO INPUT DATA F				
8	POINTER TO OUTPUT VALUE C				

* OPERATOR
0 - Store result
1 - Add
2 - Subtract

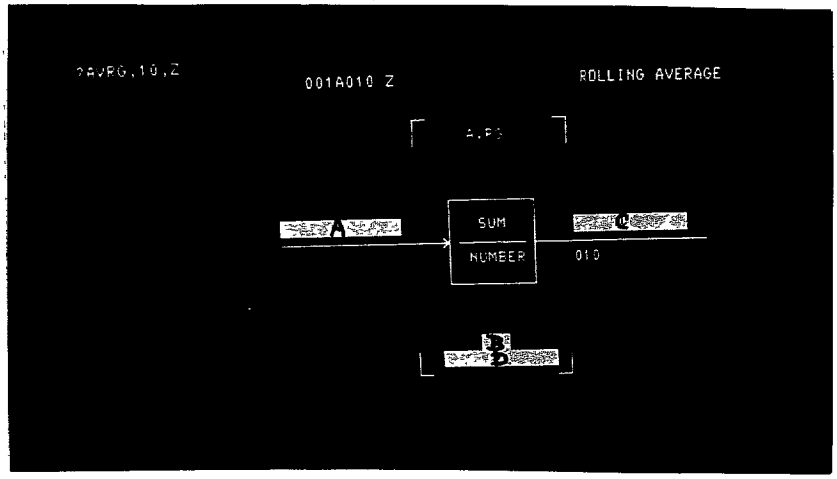
Note that the operator applying to variable C must be a zero, since this is the output into which the result is stored. The number of input pointers and arguments may vary from 2 to 5 with corresponding change in total block size.

5. ADDITIONAL INFORMATION:

A flow-chart appears in Appendix D.

AVRG

ANALOG AVERAGE



1. FUNCTION:

$$C = \frac{\Sigma A}{B}, \quad \text{COUNT } B \leq 99$$

The analog output C is the average of B values in a first in, first out (FIFO) list. At first execution the entire list is set to the very first input A value.

2. PARAMETERS:

IDENTIFICATION	FORMS	VARIABLE TYPE	DEFAULT
INPUT			
A	SN,N,S,V,C,G	ANALOG	①
DESCRIPTOR			
B (NUMBER OF VALUES)	C	COUNT	①
D	SN,N,S,V,C,G,A	LOGIC	ENABLED ②
OUTPUT			
C	SN,N	ANALOG	-

3. SPECIAL RESTRICTIONS:

① Required input

② Optional input

AVRG

4. CBL MACRO INSTRUCTION FORMAT:

Type ANALOG Op Code 8B96 Name ROLLING AVERAGE

POS	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER	
	OPERATION CODE			
	POINTER TO NEXT CBL OP. CODE			
1	POINTER TO FIRST TIME INDICATOR			**
2	POINTER TO LIST			*
3	POINTER TO OUTPUT VALUE C			
4	POINTER TO INPUT A			

* List is an intermediate analog array, B elements long.

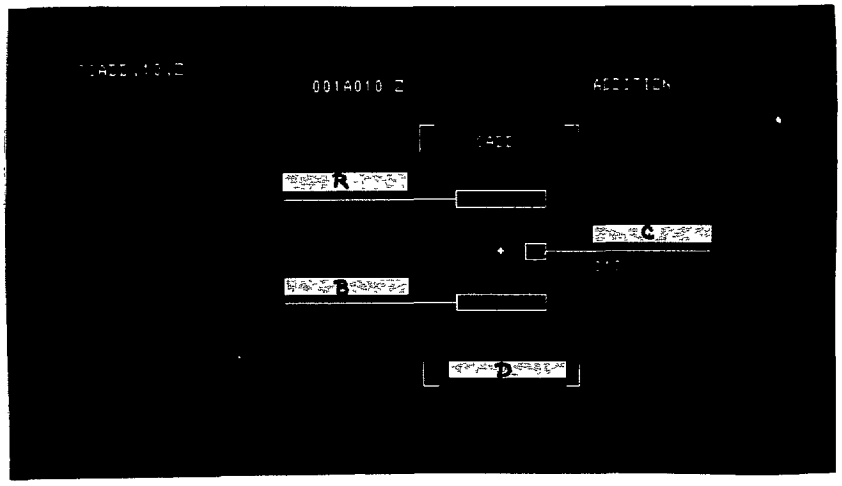
**First time indicator is a count variable in the segment status block (SSB).

5. ADDITIONAL INFORMATION:

A flow chart appears in Appendix D.

CADD

COUNT ADD



1. FUNCTION:

C = R + B

The output C is a count value equal to the 2's complement summation of count inputs R and B.

Range: $+32,767 \geqq \begin{matrix} C \\ R \\ B \end{matrix} \geqq -32,768$

2. PARAMETERS:

IDENTIFICATION INPUT	FORMS	VARIABLE TYPE	DEFAULT
R	SN,N,S,C,G,V,A	COUNT	①
B	SN,N,S,C,G,V,A	COUNT	①
DESCRIPTOR			
D	SN,N,S,C,G,V,A	LOGIC	ENABLED ②
OUTPUT			
C	SN,N	COUNT	

3. SPECIAL RESTRICTIONS:

No protection from overflow.
32,767 + 1 = 32,768

① Required input

② Optional input

CADD

4. CBL MACRO INSTRUCTION FORMAT:

Type COUNT Op Code 8B20 Name ADDITION

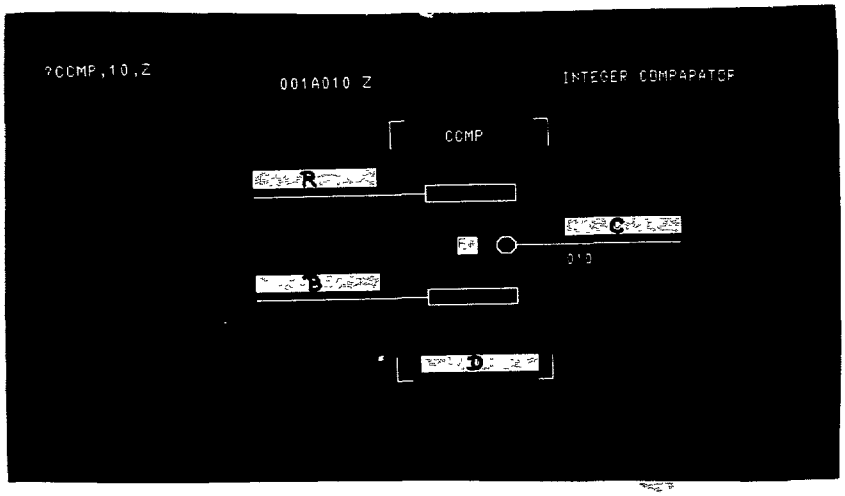
POS			
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER
	OPERATION CODE		
	POINTER TO NEXT CBL OP. CODE		
1	POINTER TO INPUT R		
2	POINTER TO INPUT B		
3	POINTER TO OUTPUT C		

5. ADDITIONAL INFORMATION:

None

CCMP

COUNT COMPARATOR



1. FUNCTION:

R :: B If true C = true
otherwise C = false

The logical bit output C is true or false based on the comparison of count inputs R and B. This comparison is entered in slot F as:

= Equal
< > Not equal (≠)
< Less than
> Greater than
> = Greater than or equal
< = Less than or equal

Such as:

R		B
ON LEFT	<	> ON RIGHT

2. PARAMETERS:

IDENTIFICATION INPUT	FORMS	VARIABLE TYPE	DEFAULT
R	SN,N,S,C,G,A	COUNT	①
B	SN,N,S,C,G,A	COUNT	①

CCMP

DESCRIPTOR	FORMS	VARIABLE TYPE	DEFAULT
F (COMPARATOR)	OC	-	①
D	SN,N,S,C,G,V,A	LOGIC	ENABLED ②
OUTPUT			
C	SN,N,A	LOGIC	-

3. SPECIAL RESTRICTIONS:

① Required input

② Optional input

4. CBL MACRO INSTRUCTION FORMAT:

Type COUNT Op Code 8B21 Name COUNT COMPARATOR

POS					
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER		
	OPERATION CODE				
	POINTER TO NEXT CBL OP. CODE				
1	0	0	0	CODE F	*
2	POINTER TO INPUT R				
3	POINTER TO INPUT B				
4	POINTER TO OUTPUT C				

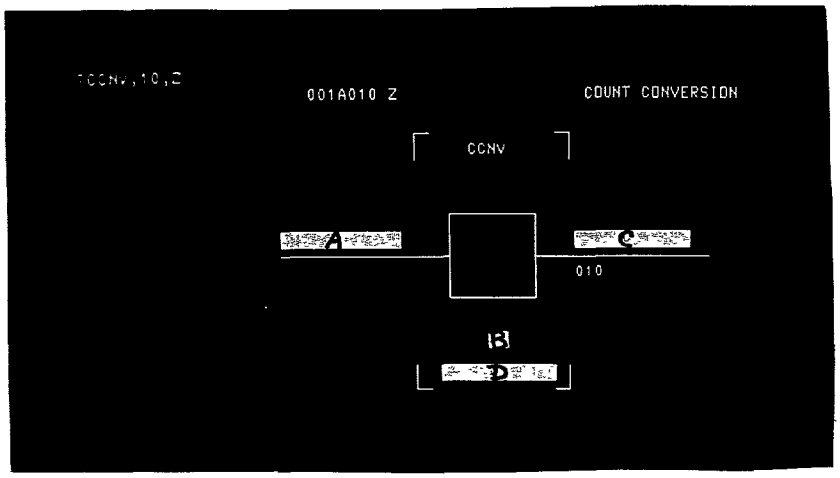
*CODE F	
0	=
1	<>
2	<
3	>
4	>=
5	<=

5. ADDITIONAL INFORMATION:

None

CCNV

COUNT CONVERSION



1. FUNCTION:

C = Function (A), Conversion controlled by code B.

The integer portion of the analog input A or the 5 decade B.C.D. value input A is converted to COUNT output C as determined by Code B.

CODE B:
1 = Analog to count
2 = 5 decade B.C.D. to count

2. PARAMETERS:

IDENTIFICATION		VARIABLE	
INPUT	FORMS	TYPE	DEFAULT
A	SN,N,S,V,C G	ANALOG	①
DESCRIPTOR			
B (CONVERSION CODE)	OC	-	①
D	SN,N,S,V,C,G,A	LOGIC	ENABLED ②
OUTPUT			
C	SN,N	COUNT	-

3. SPECIAL RESTRICTIONS:

The output value is limited to -32768 to +32767 for analog input and 0 to +32767 for decade B.C.D. input.

① Required input

② Optional input

4. CBL MACRO INSTRUCTION FORMAT:

Type COUNT Op Code 8B22 Name COUNT CONVERSION

POS	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER		
	OPERATION CODE				
	POINTER TO NEXT CBL OP. CODE				
I	0	0	0	CODE "B"	*
2	POINTER TO INPUT "A"				
3	POINTER TO OUTPUT "C"				

* Code B
1 Analog to count
2 B.C.D. to count

5. ADDITIONAL INFORMATION:

Five decade B.C.D. characters reside in two consecutive words in the following manner:

```
          MS                                  LS
          M+1                                 M
|1  1|1  0|0  0|0  0|   |1  1|1  0|0  0|0  0|
|5  2|1  8|7  4|3  1|   |5  2|1  8|7  4|3  0|
|0000|0000|0000|0011|   |0001|1001|0101|0110|
|3   |    |    |   1|   |1   |    |    |   0|
|1   |    |    |   6|   |5   |    |    |   0|

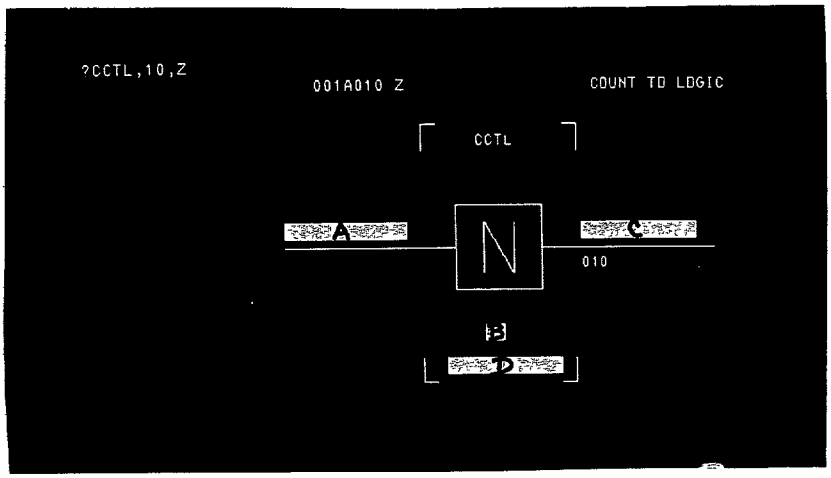
 BCD7 BCD6 BCD5 BCD4     BCD3 BCD2 BCD1 BCD0
```

EXAMPLE

$00\quad 00\quad 06\quad 14\quad 52\ 6_8 = 00\quad 03\quad 19\quad 56_{BCD}$

CCTL

COUNT TO LOGIC



1. FUNCTION:

C = A (BIT B)

The logic output C is bit B out of count input A. A is 16 bits.

2. PARAMETERS:

IDENTIFICATION	FORMS	VARIABLE TYPE	DEFAULT
INPUT			
A	SN,N,S,V,C,G,A	COUNT	①
DESCRIPTOR			
B (BIT POINTER)	OC	—	①
D	SN,N,S,V,C,G,A	LOGIC	ENABLED ②
OUTPUT			
C	SN,N,A	LOGIC	-

3. SPECIAL RESTRICTIONS:

① Required input

② Optional input

4. CBL MACRO INSTRUCTION FORMAT:

Type COUNT Op Code 8B23 Name COUNT TO LOGIC

POS					
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER		
	OPERATION CODE				
	POINTER TO NEXT CBL OP. CODE				
1	0	0	0	CODE "B"	*
2	POINTER TO INPUT "A"				
3	POINTER TO OUTPUT "C"				

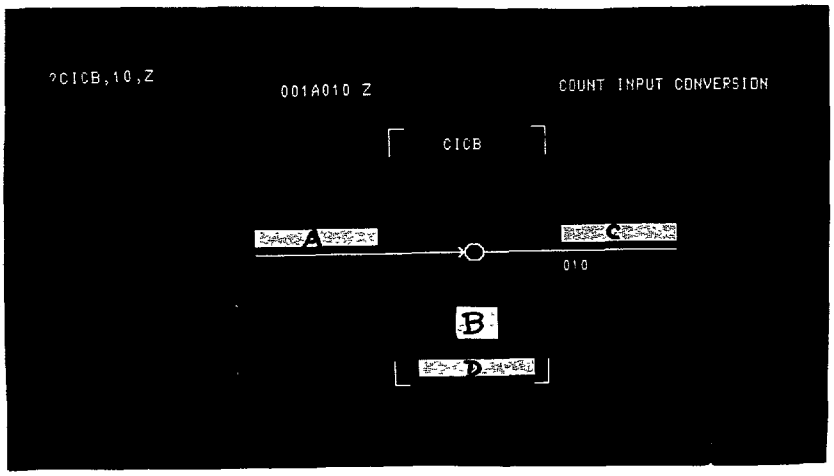
* Code B is bit selector with values 0-F.

5. ADDITIONAL INFORMATION:

None

CICB

COUNT INPUT CONTROL



1. FUNCTION:

C = A According to B-Code

This input block causes the PMC to schedule the input of data at CSH address A prior to execution of this block the input data is converted according to code B and stored in C.

Code B:
0 = No conversion
1 = 16 bit binary
2 = 4 digit B.C.D.

2. PARAMETERS:

IDENTIFICATION	FORMS	VARIABLE TYPE	DEFAULT
INPUT			
A	A	COUNT	①
DESCRIPTOR			
B (CONVERSION CODE)	OC	-	①
D	SN,N,S,V,C,G,V,A	LOGIC	ENABLED ②
OUTPUT			
C	SN,N	COUNT	-

3. SPECIAL RESTRICTIONS:

① Required input

② Optional input

4. CBL MACRO INSTRUCTION FORMAT:

Type COUNT Op Code 8B24 Name COUNT INPUT INVERSION

POS	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER		
	OPERATION CODE				
	POINTER TO NEXT CBL OP. CODE				
1	0	0	0	CODE "B"	*
2	POINTER TO INPUT "A"				
3	POINTER TO OUTPUT "C"				

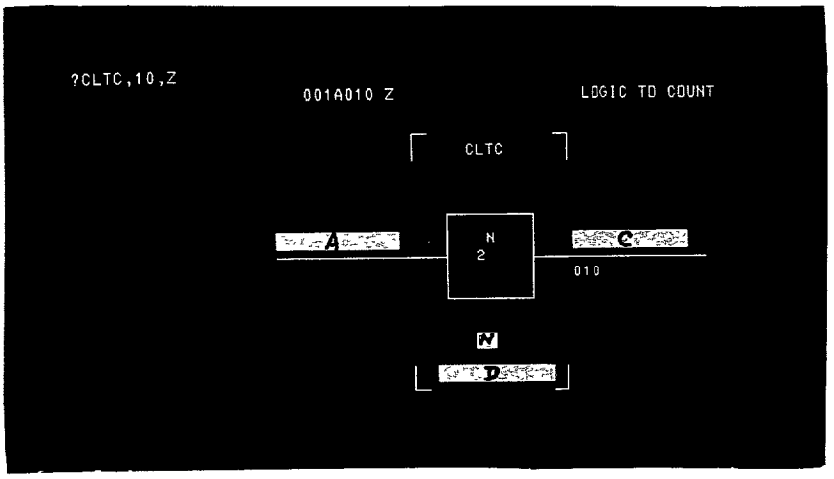
* Code B
0 - No conversion
1 - 16 bit binary
2 - 4 digit B.C.D.

5. ADDITIONAL INFORMATION:

None

CLTC

LOGIC TO COUNT



1. FUNCTION:

$C = 2^N$ if A true
C = 0 if A false

Output count C will be a value 2^N if logic input A is true. Otherwise, if logic input A is false, count C will be 0. N may be any number from 0 thru 15.

Note: 2^N N = 0 is $2^0 = 1$,
$2^{15} = -32,768$ (max. neg. #)
$2^{14} = 16,384$
$2^{13} = 8,192$

2. PARAMETERS:

IDENTIFICATION		VARIABLE	
INPUT	FORMS	TYPE	DEFAULT
A	SN,N,S,V,C,G,A	LOGIC	①
DESCRIPTOR			
N (CONVERSION CODE)	OC	-	①
D	SN,N,S,V,C,G,A	LOGIC	ENABLED ②
OUTPUT			
C	SN,N	COUNT	-

3. SPECIAL RESTRICTIONS:

① Required input

② Optional input

4. CBL MACRO INSTRUCTION FORMAT:

Type COUNT Op Code 8B25 Name LOGIC TO COUNT

POS				
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER	
	OPERATION CODE			
	POINTER TO NEXT CBL OP. CODE			
1	0	0	0	CODE "N"
2	POINTER TO INPUT "A"			
3	POINTER TO OUTPUT "C"			

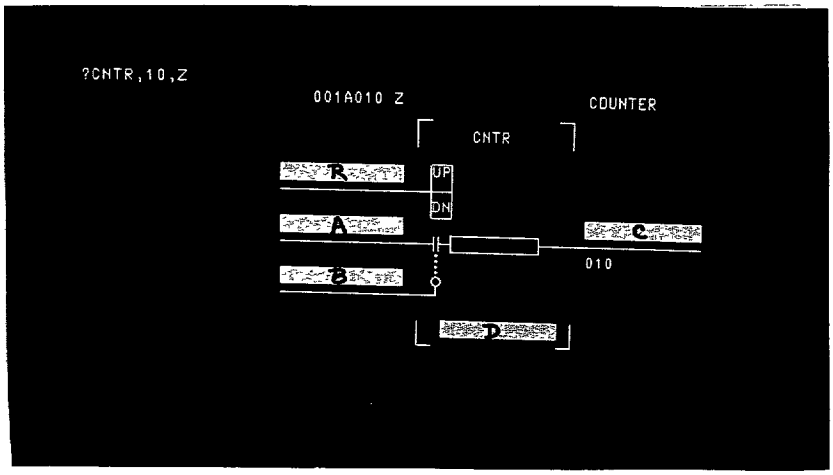
* Code N is bit selector with values 0-F.

5. ADDITIONAL INFORMATION:

None

CNTR

COUNT COUNTER



1. FUNCTION:

The count output C is a number equal to the 16 bit counter. This counter is preset to a value A count if logic input B is true. Each time this block is executed, it is counted up or down depending on logic input R being true or false, respectively. The rate at which this counter counts depends on the value "rate" in the segment ID block times the system rate 10, 20, 30, 40 or 50 milliseconds.

If segment rate is #2 and if system rate is 30 ms, the maximum count rate =

$$2 \times 30 = 60 \frac{\text{ms}}{\text{count}}$$

$$= 17 \text{ counts/sec.}$$

2. PARAMETERS:

IDENTIFICATION INPUT	FORMS	VARIABLE TYPE	DEFAULT
A	SN,N,S,V,C,G	COUNT	(1)
B	SN,N,S,V,G	LOGIC	(1)
R	SN,N,S,V,G	LOGIC	(1)

DESCRIPTOR	FORMS	VARIABLE TYPE	DEFAULT
D	SN,N,S,V,C,G,A,	LOGIC	ENABLED ②
OUTPUT			
C	SN,N	COUNT	-

3. SPECIAL RESTRICTIONS:

No overflow protection. 32767 + 1 = 32768

① Required input

② Optional input

4. CBL MACRO INSTRUCTION FORMAT:

Type COUNT Op Code 8B26 Name COUNTER

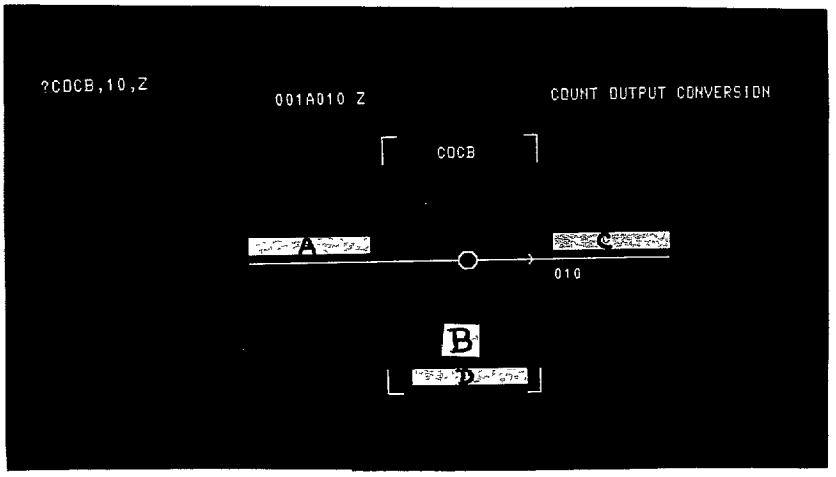
POS		
	M BLOCK SIZE	BLOCK SEQUENCE NUMBER
	OPERATION CODE	
	POINTER TO NEXT CBL OP. CODE	
1	POINTER TO INPUT "A"	
2	POINTER TO INPUT "B"	
3	POINTER TO INPUT "R"	
4	POINTER TO OUTPUT "C"	

5. ADDITIONAL INFORMATION:

None

COCB

COUNT OUTPUT CONTROL



1. FUNCTION:

C = A (According to B code)

Execution of this block causes count input A to be converted according to code B. At the completion of this segment, output C is transmitted on CSH address /----/.

Code B:

0 } No conversion
1 }

2 4 digit B.C.D.

2. PARAMETERS:

IDENTIFICATION		VARIABLE	
INPUT	FORMS	TYPE	DEFAULT
A	SN,N,S,V,C,G	COUNT	①
DESCRIPTOR			
B (CONVERSION CODE)	OC	-	①
D	SN,N,S,V,C,A	LOGIC	ENABLED ②
OUTPUT			
C	A	-	-

3. SPECIAL RESTRICTIONS:

① Required input

② Optional input

4. CBL MACRO INSTRUCTION FORMAT:

Type COUNT Op Code 8B27 Name COUNT OUTPUT CONVERSION

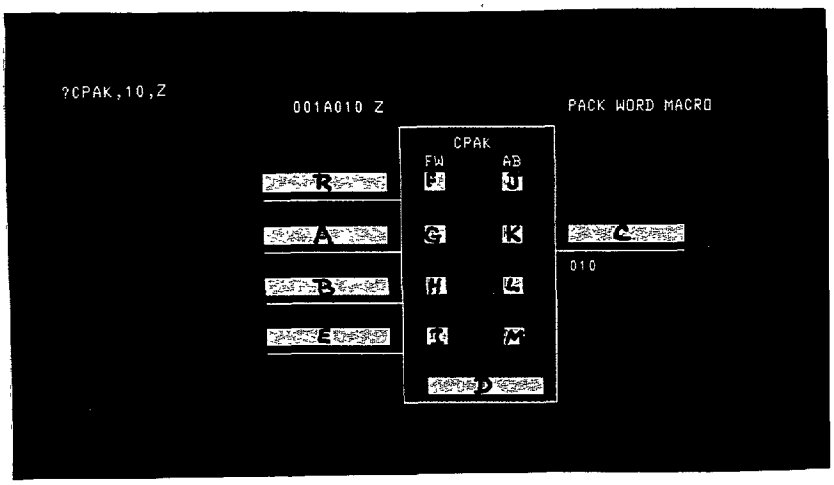
POS	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER			
	OPERATION CODE					
	POINTER TO NEXT CBL OP. CODE					
1	0		0	0	CODE "B"	*
2	POINTER TO INPUT "A"					
3	POINTER TO OUTPUT "C"					

* Code B
- 0 - No conversion
- 1 - 16 bit binary
- 2 - 4 digit B.C.D.

5. ADDITIONAL INFORMATION:

None

CPAK COUNT PACK WORD



1. FUNCTION:

This block packs the 16 bit output count C from parts of one to four inputs R, A, B or E (all count-type signals) according to codes FW = FIELD WIDTH (how many bits) and AB = ANCHOR BIT (defines bit number of right-most bit of FW in R, A, B or E).

Note: 1) Undefined fields in C will be zero.

2) Bits overflow out the right-hand side of output word C if more than 16 bits (limit) are defined by FW's.

2. PARAMETERS:

IDENTIFICATION INPUT	FORMS	VARIABLE TYPE	DEFAULT
R	SN,N,S,V,C,G	COUNT	①
A	SN,N,S,V,C,G	COUNT	②
B	SN,N,S,V,C,G	COUNT	②
E	SN,N,S,V,C,G	COUNT	②

DESCRIPTORS	FORMS	VARIABLE TYPE	DEFAULT
F (FIELD WIDTH)	OC	-	①
G (FIELD WIDTH)	OC	-	②
H (FIELD WIDTH)	OC	-	②
I (FIELD WIDTH)	OC	-	②
J (ANCHOR BIT)	OC	-	①
K (ANCHOR BIT)	OC	-	②
L (ANCHOR BIT)	OC		②
M (ANCHOR BIT)	OC		②
D	SN,N,S,V,C,G,A	LOGIC	ENABLED ③
OUTPUT			
C	SN,N	COUNT	-

3. SPECIAL RESTRICTIONS:

① Required input

② At least one input required; others are optional with null input default.

③ Optional input

1) If F+G+H+I<16, the determined field in word C (output) will be zeros.

2) If F+G+H+I>16, the output word will retain the last 16 bits specified by the four fields in the order R thru E, i.e., the bits overflow out the right hand end of output word C after the 16 bit limit has been exceeded.

Note: $0<FW<16$
$0\leq AB\leq 15$
FW and AB are 2-digit fields.

4. CBL MACRO INSTRUCTION FORMAT:

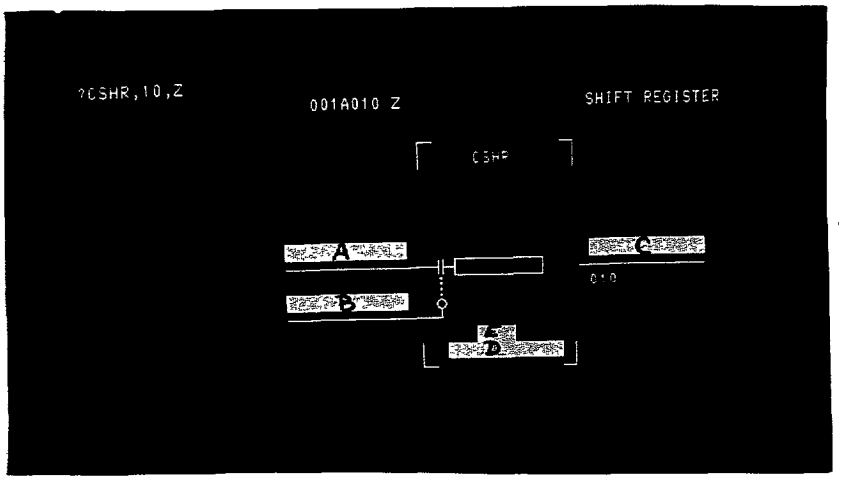
Type COUNT Op Code 8B2C Name COUNT DATA PACK

POS					
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER		
	OPERATION CODE				
	POINTER TO NEXT CBL OP. CODE				
1	FW_A	AB_A	FW_R	AB_R	*
2	POINTER TO INPUT DATA "R"				
3	POINTER TO INPUT DATA "A"				
4	FW_E	AB_E	FW_B	AB_B	*
5	POINTER TO INPUT DATA "B"				
6	POINTER TO INPUT DATA "E"				
7	POINTER TO OUTPUT VALUE "C"				

* Field width and anchor bit codes have values 0-F. The number of input pointers and codes may vary from 1 to 4 causing variable block size.

CSHR

COUNT SHIFT REGISTER



1. FUNCTION:

The count output C is a 16-bit pattern which results after a logical shift of (±) E positions of the value in the register (box) position shown. If E is +, the shift of bits is to the right with 0's filling the upper bits. If E is -, the shift of bits is to the left with 0's filling the lower bits vacated.

If the B input is true (logic), the count preset value A will be placed in the register before C is determined.

If B is logic false, the shift will be performed on the old output count C.

2. PARAMETERS:

IDENTIFICATION		VARIABLE	
INPUT	FORMS	TYPE	DEFAULT
A	SN,N,S,V,C,G	COUNT	①
B	SN,N,S,V,C,G	LOGIC	①
DESCRIPTOR			
E (SHIFT COUNT)	OC	-	①
D	SN,N,S,V,C,G,A	LOGIC	ENABLED ②
OUTPUT			
C	SN,N	COUNT	-

CSHR

3. SPECIAL RESTRICTIONS:

Range of E is -15 to +15

① Required input

② Optional input

4. CBL MACRO INSTRUCTION FORMAT:

Type COUNT Op Code 8B28 Name SHIFT REGISTER

POS			
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER
	OPERATION CODE		
	POINTER TO NEXT CBL OP. CODE		
1	0	0	CODE "E"
2	POINTER TO INPUT "A"		
3	POINTER TO INPUT "B"		
4	POINTER TO OUTPUT "C"		

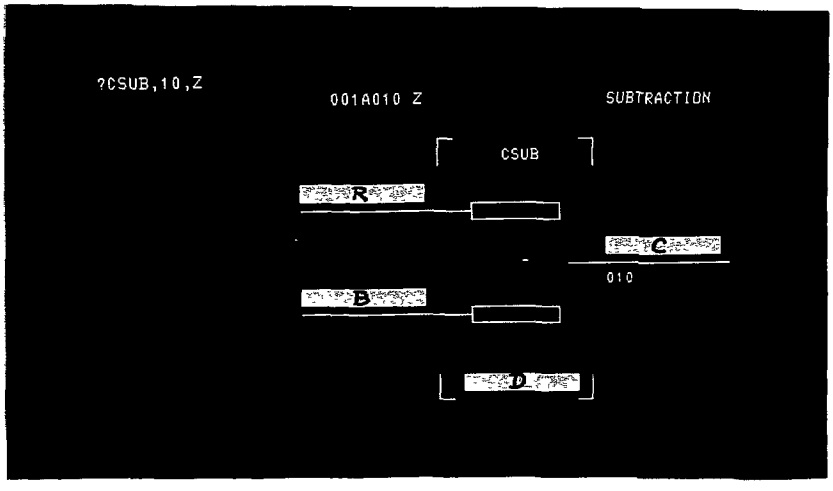
* Code E
00-0F for shift right 0-15
1F-10 for shift left 1-15

5. ADDITIONAL INFORMATION:

None

CSUB

COUNT SUBTRACT



1. FUNCTION:

C = R - B

The count output C is equal to the 2's complement subtraction R - B. Maximum range: $+32,767 \geq \begin{matrix} C \\ R \\ B \end{matrix} \geq -32,768$

2. PARAMETERS:

IDENTIFICATION INPUT	FORMS	VARIABLE TYPE	DEFAULT
R	SN,N,S,V,C,G	COUNT	①
B	SN,N,S,V,C,G	COUNT	①
DESCRIPTOR			
D	SN,N,S,V,C,G,A	LOGIC	ENABLED②
OUTPUT			
C	SN,N	COUNT	-

3. SPECIAL RESTRICTIONS:

① Required input

② Optional input

4. CBL MACRO INSTRUCTION FORMAT:

Type COUNT Op Code 8B29 Name SUBTRACTION

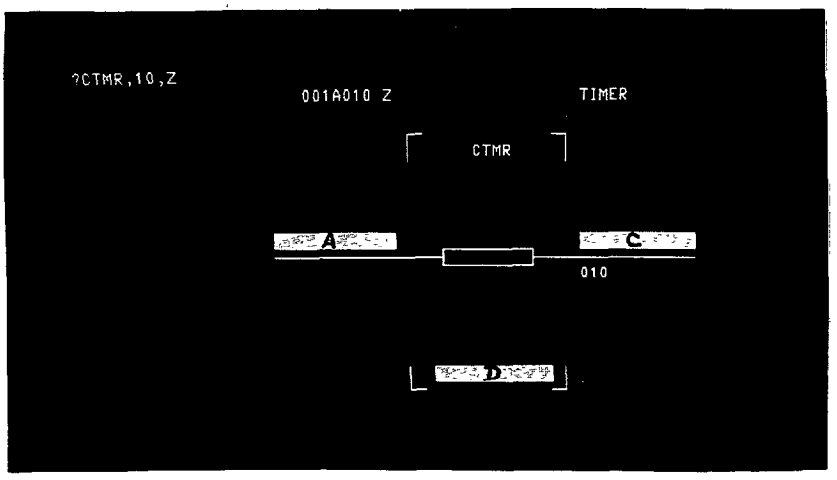
POS	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER
	OPERATION CODE		
	POINTER TO NEXT CBL OP. CODE		
1	POINTER TO INPUT "R"		
2	POINTER TO INPUT "B"		
3	POINTER TO OUTPUT "C"		

5. ADDITIONAL INFORMATION:

None

CTMR

COUNT TIMER



1. FUNCTION:

C = Time count in time increments of 10 milliseconds if A = false.

C = 0 count if A = true.

Count output C is equal to the number of time increments that have elapsed since the last initialize signal (Input A = true). Each count is equivalent to 10 milliseconds.

2. PARAMETERS:

IDENTIFICATION	FORMS	VARIABLE TYPE	DEFAULT
INPUT			
A	SN,N,S,V,C,G	LOGIC	①
DESCRIPTOR			
D	SN,N,S,V,C,G,A	LOGIC	ENABLED ②
OUTPUT			
C	SN,N	COUNT	-

3. SPECIAL RESTRICTIONS:

① Required input

② Optional input

4. CBL MACRO INSTRUCTION FORMAT:

Type COUNT Op Code 8B2A Name TIMER

POS	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER
	OPERATION CODE		
	POINTER TO NEXT CBL OP. CODE		
1	POINTER TO INPUT "A"		
2	POINTER TO SEGMENT INTERVAL		*
3	POINTER TO OUTPUT "C"		

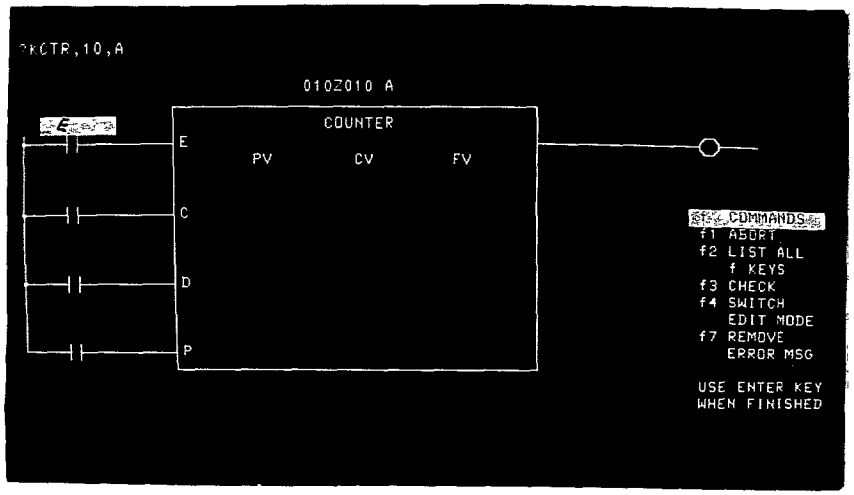
* Segment interval is a count variable in segment status block which contains the segments execution rate in 10-second intervals.

5. ADDITIONAL INFORMATION:

None

KCTR

COUNTER



1. FUNCTION:

This counter has a primary output (COIL) and a secondary output (CV). The primary (coil) is on when the secondary (current value) is $\geq$the final value (FV).

The counter will increment/decrement the current value on the transition of the count signal (C) from low to high state when the enable variable (E) is true.

The up/down input (D) causes the counter to decrement when true and increment when false.

The preset input (P) causes the current value (CV) to be preset to the preset value (PV) when true.

Input contacts are defined by specifying an open or closed contact on one or more of the input leads. If no contact is specified, the corresponding input signal (E, C, D or P) is not required. Refer to FSS Operator Manual (GEK-65435) for additional information on editing contacts.

KCTR

2. PARAMETERS:

IDENTIFICATION INPUT	FORMS	VARIABLE TYPE	DEFAULT
E	SN,N,S,C,A,G	LOGIC	①
C	SN,N,S,A,G	LOGIC	①
D	SN,N,S,A,G,C	LOGIC	①
P	SN,N,S,A,G,C	LOGIC	①
PV	SN,N,S,G,C	COUNT	②
FV	SN,N,S,G,C	COUNT	②

DESCRIPTOR

None

OUTPUTS

COIL	SN,N,A	LOGIC	-
CV	S	COUNT	③

3. SPECIAL RESTRICTIONS:

① Required input if contact is defined

② Required input

③ Clamped $-32768 \leq CV \leq 32767$

4. CBL MACRO INSTRUCTION FORMAT:

Type RELAY Op Code 8B51 Name COUNTER

POS		
	M	BLOCK SIZE / BLOCK SEQUENCE NUMBER
	OPERATION CODE	
	POINTER TO NEXT CBL OP. CODE	
1	LOP	POINTER TO INPUT LOGICAL E
2	LOP	POINTER TO INPUT LOGICAL C
3	5	POINTER TO INTERMEDIATE LOGICAL
4	LOP	POINTER TO INPUT LOGICAL D
5	LOP	POINTER TO INPUT LOGICAL P
6	POINTER TO INPUT COUNT PV	
7	POINTER TO INPUT COUNT FV	
8	POINTER TO OUTPUT COUNT CV	
9	6	POINTER TO LOGICAL OUTPUT COIL

KCTR

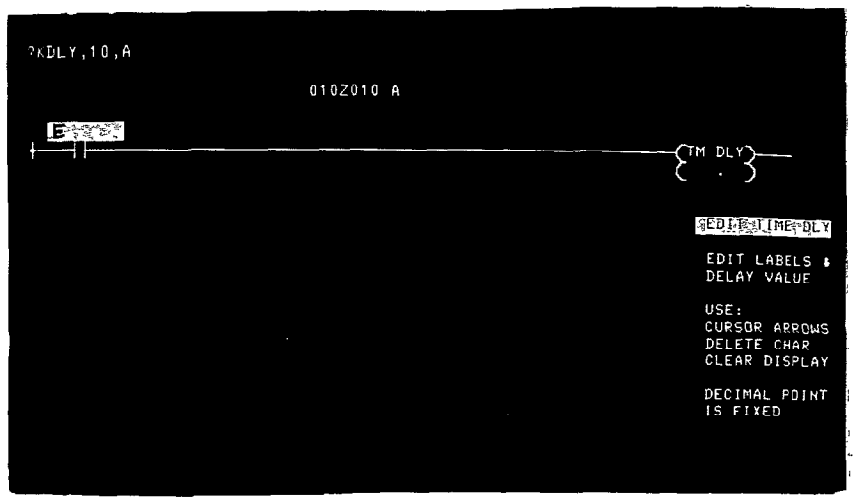
LOP	- LOGICAL OPERATION
4	- LOAD FALSE INPUT LOGICAL
5	- LOAD TRUE INPUT LOGICAL
6	- STORE RESULT TO OUTPUT LOGICAL
7	- "6" = SHORTED CONTACT
7	- "7" = OPEN CONTACT

The counter diagram may be edited to short open (invert) the input logic signals.

NOTES

KDLY

TIME DELAY



1. FUNCTION:

The time delay output coil becomes true TMDLY seconds after the enable input (E) becomes true. The output coil is false when the input (E) is false.

2. PARAMETERS:

IDENTIFICATION INPUT	FORMS	VARIABLE TYPE	DEFAULT
E	SN,N,S,G,A	LOGIC	①
TMDLY	C	COUNT	②

DESCRIPTOR

None

OUTPUT			
COIL	SN,N,A	LOGIC	-

3. SPECIAL RESTRICTIONS:

① Required input

② Value restricted $0.00 \leq TMDLY \leq 327.67$

4. CBL MACRO INSTRUCTION FORMAT:

Type RELAY Op Code 8B45 Name TIME DELAY

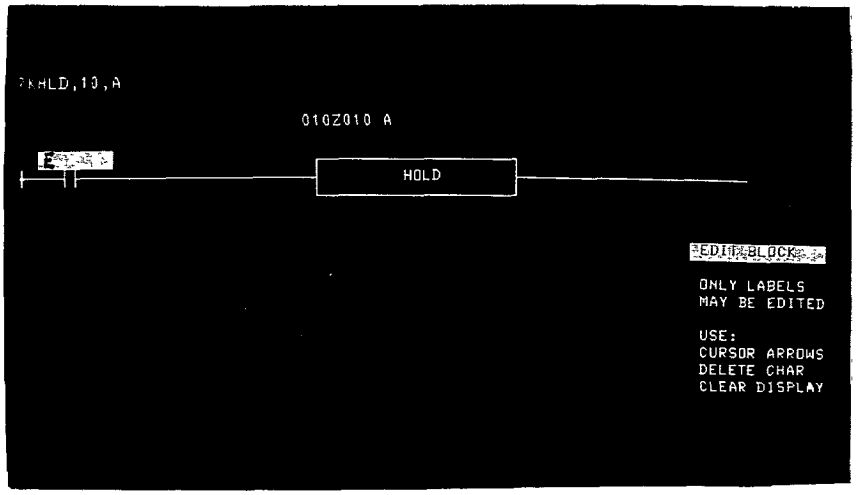
POS	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER	
	OPERATION CODE			
	POINTER TO NEXT CBL OP. CODE			
1	POINTER TO INPUT LOGICAL E			
2	POINTER TO TIME DELAY TM DLY			
3	POINTER TO ELAPSED TIME			*
4	POINTER TO EXECUTION RATE			**
5	POINTER TO LOGICAL OUTPUT COIL			

* Pointer to intermediate count

** Pointer to Segment Status Block (SSB) variable (REPR) - repetition rate.

KHLD

HOLD LOGIC



1. FUNCTION:

The "hold" block causes the execution of all control blocks with higher sequence numbers to be bypassed if the input enable variable (E) is true. However, all I/O points in the segment are updated in the normal manner.

2. PARAMETERS:

IDENTIFICATION INPUT	FORMS	VARIABLE TYPE	DEFAULT
E	SN,N,S,G,A	LOGIC	①

DESCRIPTOR
None

OUTPUT
None

3. SPECIAL RESTRICTIONS:

① Required input

4. CBL MACRO INSTRUCTION FORMAT: KHLD

Type RELAY Op Code 8B4A Name HOLD LOGIC

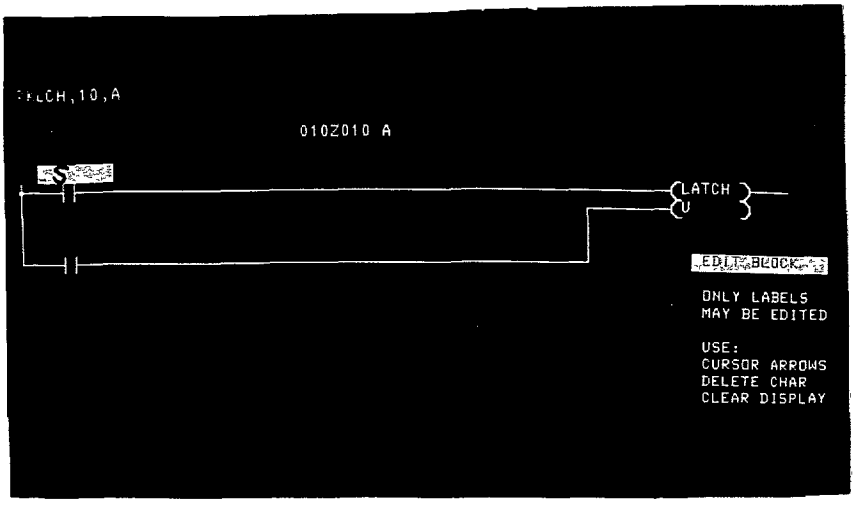
POS				
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER	
	OPERATION CODE			
	POINTER TO NEXT CBL OP. CODE			
1	POINTER TO INPUT LOGICAL E			
2	8	3	9	6

5. ADDITIONAL INFORMATION:

None

KLCH

LATCHED RELAY



1. FUNCTION:

The latched relay output coil is true when the set latch input (S) is true. The output coil is false when the reset latch input (under S) is true. The output coil does not change when both inputs are the same.

2. PARAMETERS:

IDENTIFICATION INPUT	FORMS	VARIABLE TYPE	DEFAULT
S	SN,N,S,G,A	LOGIC	①
RESET	SN,N,S,G,A	LOGIC	①

DESCRIPTOR
None

OUTPUT			
COIL	SN,N,A	LOGIC	-

3. SPECIAL RESTRICTIONS:

① Required input

4. CBL MACRO INSTRUCTION FORMAT:

Type RELAY Op Code 8B42 Name LATCHED RELAY

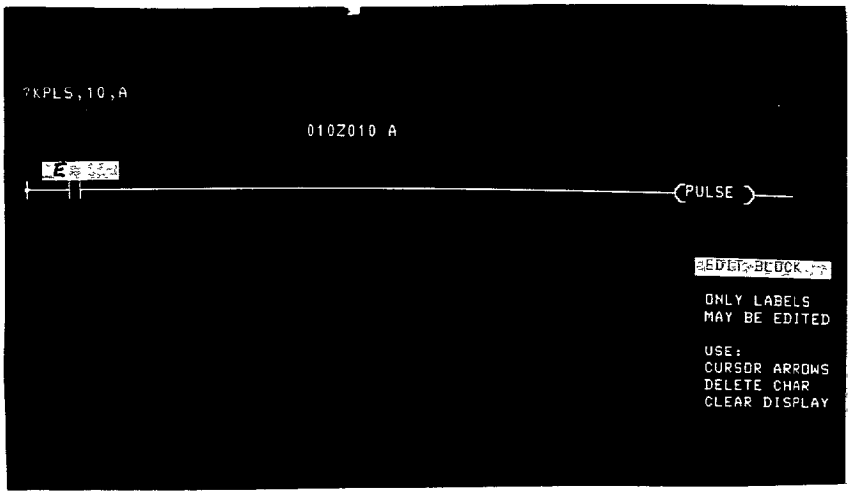
POS	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER
	OPERATION CODE		
	POINTER TO NEXT CBL OP. CODE		
I	POINTER TO INPUT LOGICAL S		
2	POINTER TO INPUT LOGICAL RESET		
3	POINTER TO LOGICAL OUTPUT COIL		

5. ADDITIONAL INFORMATION:

None

KPLS

PULSED RELAY



1. FUNCTION:

When the input pulse (E) transitions from low to high, output (COIL) will go true and remain true until the next execution of the block.

The input pulse (E) must cycle false, then true before another pulse output goes true.

2. PARAMETERS:

IDENTIFICATION		VARIABLE	
INPUT	FORMS	TYPE	DEFAULT
E	SN,N,S,G,A	LOGIC	①
DESCRIPTOR			
None			
OUTPUT			
COIL	SN,N,A	LOGIC	-

3. SPECIAL RESTRICTIONS:

① Required input

4. CBL MACRO INSTRUCTION FORMAT:

Type RELAY Op Code 8B44 Name PULSED RELAY

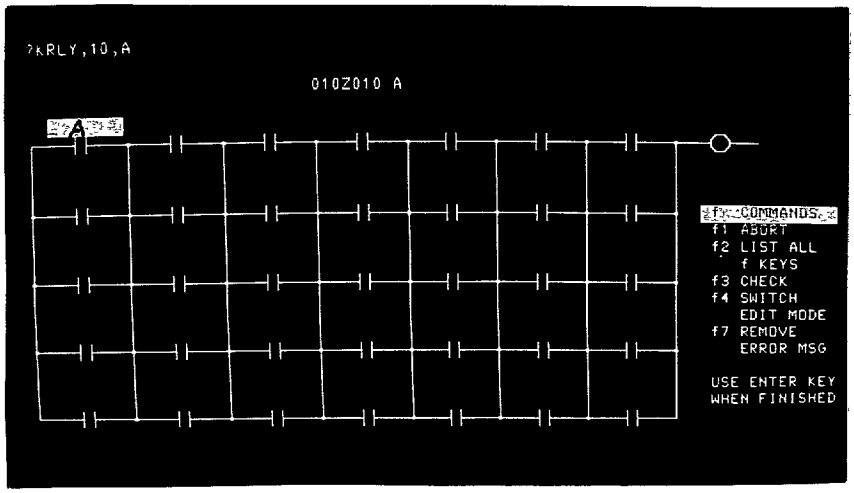
POS	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER
	OPERATION CODE		
	POINTER TO NEXT CBL OP. CODE		
1	POINTER TO INPUT LOGICAL E		
2	POINTER TO INTERMEDIATE LOGICAL		
3	POINTER TO LOGICAL OUTPUT COIL		

5. ADDITIONAL INFORMATION:

None

KRLY

RELAY LADDER



1. FUNCTION:

The relay ladder can have up to 35 input contacts (A). These contacts are ORed and ANDed together to produce an output at the coil. Vertical connections represent OR functions. Horizontal connections represent AND functions. The relay ladder diagram is edited to produce the required function. Contacts may be removed, inverted or replaced by a horizontal line. Vertical lines may be removed. This process is described in detail in the FSS Operator Manual (GEK-65435).

2. PARAMETERS:

IDENTIFICATION		VARIABLE	
INPUT	FORMS	TYPE	DEFAULT
A	SN,N,S,G,C,A	LOGIC	①
DESCRIPTOR			
None			
OUTPUT			
COIL	SN,N,A	LOGIC	-

3. SPECIAL RESTRICTIONS:

① Up to 35 input contact signals are allowed. If a contact is defined, an input is required.

4. CBL MACRO INSTRUCTION FORMAT:

Type RELAY Op Code 8B49 Name RELAY LADDER

POS			
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER
	OPERATION CODE		
	POINTER TO NEXT CBL OP. CODE		
1	LOP	POINTER TO INPUT LOGICAL	A(1)
2	LOP	POINTER TO INPUT LOGICAL	A(2)
3	(LENGTH IS VARIABLE AS REQUIRED)		
4	6	POINTER TO LOGICAL OUTPUT COIL	

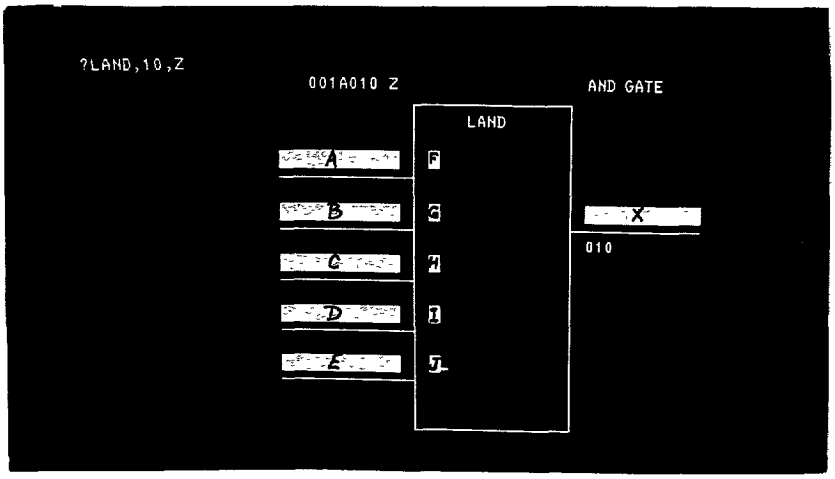
LOP	LOGICAL OPERATION
0	OR FALSE INPUT LOGICAL
1	OR TRUE INPUT LOGICAL
2	AND FALSE INPUT LOGICAL
3	AND TRUE INPUT LOGICAL
4	LOAD FALSE INPUT LOGICAL
5	LOAD TRUE INPUT LOGICAL
6	STORE RESULT TO OUTPUT LOGICAL
7	1 = STACK OR
7	2 = STACK AND

5. ADDITIONAL INFORMATION:

None

LAND

LOGIC AND



1. FUNCTION:

Output (X) is the logical AND of the five input signals. If F, G, H, I or J contains a minus (-) or circle (0), its signal input sense is inverted before the AND function is performed.

2. PARAMETERS:

IDENTIFICATION		VARIABLE	
INPUT	FORMS	TYPE	DEFAULT
A	SN,N,S,V,G,A	LOGIC	①
B	SN,N,S,V,G,A	LOGIC	①
C	SN,N,S,V,G,A	LOGIC	②
D	SN,N,S,V,G,A	LOGIC	②
E	SN,N,S,V,G,A	LOGIC	②
DESCRIPTOR			
F	(INVERTER) OC	-	③
G	(INVERTER) OC	-	③
H	(INVERTER) OC	-	③
I	(INVERTER) OC	-	③
J	(INVERTER) OC	-	③
OUTPUT			
X	SN,N,A	LOGIC	-

LAND

3. SPECIAL RESTRICTIONS:

① Required input

② Optional input(s)

③ Optional input, Default is +.

4. CBL MACRO INSTRUCTION FORMAT:

Type LOGIC Op Code 8B47 Name LOGIC AND

POS			
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER
	OPERATION CODE		
	POINTER TO NEXT CBL OP. CODE		
1	OP	POINTER TO LOGICAL A	
2	OP	POINTER TO LOGICAL B	
3	OP	POINTER TO LOGICAL D	*
4	OP	POINTER TO LOGICAL E	*
5	OP	POINTER TO LOGICAL F	*
6	OP	POINTER TO LOGICAL X	

LOGIC
OP

Maximum number of five logical inputs which may be inverted if desired. The following logical operators may be used:

HEX	LOGIC OP - CODE	NAME	DESCRIPTION OF OPERATION
2	0010	LAF	Logical AND stack with complement of input
3	0011	LAT	Logical AND stack with input.
4	0100	LDF	Load stack with complement of input.
5	0101	LDT	Load stack with input
6	0110	SST	Store stack into output

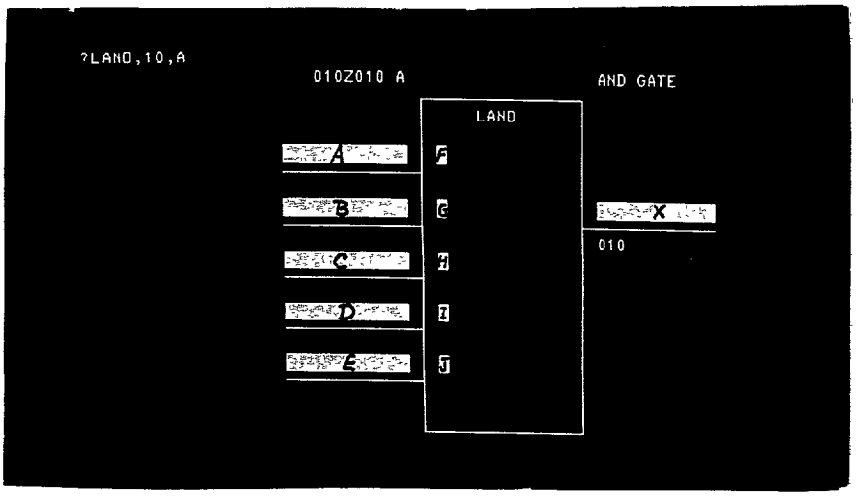
* Macro length is variable. These pointers not present if not input.

5. ADDITIONAL INFORMATION:

The first logic op. code must be an LDF or LDT. The last logic op. code must be an SST.

LAND

LOGIC AND OUTPUT



1. FUNCTION: Same as LAND but causes processor to schedule output via CSH on completion of this segment.

2. PARAMETERS:

IDENTIFICATION INPUT	FORMS	VARIABLE TYPE	DEFAULT
A	SN,N,S,V,G,A	LOGIC	①
B	SN,N,S,V,G,A	LOGIC	①
C	SN,N,S,V,G,A	LOGIC	②
D	SN,N,S,V,G,A	LOGIC	②
E	SN,N,S,V,G,A	LOGIC	②
DESCRIPTOR			
F	(INVERTER) OC	-	③
G	(INVERTER) OC	-	③
H	(INVERTER) OC	-	③
I	(INVERTER) OC	-	③
J	(INVERTER) OC	-	③
OUTPUT			
X	A	-	-

3. SPECIAL RESTRICTIONS:

① Required input

② Optional input(s).

③ Optional input(s). Default is +.

4. CBL MACRO INSTRUCTION FORMAT:

Type LOGIC Op. Code 8347 Name LOGIC AND OUTPUT

POS	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER	
	OPERATION CODE			
	POINTER TO NEXT CBL OP. CODE			
1	OP	POINTER TO LOGICAL A		
2	OP	POINTER TO LOGICAL B		
3	OP	POINTER TO LOGICAL D		*
4	OP	POINTER TO LOGICAL E		*
5	OP	POINTER TO LOGICAL F		*
6	OP	POINTER TO LOGICAL X		

LOGIC
OP

Maximum number of five logical inputs which may be inverted if desired. The following logical operations may be used:

HEX	LOGIC OP-CODE	NAME	DESCRIPTION OF OPERATION
2	0010	LAF	Logical AND stack with complement of input
3	0011	LAT	Logical AND stack with input
4	0100	LDF	Load stack with complement of input
5	0101	LDT	Load stack with input
6	0110	SST	Store stack into output

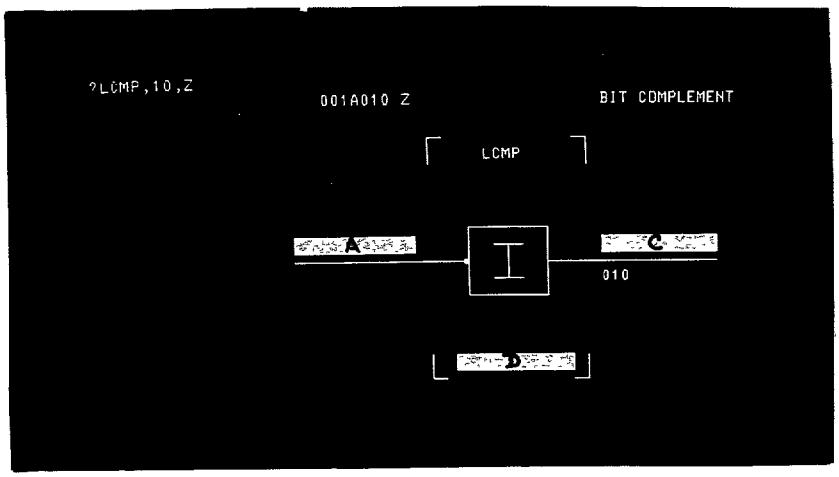
* Macro length is variable. These pointers not present if not input.

5. ADDITIONAL INFORMATION:

The first logic op. code must be an LDF or LDT.
The last logic op. code must be an SST.

The functions performed by this control block can also be performed by the LAND block.

LCMP LOGIC COMPLEMENT



1. FUNCTION:

$C = \overline{A}$

The logic output (bit) C is the complement (inverse) of the input.

2. PARAMETERS:

IDENTIFICATION	FORMS	VARIABLE TYPE	DEFAULT
INPUT			
A	SN,N,S,V,G,A	LOGIC	①
DESCRIPTOR			
B	SN,N,S,V,G,A	LOGIC	ENABLED ②
OUTPUT			
C	SN,N,A	LOGIC	

3. SPECIAL RESTRICTIONS:

① Required input

② Optional input

LCMP

4. CBL MACRO INSTRUCTION FORMAT:

Type LOGIC Op. Code 8B4D Name LOGIC COMPLEMENT

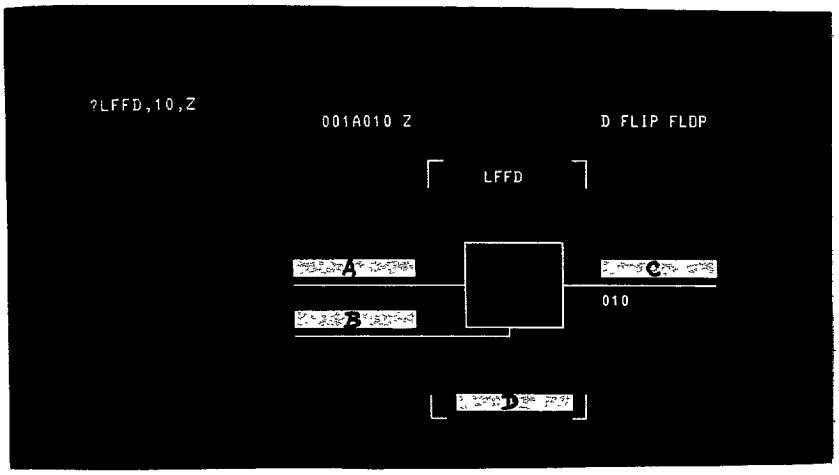
POS	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER
	OPERATION CODE		
	POINTER TO NEXT CBL OP. CODE		
1	POINTER TO INPUT LOGICAL A		
-2	POINTER TO OUTPUT LOGICAL C		

5. ADDITIONAL INFORMATION:

None

LFFD

LOGIC D TYPE FLIP-FLOP



1. FUNCTION:

If B = true, C = A
If B = false, C does not change

The logic output C is set equal to the logic input A if logic input B is true. If logic B is false, output C remains latched to its old value.

2. PARAMETERS:

IDENTIFICATION		VARIABLE	
INPUT	FORMS	TYPE	DEFAULT
A	SN,N,S,V,G,A	LOGIC	①
B	SN,N,S,V,G,A	LOGIC	①
DESCRIPTOR			
D	SN,N,S,V,G,A	LOGIC	ENABLED ②
OUTPUT			
C	SN,N,A	LOGIC	-

3. SPECIAL RESTRICTIONS:

① Required input

② Optional input

4. CBL MACRO INSTRUCTION FORMAT: LFFD

Type LOGIC Op Code 8B41 Name D FLIP-FLOP

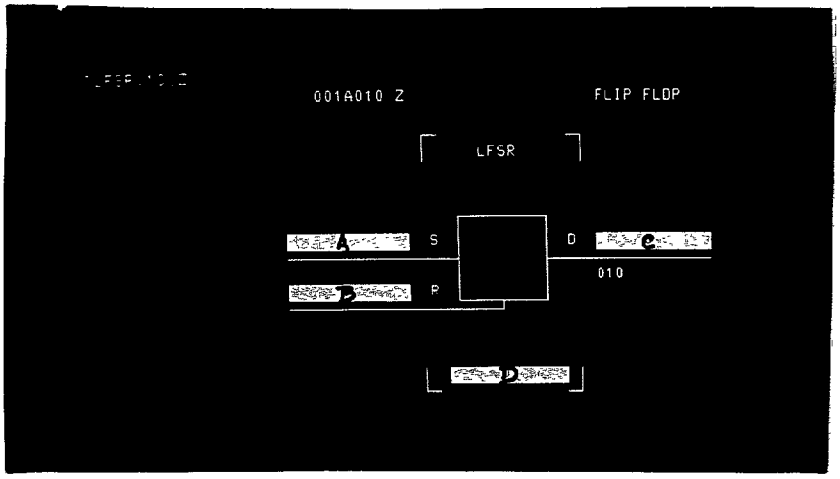
POS	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER
	OPERATION CODE		
	POINTER TO NEXT CBL OP. CODE		
1	POINTER TO INPUT LOGICAL "A"		
2	POINTER TO INPUT LOGICAL "B"		
3	POINTER TO OUTPUT LOGICAL "C"		

5. ADDITIONAL INFORMATION:

None

LFSR

LOGIC SET RESET FLIP-FLOP



1. FUNCTION:

Logic output C is set true when input A is true and B is false. C is cleared (false) when B is true and A is false. If A and B are both false, C remains in its old state.

2. PARAMETERS:

IDENTIFICATION INPUT	FORMS	VARIABLE TYPE	DEFAULT
A	SN,N,S,V,G,A	LOGIC	①
B	SN,N,S,V,G,A	LOGIC	①
DESCRIPTOR			
D	SN,N,S,V,G,A	LOGIC	ENABLED ②
OUTPUT			
C	SN,N,A	LOGIC	—

3. SPECIAL RESTRICTIONS:

① Required input

② Optional input

LFSR

4. CBL MACRO INSTRUCTION FORMAT:

Type LOGIC Op Code 8B42 Name SR FLIP-FLOP

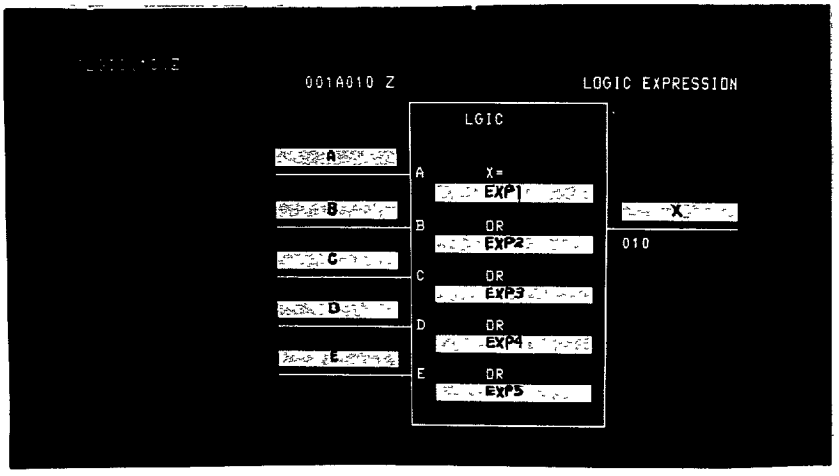
POS	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER
	OPERATION CODE		
	POINTER TO NEXT CBL OP. CODE		
1	POINTER TO INPUT LOGICAL "A"		
2	POINTER TO INPUT LOGICAL "B"		
3	POINTER TO OUTPUT LOGICAL "C"		

5. ADDITIONAL INFORMATION:

None

LGIC

LOGIC BOOLEAN OPERATOR



1. FUNCTION:

X = Logic OR of 5 logic expressions

The logic output X is a logic 1 if any of the 5 Boolean expressions (EXP1-EXP5) are satisfied. Allowable Booelan operators are:

1) ( ) Parentheses
2) - Complement (-A = $\overline{A}$)
3) • AND
4) + OR

An $\oplus$ exclusive OR can be generated with (A• -B) + (-A•B) $\Rightarrow$ A $\oplus$ B

Each expression is limited to 5 inputs (A-E) and 16 characters. The expressions will be independently evaluated from left to right, top to bottom.

2. PARAMETERS:

IDENTIFICATION INPUT	FORMS	VARIABLE TYPE	DEFAULT
A	SN,N,S,V,G,A	LOGIC	(1)
B	SN,N,S,V,G,A	LOGIC	(1)
C	SN,N,S,V,G,A	LOGIC	(2)
D	SN,N,S,V,G,A	LOGIC	(2)
E	SN,N,S,V,G,A	LOGIC	(2)

HEX	LOGICAL OP-CODE	NAME	DESCRIPTION OF EXTENDED OPERATION
0	0000	NOP	No operation
1	0001	SORT	Logical OR the second element in the stack
2	0010	SANT	Logical AND the second element in the stack with the first element
3	0011	SCPL	Complement the first element in the stack
4	0100	XTRU	Exit to the SST if the first element of stack is true
5	0101	XFAL	Exit to the SST if the first element of stack is false
6	0110	SHRT	Load stack short ***
7	0111	OPEN	Load stack open ***

A.M. Specifies an extended addressing mode. (Refer to Appendix C).

*** These two extended operator codes are applicable only to the ladder relay macro.

HEX	LOGICAL OP-CODE	NAME	DESCRIPTION OF EXTENDED OPERATION
0	0000	LOF	Logical OR complement of input with stack
1	0001	LOT	Logical OR input with stack
2	0010	LAF	Logical AND complement of input with stack
3	0011	LAT	Logical AND input with stack
4	0100	LDF	Load stack with complement of input
5	0101	LDT	Load stack with input
6	0110	SST	Store stack into output
7	0111	EOP	Extended operation code

Operand is a 12-bit field defined as follows:

ARGUMENT DESCRIPTION

FIELD IN WORD
LOGICAL OP CODE
EXTENDED OP CODE
ADDRESSING MODE
LOGICAL VARIABLE

WORD DESCRIPTION

OP	OPERAND	
15 12	11 4	3 0
LOP	WORD	BIT
LOP	OFFSET	EOC
LOP	OFFSET	A.M.
	WORD	BIT

WORD/BIT specifies a logical address. Offset is a word count from a reference point.

The following extended operation codes have been defined to manipulate logical data in the multi-level stack:

LGIC

DESCRIPTOR INPUT	FORMS		VARIABLE TYPE	DEFAULT
EXP1	(EXPRESSION)	OC 3	-	①
EXP2	(EXPRESSION)	OC 3	-	② None
EXP3	(EXPRESSION)	OC 3	-	② None
EXP4	(EXPRESSION)	OC 3	-	② None
EXP5	(EXPRESSION)	OC 3	-	② None
OUTPUT				
X	SN,N,A		LOGIC	-

3. SPECIAL RESTRICTIONS:

① Required input

② Optional input

③ Boolean Expressions

4. CBL MACRO INSTRUCTION FORMAT:

Type LOGIC Op Code 8B49 Name BOOLEAN OPERATOR

POS			
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER
	OPERATION CODE		
	POINTER TO NEXT CBL OP. CODE		
1	OP	OPERAND	
2	OP	OPERAND	
3	OP	OPERAND	
4	OP	OPERAND	
5	OP	OPERAND	
6	OP	OPERAND	
7	OP	OPERAND	
8	OP	OPERAND	
9	OP	OPERAND	
10	OP	OPERAND	*

* Length is variable

** Block size = size of block +1)/2
The first OP must be an LDF or LDT
The last OP must be an SST

5. ADDITIONAL INFORMATION:

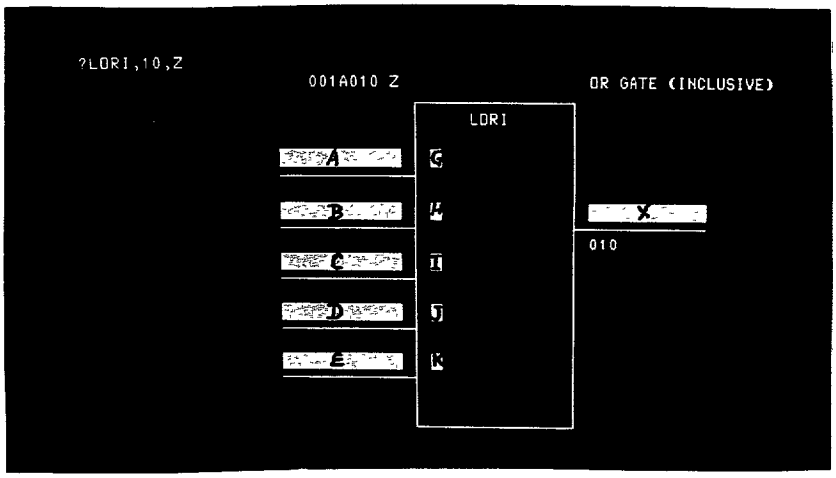
The logical operators (OP) may be used. They are common to both the LGIC and ladder relay control blocks.

5. ADDITIONAL INFORMATION:

The following logical operators (OP) may be used. They are common to both the LGIC and ladder relay control blocks.

LORI

LOGIC OR



1. FUNCTION:

The output (x) is the logical OR of the five input signals. If G, H, I, J or K contains a minus (-) or circle (o), its signal input sense is inverted before the OR function is performed.

2. PARAMETERS:

IDENTIFICATION INPUT	FORMS	VARIABLE TYPE	DEFAULT
A	SN,N,S,V,G,A	LOGIC	①
B	SN,N,S,V,G,A	LOGIC	①
C	SN,N,S,V,G,A	LOGIC	②
D	SN,N,S,V,G,A	LOGIC	②
E	SN,N,S,V,G,A	LOGIC	②
DESCRIPTOR			
G (INVERTER)	OC	-	③
H (INVERTER)	OC	-	③
I (INVERTER)	OC	-	③
J (INVERTER)	OC	-	③
K (INVERTER)	OC	-	③
OUTPUT			
X	SN,N,S,A	LOGIC	-

LORI

3. SPECIAL RESTRICTIONS:

① Required input

② Optional input(s). Default is none. Input required if descriptor is present.

③ Optional Inputs Default is +.

4. CBL MACRO INSTRUCTION FORMAT:

Type LOGIC Op Code 8B48 Name LOGIC INCLUSIVE OR

POS	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER	
	OPERATION CODE			
	POINTER TO NEXT CBL OP. CODE			
1	OP	POINTER TO LOGICAL "A"		
2	OP	POINTER TO LOGICAL "B"		
3	OP	POINTER TO LOGICAL "C"		*
4	OP	POINTER TO LOGICAL "D"		*
5	OP	POINTER TO LOGICAL "E"		*
6	OP	POINTER TO LOGICAL "X"		

A maximum of 5 logical inputs may be inverted if desired. The following logic operators may be used:

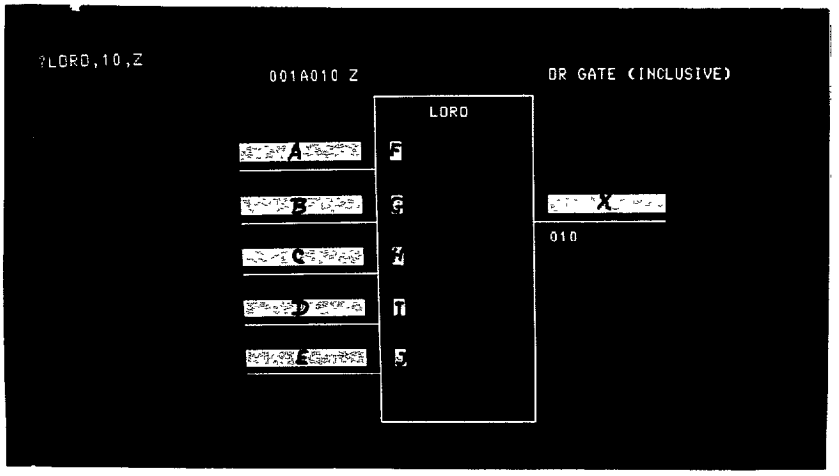
HEX	LOGIC OP-CODE	NAME	DESCRIPTION OF OPERATION
0	0000	LOF	Logical OR stack with complement of input
1	0001	LOT	Logical OR stack with input
4	0100	LDF	Load stack with complement of input
5	0101	LDT	Load stack with input
6	0110	SST	Store stack into output

* Length is variable. These pointers not present if not input.

The first logic op. code must be an LDF or LDT.
The last logic op. code must be an SST.

LORO

LOGIC OR INPUT (INCLUSIVE)



1. FUNCTION:

The output is the logical OR of the 5 input signals. If F, G, H, I or J contains a minus sign (-) or circle (o), its signal input sense is inverted before the OR function is performed. This function is the same as the LORI block except that this block causes the controller to schedule the CSH output address X = /CSGG/N on completion of the segment.

2. PARAMETERS:

IDENTIFICATION INPUT	FORMS	VARIABLE TYPE	DEFAULT
A	SN,N,S,V,G,A	LOGIC	①
B	SN,N,S,V,G,A	LOGIC	①
C	SN,N,S,V,G,A	LOGIC	②
D	SN,N,S,V,G,A	LOGIC	②
E	SN,N,S,V,G,A	LOGIC	②
DESCRIPTOR			
F (INVERTER)	OC	-	③
G (INVERTER)	OC	-	③
H (INVERTER)	OC	-	③
I (INVERTER)	OC	-	③
J (INVERTER)	OC	-	③
OUTPUT			
X	A	-	-

LORO

3. SPECIAL RESTRICTIONS:

① Required input

② Optional inputs. Default is none. Input is required if descriptor present.

③ Optional inputs. Default is +.

The functions performed by this block can also be performed by the LORI control block.

4. CBL MACRO INSTRUCTION FORMAT:

Type LOGIC Op. Code 8348 Name LOGIC INCLUSIVE OR OUTPUT

POS	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER	
	OPERATION CODE			
	POINTER TO NEXT CBL OP. CODE			
1	OP	POINTER TO LOGICAL "A"		
2	OP	POINTER TO LOGICAL "B"		
3	OP	POINTER TO LOGICAL "C"		*
4	OP	POINTER TO LOGICAL "D"		*
5	OP	POINTER TO LOGICAL "E"		*
6	OP	POINTER TO LOGICAL "X"		

LOGIC
OP

A maximum of 5 logical inputs may be inverted if desired. The following logic operators may be used:

HEX	LOGIC OP-CODE	NAME	DESCRIPTION OF OPERATION
0	0000	LOF	Logical OR stack with complement of input
1	0001	LOT	Logical OR stack with input
4	0100	LDF	Load stack with complement of input
5	0101	LDT	Load stack with input
6	0110	SST	Store stack into output

* Length is variable. These pointers not present if not input.

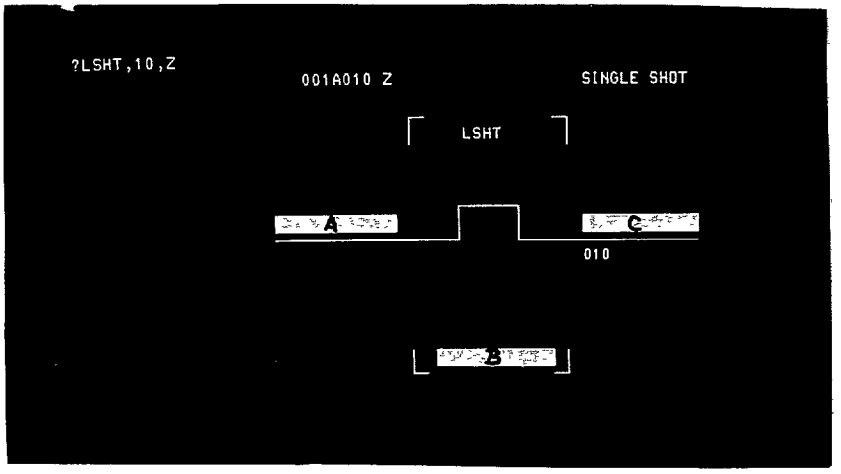
The first logic op. code must be an LDF or LDT. The last logic op. code must be an SST.

5. ADDITIONAL INFORMATION:

None

LSHT

LOGIC SINGLE SHOT



1. FUNCTION:

Logic output C will go true when input A goes from false to true. It will remain true until the next execution of this block when C will go false regardless of A.

2. PARAMETERS:

IDENTIFICATION	FORMS	VARIABLE TYPE	DEFAULT
INPUT			
A	SN,N,S,V,G,A	LOGIC	①
DESCRIPTOR			
D	SN,N,S,V,G,A	LOGIC	ENABLED ②
OUTPUT			
C	SN,N,A	LOGIC	-

3. SPECIAL RESTRICTIONS:

① Required input

② Optional input

4. CBL MACRO INSTRUCTION FORMAT:

Type LOGIC Op. Code 8B44 Name SINGLE SHOT

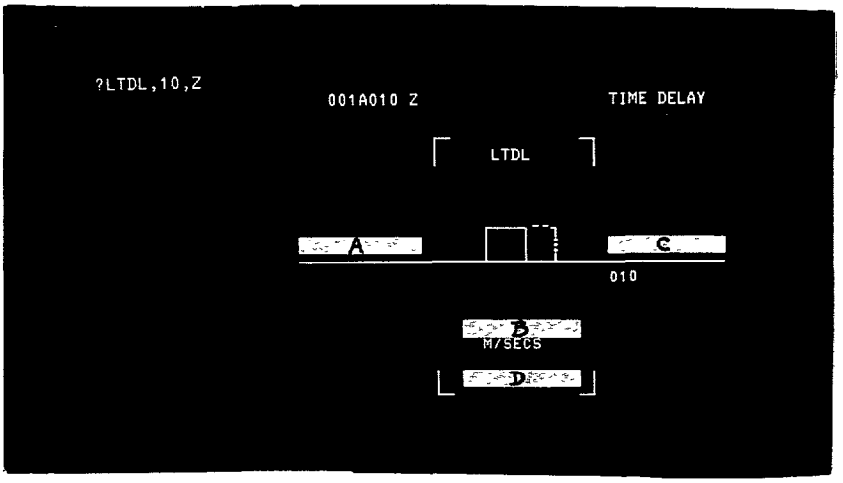
POS		
	M BLOCK SIZE	BLOCK SEQUENCE NUMBER
	OPERATION CODE	
	POINTER TO NEXT CBL OP. CODE	
1	POINTER TO INPUT LOCIAL "A"	
2	POINTER TO INTERMEDIATE LOGICAL	
3	POINTER TO OUTPUT LOGICAL "C"	

5. ADDITIONAL INFORMATION:

None

LTDL

LOGIC TIME DELAY



1. FUNCTION:

If logic input A goes from false to true, and remains true for the time specified by B, the output will go true and will return to false immediately when input A returns to false. If the input does not remain true for at least the period B, output C will remain false. Reset time for the timer is one execution. If B is not a multiple of the segment executation rate, B is rounded up to the next multiple of the segment executation rate.

2. PARAMETERS:

IDENTIFICATION	FORMS	VARIABLE TYPE	DEFAULT
INPUT			
A	SN,N,S,V,G,A	LOGIC	①
DESCRIPTOR			
B (DELAY TIME)	SN,N,S,V,G,A	COUNT	-
D	SN,N,S,V,G,A	LOGIC	ENABLED ②
OUTPUT			
C	SN,N,A	LOGIC	-

LTDL

3. SPECIAL RESTRICTIONS:

① Required input

② Optional input

4. CBL MACRO INSTRUCTION FORMAT:

Type LOGIC Op. Code 8B45 Name TIME DELAY

POS			
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER
	OPERATION CODE		
	POINTER TO NEXT CBL OP. CODE		
1	POINTER TO INPUT LOGICAL "A"		
2	POINTER TO TIME DELAY "B"		
3	POINTER TO ELAPSED TIME		*
4	POINTER TO EXECUTION RATE		**
5	POINTER TO OUTPUT LOGICAL "C"		

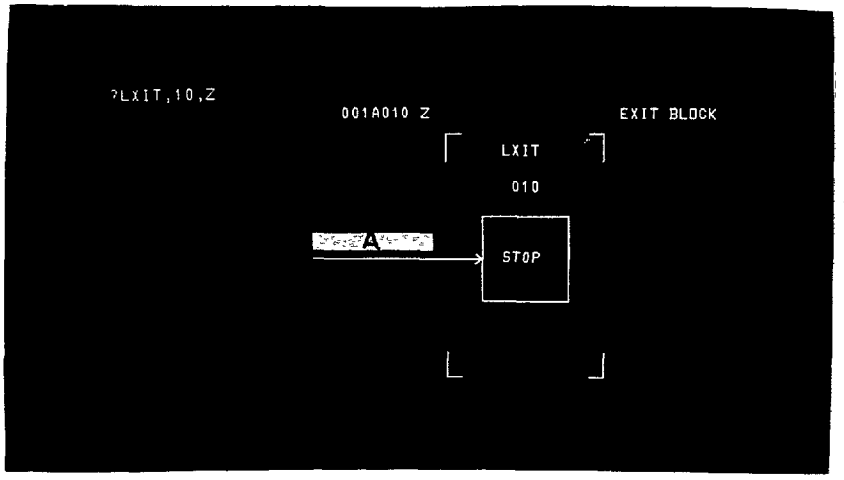
* Pointer to intermediate count
** Pointer to signal status block (SSB) variable (REPR) - repetition rate.

5. ADDITIONAL INFORMATION:

None

LXIT

LOGIC EXIT



1. FUNCTION:

If input A is a logic true signal, all further processing of blocks with higher sequence numbers is bypassed. However, all outputs normally done anywhere in the segment will still go out on the CSH. The next time the segment runs, processing will begin in its normal manner with the first block - lowest sequence number.

2. PARAMETERS:

IDENTIFICATION INPUT	FORMS	VARIABLE TYPE	DEFAULT
A	SN,N,S,V,G,A	LOGIC	①

DESCRIPTOR
None

OUTPUT
None

3. SPECIAL RESTRICTIONS:

This block can be used with regulating and sequencing (logic block type) segments.

① Required input

LXIT

4. CBL MACRO INSTRUCTION FORMAT:

Type LOGIC Op. Code 8B4A Name EXIT BLOCK

POS				
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER	
	OPERATION CODE			
	POINTER TO NEXT CBL OP. CODE			
I	POINTER TO INPUT LOGICAL "A"			
-2	8	3	9	6

FORM FOR BLOCK 1000

M	BLOCK SIZE		1000
8	3	4	A
F	F	F	F
0			
8	3	9	6

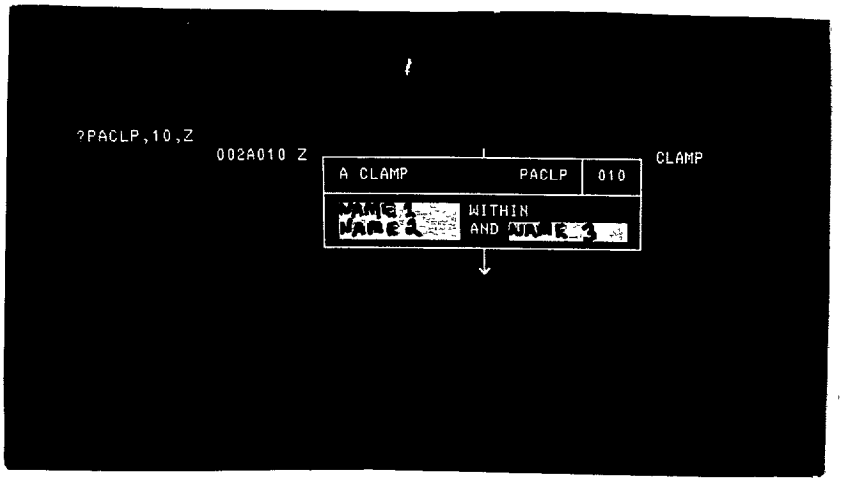
This block is inserted automatically at the end of each segment.

5. ADDITIONAL INFORMATION:

None

PACLP

CLAMP



1. FUNCTION:

NAME1 is clamped to within
NAME2 $\leq$ NAME1 $\leq$ NAME3

This is the same as the continuous block ACLP.

2. PARAMETERS:

IDENTIFICATION INPUTS	FORM	VARIABLE TYPE	DEFAULT
NAME1 - Value to be limit checked	S,V	ANALOG	①
NAME2 - High Clamp	N,S,V,C,G,SN	ANALOG	+1.7E38
NAME3 - Low Clamp	N,S,V,C,G,SN	ANALOG	-1.7E38
RESULTS			
NAME1 - Limit checked value	S,V	ANALOG	-

3. SPECIAL RESTRICTIONS:

① Required input

4. CBL MACRO INSTRUCTION FORMAT:

Type PROCEDURE Op Code 8A60 Name CLAMP

POS			
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER
	OPERATION CODE		
	POINTER TO NEXT CBL OP. CODE		
1	POINTER TO INPUT VALUE		*
2	POINTER TO HIGH CLAMP		
3	POINTER TO LOW CLAMP		
4	POINTER TO OUTPUT VALUE		*

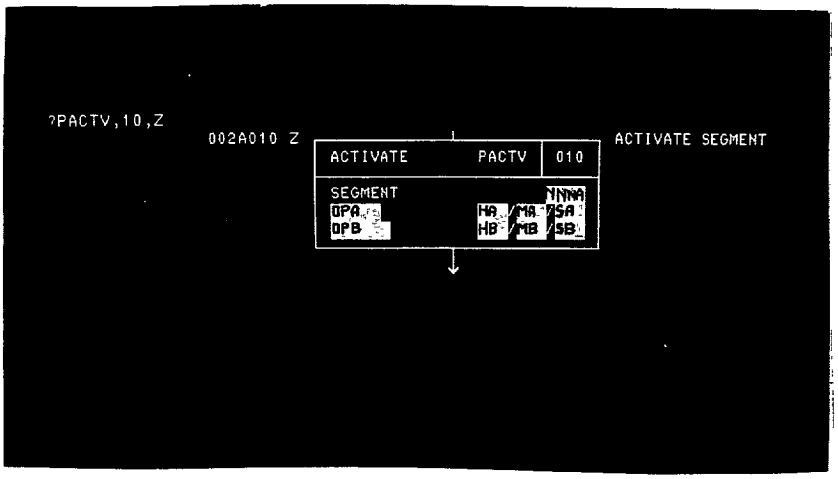
* Input and output pointers will both point to same value.

5. ADDITIONAL INFORMATION:

Flowchart in Appendix D.

PACTV

ACTIVATE SEGMENT



1. FUNCTION:

The procedure segment specified by NNNA is activated (turned on) after or at (OPA) time (HA/MA/SA). If OPB contains "period", the segment will be periodically turned on as specified by HB/MB/SB. If OPB is blank, segment specified will be turned on immediately. Time must be entered in form #HH/#MM/#SS where H, M, S are non-blank, and specify hours, minutes and seconds. An immediate activation occurs if no time entered and OPA equals "AFTER".

2. PARAMETERS:

INPUTS	FORMS	VARIABLE TYPE	DEFAULT
HA/MA/SA	C/C/C	COUNT	0/0/0
HB/MB/SB	C/C/C	COUNT	0/0/0
DESCRIPTOR			
NNNA	C	TEXT	(2)
OPA(1)	OC	-	(2)
OPB(1)	OC	-	-

RESULT

Segment specified by NNNA is activated.

3. SPECIAL RESTRICTIONS:

This block may activate only procedure type segments.

① OPA = "AFTER" or "AT"
OPB = "PERIOD" or blank

② Required input

4. CBL MACRO INSTRUCTION FORMAT:

Type PROCEDURAL Op Code 8A70 Name ACTIVATE SEGMENT

POS					
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER		
	OPERATION CODE				
	POINTER TO NEXT CBL OP. CODE				
1	9	B	9	1	
2	POINTER TO * + 9 (POSITION 11)				
3	POINTER TO REQUEST CODE (POSITION 13)				
4	POINTER TO SEGMENT NAME (POSITION 14)				
5	POINTER TO RESOLUTION CODE (POSITION 17)				
6	POINTER TO EXECUTION MULTIPLE (POSITION 18)				
7	POINTER TO HOURS (POSITION 19)				
8	POINTER TO MINUTES (POSITION 20)				
9	POINTER TO SECONDS (POSITION 21)				
10	POINTER TO 10's MSEC (POSITION 22)				
11	8	A	7	A	
12	F	F	F	3	
13	0	0	0	C	
14	P		N		
15	N		N		
16	A		b		
17	0	0	0	CODE	*
18	MULTIPLE				**
19	HA				
20	MA				
21	SA				
22	0	0	0	0	

NOTE: This format used if OPA is "AT".
* Code may be: 2 - SECONDS
3 - MINUTES
4 - HOURS
** Determined by HB/MB/SB

PACTV

POS					
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER		
	OPERATION CODE				
	POINTER TO NEXT CBL OP. CODE				
1	9	B	9	1	
2	POINTER TO * + 6 (POSITION 8)				
3	POINTER TO REQUEST CODE (POSITION 10)				
4	POINTER TO SEGMENT NAME (POSITION 11)				
5	POINTER TO RESOLUTION CODE (POSITION 14)				
6	POINTER TO EXECUTION MULTIPLE (POSITION 15)				
7	POINTER TO INITIAL TIME OFFSET (POSITION 16)				
8	8	A	7	A	
9	F	F	F	6	
10	0	0	0	C/A	****
11	P		N		
12	N		N		
13	A		Ø		
14	0	0	0	CODE	*
15	MULTIPLE				**
16	OFFSET				***

* Code may be: 2 - SECONDS
3 - MINUTES
4 - HOURS

** Units per code, value determined by HB/MB/SB
*** Units per code, value determined by HA/MA/SA
**** Request code is if immediate activation.

NOTE: This format used if OPA is "AFTER".

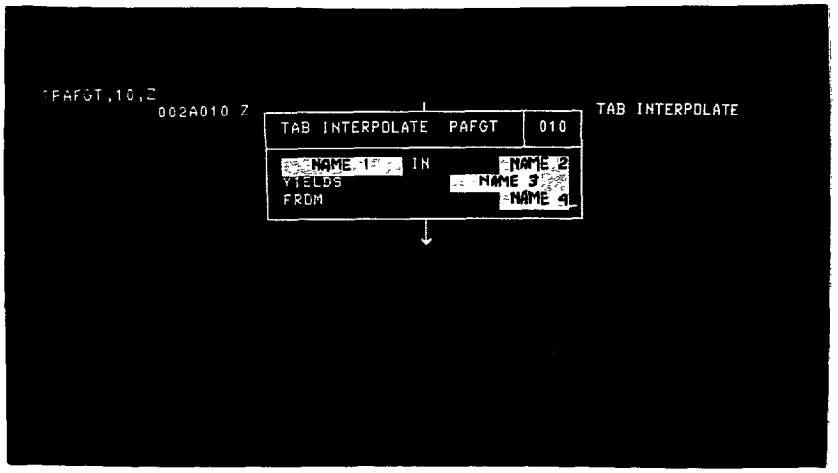
5. ADDITIONAL INFORMATION:

None

NOTES

PAFGT

FUNCTION GENERATOR (TABULAR)



1. FUNCTION:

NAME1 and NAME3 are analog signals. NAME2 and NAME4 are names of DATV/PDATV/INZV tables of equal length and containing analog values. The closest value to NAME1 found in table NAME2 is used to locate (in the same relative numbered position) a value in table NAME4 to provide the answer in NAME3.

2. PARAMETERS:

INPUTS	FORMS	VARIABLE TYPE	DEFAULT
NAME1	N,S,V,C,G	ANALOG	①
NAME2	AN	ANALOG	①
NAME4	AN	ANALOG	①
RESULTS			
NAME3	S,V	ANALOG	①

3. SPECIAL RESTRICTIONS:

Each vector must have the same number of elements.
Vectors must have values of increasing magnitude.

① Required input

4. CBL MACRO INSTRUCTION FORMAT:

Type PROCEDURAL Op Code 8A61 Name FUNCTION GENERATOR (TABULAR)

POS				
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER	
	OPERATION CODE			
	POINTER TO NEXT CBL OP. CODE			
1	POINTER TO INPUT VALUE "NAME1"			
2	POINTER TO INPUT TABLE "NAME2"			
3	0	0	0	0
4	POINTER TO INPUT TABLE "NAME4"			
5	POINTER TO RESULT "NAME3"			

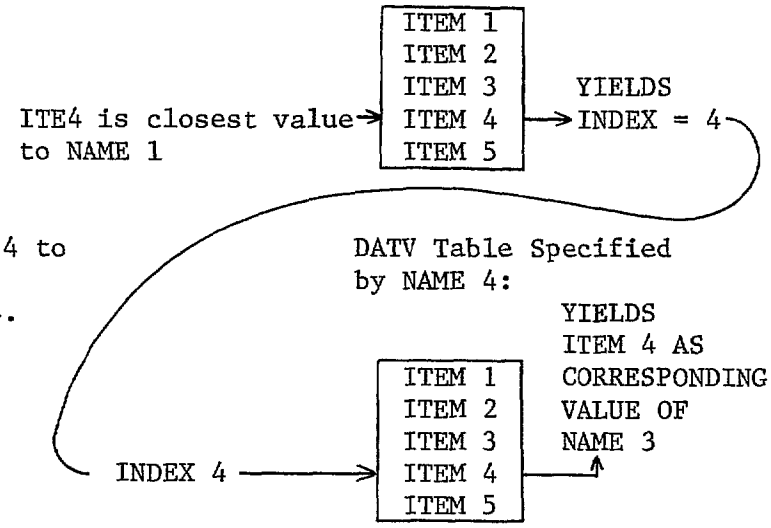
Note that position 3 must always contain zeroes.

5. ADDITIONAL INFORMATION:

Following is an example of PAFGT execution:

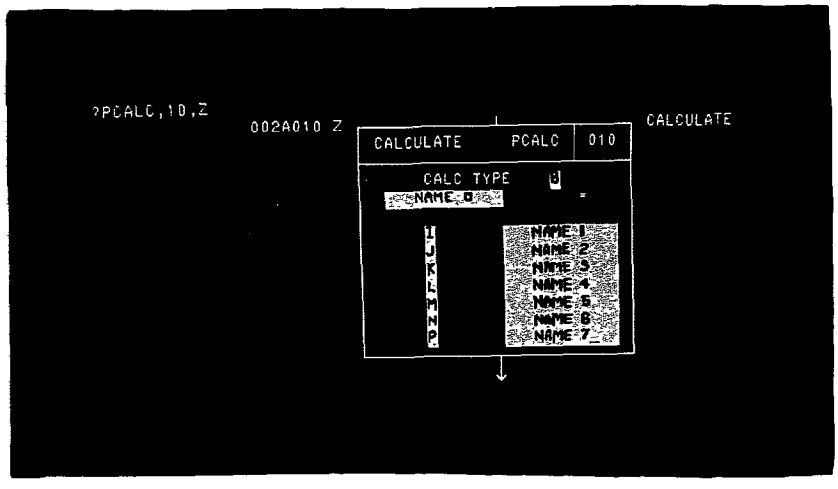
Find an item in the table specified by NAME2 that is closest to the value of NAME1

DATV Table Specified by NAME 2:



PCALC

CALCULATOR



1. FUNCTION:

Performs an equation on Names 1-7 according to operators in the operator fields (I-P):

+ (ADD)
- (SUBTRACT)
X (MULTIPLY)
/ (DIVIDE)

and puts answer in NAME0. The type of signal (A - analog or C - count) is indicated in field B. One to seven names and operators can be specified.

2. PARAMETERS:

INPUTS	FORMS	VARIABLE TYPE	DEFAULT
NAME1	SN,N,S,V,G,C	ANALOG/COUNT	(1)
NAME2	SN,N,S,V,C,G	ANALOG/COUNT	(1)
NAME3	SN,N,S,V,C,G	ANALOG/COUNT	(2)
NAME4	SN,N,S,V,C,G	ANALOG/COUNT	(2)
NAME5	SN,N,S,V,C,G	ANALOG/COUNT	(2)
NAME6	SN,N,S,V,C,G	ANALOG/COUNT	(2)
NAME7	SN,N,S,V,C,G	ANALOG/COUNT	(2)
I-P	OC	-	
DESCRIPTOR			
B	OC	-	-
RESULTS			
NAME0	S,V	-	-

PCALC

3. SPECIAL RESTRICTIONS:

All arguments and the result must be same signal type. Calculation is performed left to right (top to bottom in block diagram). Data type must be specified.

(1) Required input

(2) Optional input

4. CBL MACRO INSTRUCTION FORMAT:

Type PROCEDURAL Op Code 8A62 Name CALCULATOR

POS					
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER		
	OPERATION CODE				
	POINTER TO NEXT CBL OP. CODE				
1	OP1	OP2	OP3	OP4	
2	POINTER TO 1st ARGUMENT "NAME1"				
3	POINTER TO 2nd ARGUMENT "NAME2"				
4	POINTER TO 3rd ARGUMENT "NAME3"				
5	POINTER TO 4th ARGUMENT "NAME4"				
6	OP5	OP6	OP7	0	*
7	POINTER TO 5th ARGUMENT "NAME5"				
8	POINTER TO 6th ARGUMENT "NAME6"				
9	POINTER TO 7th ARGUMENT "NAME7"				
10	POINTER TO RESULT "NAME0"				**

Code OP1-OP7

COUNT	ANALOG	OPERATION
9	0	= This terminates the calculation
A	1	+
B	2	*
C	3	X
D	4	/

* Last "OP" must always be zero.

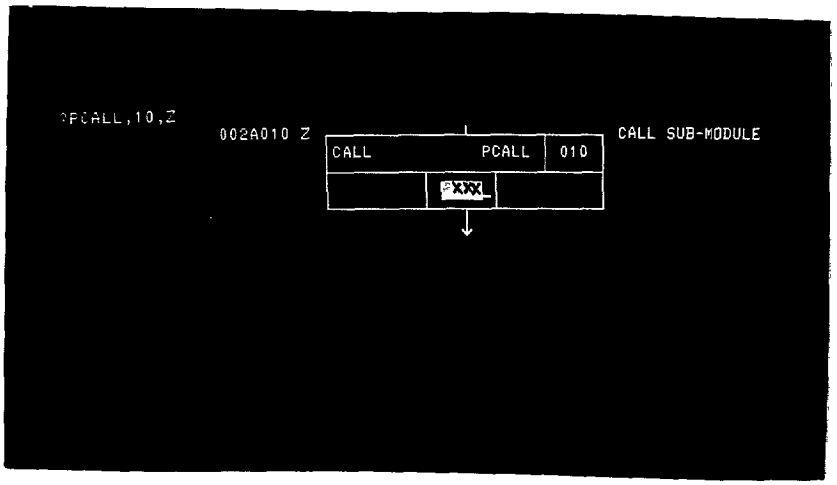
** This is a variable length instruction. Result pointer will always be last entry but its position will depend on number of arguments.

5. ADDITIONAL INFORMATION:

None

PCALL

CALL SUBMODULE



1. FUNCTION:

This is a request to the controller to transfer program control to submodule XXX within this segment. At completion of the sub-module, program control will return to the control block immediately following this one.

XXX is the block number of the first block in the submodule and must be a PSUBM block.

The effect is as if all the blocks of the named submodule had been executed in place of the PCALL block.

2. PARAMETERS:

DESCRIPTOR	FORM	VARIABLE TYPE	DEFAULT
XXX	BN	-	①

3. SPECIAL RESTRICTIONS:

XXX must be the number of a PSUBM block within the same segment. The range of XXX is 001-999.

① Required input

PCALL

4. CBL MACRO INSTRUCTION FORMAT:

Type PROCEDURAL Op Code 8A76 Name CALL SUBMODULE

POS			
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER
	OPERATION CODE		
	POINTER TO NEXT CBL OP..CODE		
I	POINTER TO SUBMODULE		*

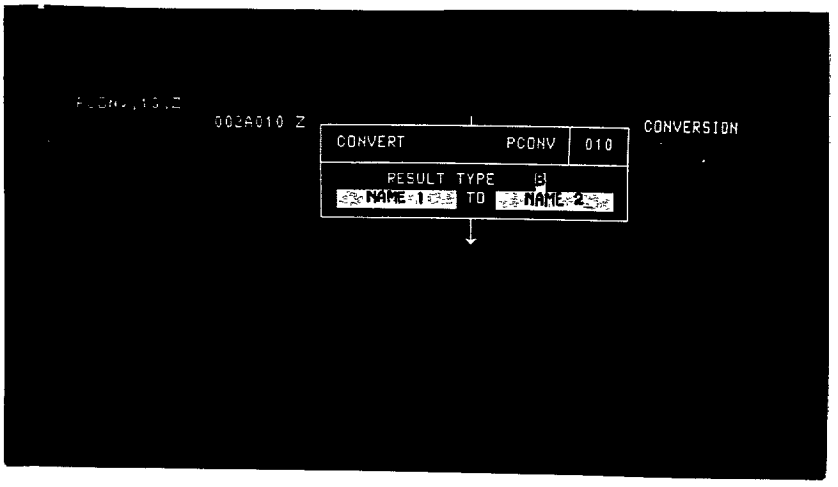
* Refer to description of PSUBM for detail of submodule address linkage.

5. ADDITIONAL INFORMATION:

None

PCONV

CONVERSION



1. FUNCTION:

The current value of input NAME1 (analog, count, text) is converted to NAME2 as specified by the content of field B;

A = analog or B = count.

2. PARAMETERS:

INPUT	FORMS	VARIABLE TYPE	DEFAULT
NAME1	N,S,V,C,G	ANALOG/COUNT/TEXT	①
DESCRIPTOR			
B	OC	-	①
RESULT			
NAME2	S,V	ANALOG/COUNT	①

3. SPECIAL RESTRICTIONS:

Data type of result is specified by code B as analog or count.

① Required input

4. CBL MACRO INSTRUCTION FORMAT:

Type PROCEDURAL Op Code 8A63 Name CONVERSION

POS					
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER		
	OPERATION CODE				
	POINTER TO NEXT CBL OP. CODE				
1	0	0	0	CODE	*
2	POINTER TO INPUT VALUE				
3	POINTER TO RESULT VALUE				

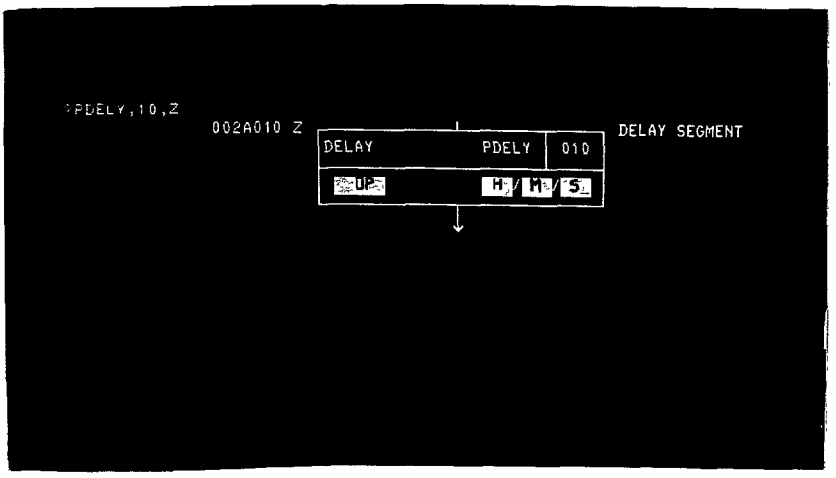
*CODE
0 Analog to count
2 Count to analog
4 Text to analog
7 Text to count

5. ADDITIONAL INFORMATION:

None

PDELY

DELAY SEGMENT



1. FUNCTION:

The execution of this segment will be delayed for time interval #H/#M/#S OR "UNTIL" time #H/#M/#S. It restarts at the next block after the delay. (PACTV can override).

"OP" may be FOR or UNTIL
H is hours (0-23)
M is minutes (0-59)
S is seconds (0-59)

2. PARAMETERS:

INPUTS	FORMS	VARIABLE TYPE	DEFAULT
H	OC	-	①
M	OC	-	①
S	OC	-	①
DESCRIPTOR			
OP	OC	-	①

RESULTS
Segment is delayed as specified

3. SPECIAL RESTRICTIONS:

A segment may only delay itself

① Required input

PDELY

4. CBL MACRO INSTRUCTION FORMAT:

Type PROCEDURAL Op Code 8A71 Name DELAY SEGMENT

POS					
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER		
	OPERATION CODE				
	POINTER TO NEXT CBL OP. CODE				
1	9	B	9	1	
2	POINTER TO * + 6 (POSITION 8)				
3	POINTER TO REQUEST CODE (POSITION 10)				
4	POINTER TO SEGMENT NAME (POSITION 11)				
5	POINTER TO RESOLUTION CODE (POSITION 12)				
6	POINTER TO EXECUTION MULTIPLE (POSITION 13)				
7	POINTER TO INITIAL TIME OFFSET (POSITION 14)				
8	8	A	7	6	
9	F	F	F	6	
10	0	0	0	C	
11	0	0	0	0	
12	0	0	0	CODE	**
13	0	0	0	0	
14	OFFSET				*

* Determined by H/M/S

** Code may be 2 (SECONDS), 3 (MINUTES) or 4 (HOURS)

NOTE: This format is used if OP = FOR.

PDELY

POS				
	M BLOCK SIZE BLOCK SEQUENCE NUMBER			
	OPERATION CODE			
	POINTER TO NEXT CBL OP. CODE			
1	9	B	9	1
2	POINTER TO * + 9 (POSITION 11)			
3	POINTER TO REQUEST CODE (POSITION 18)			
4	POINTER TO SEGMENT NAME (POSITION 19)			
5	POINTER TO RESOLUTION CODE (POSITION 20)			
6	POINTER TO EXECUTION MULTIPLE (POSITION 21)			
7	POINTER TO HOURS (POSITION 22)			
8	POINTER TO MINUTES (POSITION 23)			
9	POINTER TO SECONDS (POSITION 24)			
10	POINTER TO 10's MSECS (POSITION 25)			
11	9	B	9	1
12	POINTER TO * + 4 (POSITION 16)			
13	POINTER TO REQUEST CODE (POSITION 26)			
14	POINTER TO SEGMENT NAME (POSITION 27)			
15	POINTER TO TERMINATION TYPE (POSITION 28)			
16	8	A	7	6
17	F	F	E	E
18	0	0	0	C
19	0	0	0	0
20	0	0	0	0
21	0	0	0	0
22	H			
23	M			
24	S			
25	0	0	0	0
26	0	0	0	6
27	0	0	0	0
28	0	0	0	1

NOTE: This format is used if OP = UNTIL.

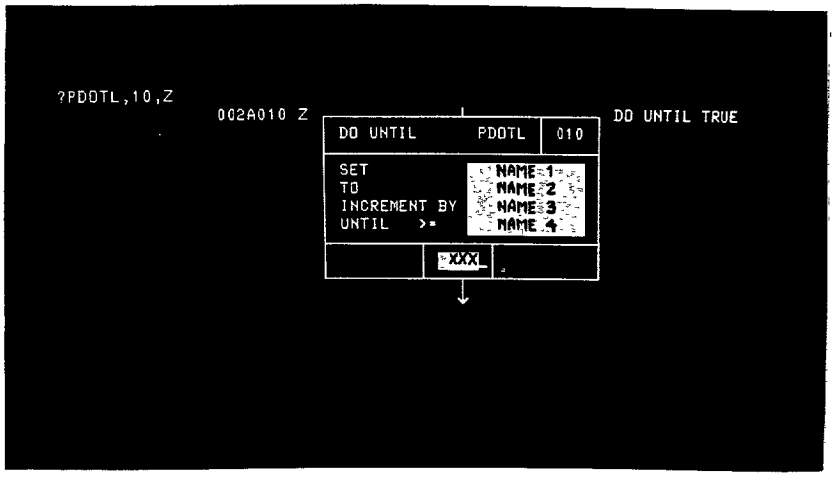
5. ADDITIONAL INFORMATION:

None

NOTES

PDOTL

DO UNTIL TRUE



1. FUNCTION:

This procedure block permits looping in a procedural program (via a submodule XXX) for an indexed number of times. NAME1 is set equal to NAME2 on block entry. Then submodule XXX is executed. Then NAME1 is incremented by NAME3. If NAME1 now exceeds or equals NAME4, looping is finished; if not, submodule XXX is again executed and NAME1 is incremented. This repeats until NAME1 equals/exceeds NAME4 before going on to the next block.

2. PARAMETERS:

INPUTS	FORMS	VARIABLE TYPE	DEFAULT
NAME1	S,V	COUNT	(1)
NAME2	SN,N,S,V,C,G	COUNT	(1)
NAME3	SN,N,S,V,C,G	COUNT	(1)
NAME4	SN,N,S,V,C,G	COUNT	(1)
XXX	BN		

RESULTS

Execution of the submodule repeated as often as specified.

3. SPECIAL RESTRICTIONS:

XXX must be the number of a PSUBM block within the same segment.

① Required input

4. CBL MACRO INSTRUCTION FORMAT:

Type PROCEDURAL Op Code 8A7D Name DO UNTIL TRUE

POS				
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER	
	OPERATION CODE			
	POINTER TO NEXT CBL OP. CODE			
1	POINTER TO RUNNING COUNT "NAME1"			
2	POINTER TO INITIAL VALUE "NAME2"			
3	POINTER TO INCREMENT VALUE "NAME3"			
4	POINTER TO FINAL VALUE "NAME4"			
5	POINTER TO SUBMODULE "XXX"			*
6	8	A	7	E

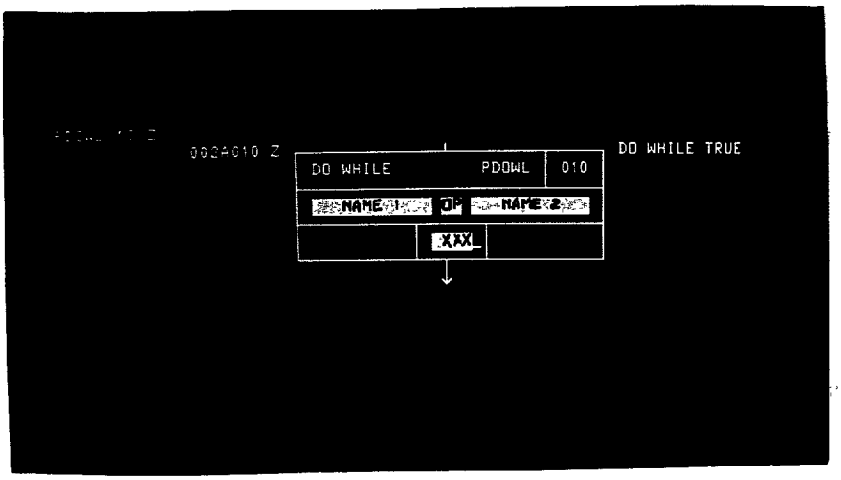
* Refer to description of PSUBM for detail of submodule address linkage.

5. ADDITIONAL INFORMATION:

Flowchart in Appendix D.

PDOWL

DO WHILE TRUE



1. FUNCTION:

Submodule XXX is executed repeatedly until the relation specified by OP between NAME1 and NAME2 goes false.

OP:
- = equal
- > greater than
- < less than
- >= or => greater than or equal
- <= or =< less than or equal
- <> not equal

Names must be the same type.
Types may be analog count, logic or text (A, C, L, T)
For text values, slash characters and corresponding characters in other values are ignored in the comparison.
For logic signals, only "=" is valid.

2. PARAMETERS:

INPUTS	FORMS	VARIABLE TYPE	DEFAULT
NAME1	SN,N,S,V,C,G	A/C/L/T	①
NAME2	SN,N,S,V,C,G	A/C/L/T	①
XXX	BN	-	NEXT BLOCK
DESCRIPTOR			
OP	OC	-	①

RESULTS

Execution of the submodule repeated as often as specified.

3. SPECIAL RESTRICTIONS:

XXX must be the number of a PSUBM block within the same segment.

① Required input

4. CBL MACRO INSTRUCTION FORMAT:

Type PROCEDURAL Op Code 8A7F Name DO WHILE TRUE

POS				
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER	
	OPERATION CODE			
	POINTER TO NEXT CBL OP. CODE			
1	0	0	CODE	TYPE
2	POINTER TO 1st ARGUMENT "NAME1"			
3	POINTER TO 2nd ARGUMENT "NAME2"			
4	POINTER TO SUBMODULE "XXX"			*

TYPE	CODE	COMPARISON
0 - Analog	0 -	=
1 - Count	1 -	<>
2 - Logic	2 -	<
3 - Text	3 -	>
	4 -	> =
	5 -	< =

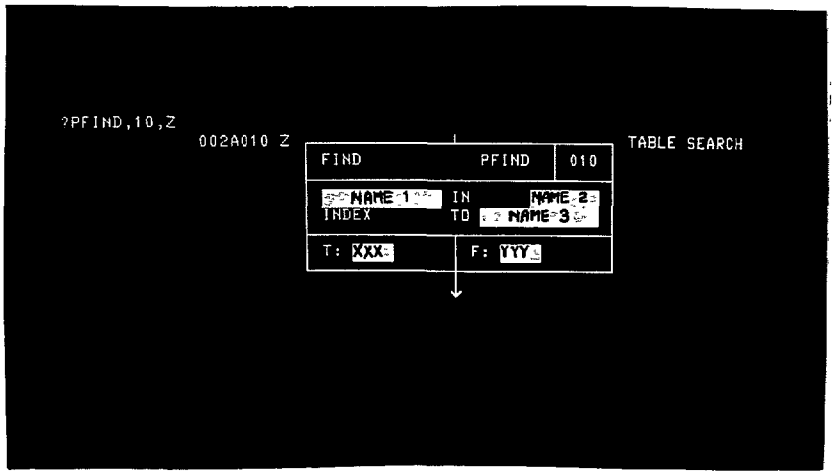
* Refer to description of PSUBM for detail of submodule address linkage.

5. ADDITIONAL INFORMATION:

A flow chart appears in Appendix D.

PFIND

TABLE SEARCH



1. FUNCTION:

NAME1 is compared to values in table NAME2 (DATV/PDATV/INZV) until a value greater or equal to is found or the end of the table occurs. The table must contain increasing values from top to bottom. If a value is found, NAME3 is set to its position (INDEX) value and submodule XXX is executed; otherwise, NAME3 = maximum table size and submodule YYY is executed.

2. PARAMETERS:

INPUTS	FORMS	VARIABLE TYPE	DEFAULT
NAME1	N,S,V,C,G	ANALOG/TEXT	①
NAME2	AN	ANALOG/TEXT	①
XXX	BN	-	NEXT BLOCK
YYY	BN	-	NEXT BLOCK
RESULTS			
NAME3	S,V	COUNT	①

3. SPECIAL RESTRICTIONS:

NAME2 and NAME1 must be same type

① Required input

PFIND

4. CBL MACRO INSTRUCTION FORMAT:

Type PROCEDURAL Op Code 8A64 Name TABLE SEARCH

POS					
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER		
	OPERATION CODE				
	POINTER TO NEXT CBL OP. CODE				
1	0	0	0	CODE	
2	POINTER TO INPUT "NAME1"				
3	POINTER TO TABLE "NAME2"				
4	POINTER TO INDEX "NAME3"				
5	POINTER TO SUBMODULE XXX				*
6	POINTER TO SUBMODULE YYY				*

CODE
0 - Analog
1 - Text

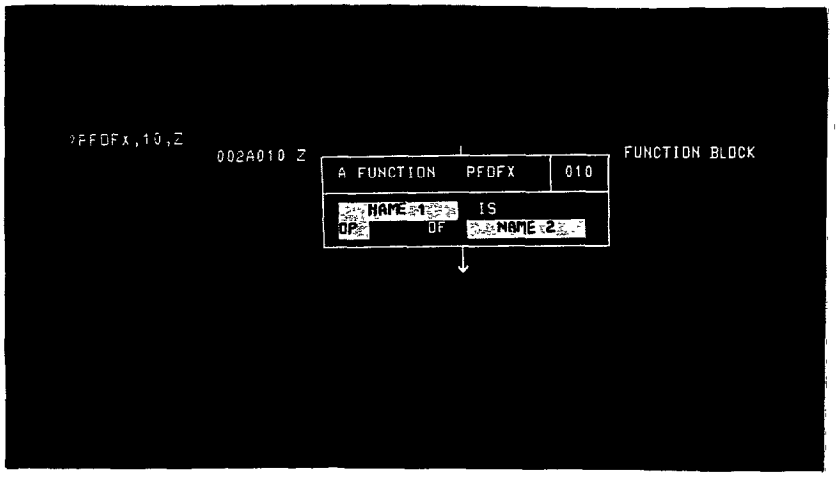
* Refer to description of PSUBM for detail of submodule address linkage.

5. ADDITIONAL INFORMATION:

A flow chart appears in Appendix D.

PFOFX

FUNCTION OF INPUT



1. FUNCTION:

The function specified by OP is performed on NAME2 to get NAME1. Both are analog signals.

OP is: SQR - square root
ABS - absolute value
EXP - e^x
LNX - ln x
COS - cos x
SIN - sin x
ATN - arc tan x

2. PARAMETERS:

INPUTS	FORMS	VARIABLE TYPE	DEFAULT
NAME2	SN,N,S,V,C,G	ANALOG	①
DESCRIPTOR			
OP	OC	-	①
RESULTS			
NAME1	S,V	ANALOG	①

3. SPECIAL RESTRICTIONS:

Only analog variables allowed

① Required input

PFOFX

4. CBL MACRO INSTRUCTION FORMAT:

Type PROCEDURAL Op Code 8A65 Name FUNCTION OF INPUT

POS	M	BLOCK SIZE		BLOCK SEQUENCE NUMBER
	OPERATION CODE			
	POINTER TO NEXT CBL OP. CODE			
1	POINTER TO ARGUMENT "NAME2"			
2	POINTER TO RESULT "NAME1"			
3	POINTER TO INTERMDIATE VALUE			
4	0	0	0	CODE 1
5	9	B	9	CODE 2
6	8	3	9	7
7	8	B	9	7

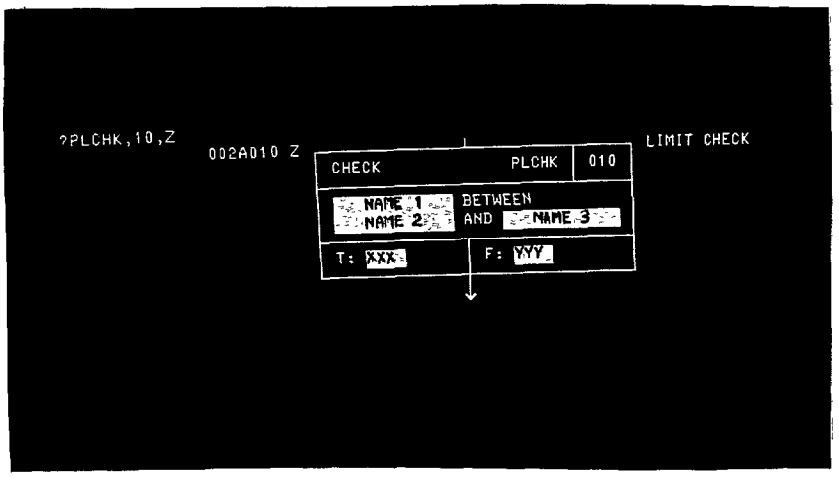
CODE 1	FUNCTION	CODE 2
0	Square root	0
1	Absolute	0
2	Exp	D
3	Ln	8
4	Cos	A
5	Sin	9
6	Arc Tan	B

5. ADDITIONAL INFORMATION:

None

PLCHK

LIMIT CHECK



1. FUNCTION:

If NAME1 is within (between) high limit NAME2 and low limit NAME3, i.e.,

NAME2 < NAME1 < NAME3 submodule XXX is executed (just once). Otherwise if NAME1 < NAME2 or NAME1 > NAME3, submodule YYY is executed (just once). Then the next block is executed.

2. PARAMETERS:

INPUTS	FORMS	VARIABLE TYPE	DEFAULT
NAME1	SN,N,S,V,C,G	ANALOG/COUNT	(1)
NAME2	SN,N,S,V,C,G	ANALOG/COUNT	(1)
NAME3	SN,N,S,V,C,G	ANALOG/COUNT	(1)
XXX	BN	-	NEXT BLOCK
YYY	BN	-	NEXT BLOCK

DESCRIPTOR
None

RESULTS
Module XXX or YYY will be executed.

3. SPECIAL RESTRICTIONS:

NAME1, NAME2 and NAME3 must all be same type. XXX and YYY must be the number of a PSUBM block within the same segment.

① Required input

4. CBL MACRO INSTRUCTION FORMAT:

Type PROCEDURAL Op Code 8A69 Name LIMIT CHECK

POS					
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER		
	OPERATION CODE				
	POINTER TO NEXT CBL OP. CODE				
1	0	0	0	TYPE	
2	POINTER TO VALUE "NAME1"				
3	POINTER TO HIGH LIMIT "NAME2"				
4	POINTER TO LOW LIMIT "NAME3"				
5	POINTER TO SUBMODULE "XXX"				*
6	POINTER TO SUBMODULE "YYY"				*

TYPE
0 - Analog
1 - Count

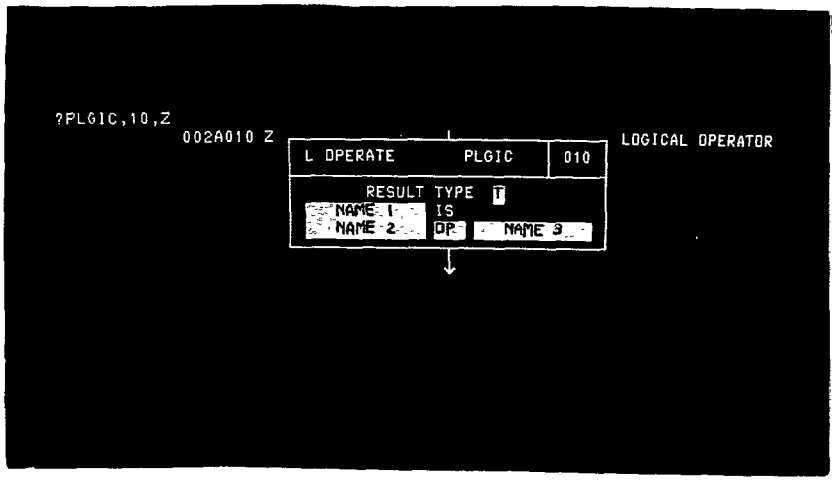
* Refer to description of PSUBM for detail of submodule address linkage.

5. ADDITIONAL INFORMATION:

A flow chart appears in Appendix D.

PLGIC

LOGICAL OPERATOR



1. FUNCTION:

NAME1 is equal to the logical OP operation on NAME2 and NAME3. NAMES1-3 may all be count or all may be logical bit - type names as specified by "result type".

"T" may be:
C = count (16 bits)
L = logical (1 bit)

"OP" may be:
AND = logical AND
OR = inclusive OR
EOR = exclusive OR

2. PARAMETERS:

INPUTS	FORMS	VARIABLE TYPE	DEFAULT
NAME2	SN,N,S,V,C,G	COUNT/LOGIC	(1)
NAME3	SN,N,S,V,C,G	COUNT/LOGIC	(1)
DESCRIPTOR			
T	OC	-	(1)
OP	OC	-	(1)
RESULTS			
NAME1	S,V	COUNT/LOGIC	(1)

3. SPECIAL RESTRICTIONS:

NAME1, NAME2, NAME3 must all be type specified by T field.

① Required input

4. CBL MACRO INSTRUCTION FORMAT:

Type PROCEDURAL Op Code 8A77 Name LOGICAL OPERATIONS

POS				
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER	
	OPERATION CODE			
	POINTER TO NEXT CBL OP. CODE			
1	0	0	CODE	TYPE
2	POINTER TO 1st ARGUMENT "NAME2"			
3	POINTER TO 2nd ARGUMENT "NAME3"			
4	POINTER TO RESULT "NAME1"			

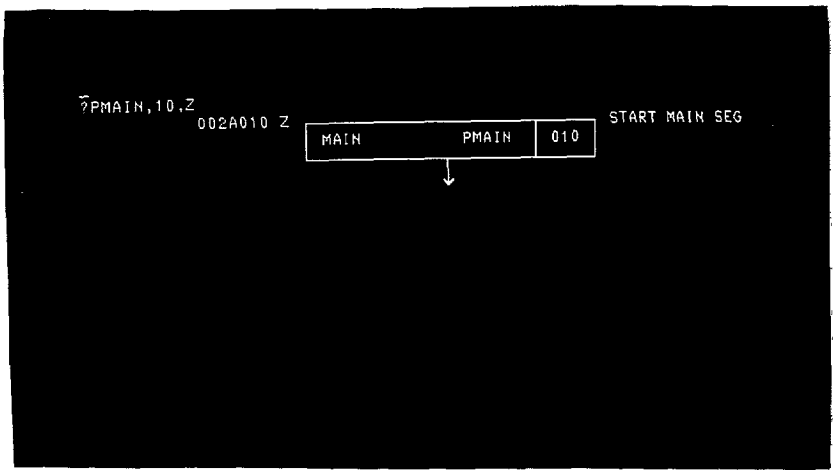
TYPE	CODE
0 - Count	0 - AND
1 - Logic	1 - OR
	2 - EOR

5. ADDITIONAL INFORMATION:

None

PMAIN

BEGINNING OF MAIN MODULE



1. FUNCTION:

No action; just identifies beginning of segment. There must be one, and only one PMAIN for every procedural segment.

2. PARAMETERS:

None

3. SPECIAL RESTRICTIONS:

None

4. CBL MACRO INSTRUCTION FORMAT:

Type PROCEDURAL Op Code 8A72 Name BEGINNING OF MAIN MODULE

POS	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER
	OPERATION CODE		
	POINTER TO NEXT CBL. OP. CODE		

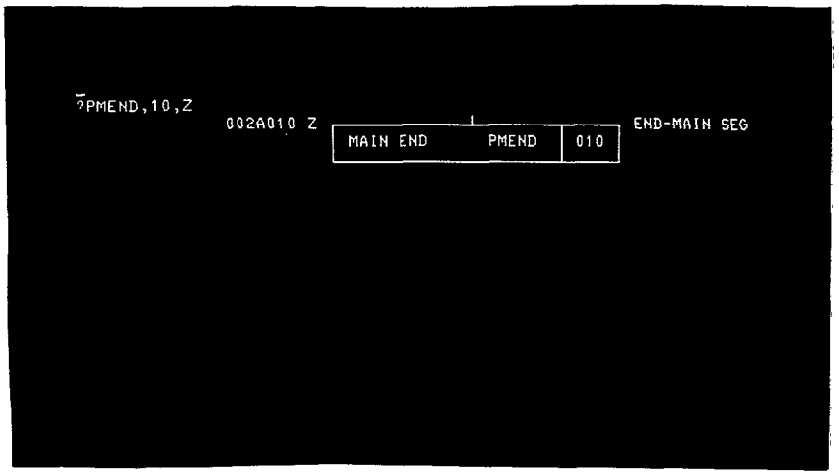
5. ADDITIONAL INFORMATION:

None

NOTES

PMEND

END OF MAIN MODULE



1. FUNCTION:

This block identifies the effective end of the segment. It results in a return to the system executive.

2. PARAMETERS:

None

3. SPECIAL RESTRICTIONS:

None

4. CBL MACRO INSTRUCTION FORMAT:

Type PROCEDURAL Op Code 8A73 Name END OF MAIN MODULE

POS				
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER	
	OPERATION CODE			
	POINTER TO NEXT CBL OP. CODE			
1	9	B	9	1
2	POINTER TO * + 2 (POSITION 4)			
3	POINTER TO REQUEST CODE (POSITION 5)			
4	0	0	0	0
5	0	0	0	6

PMEND

FOR BLOCK 1000

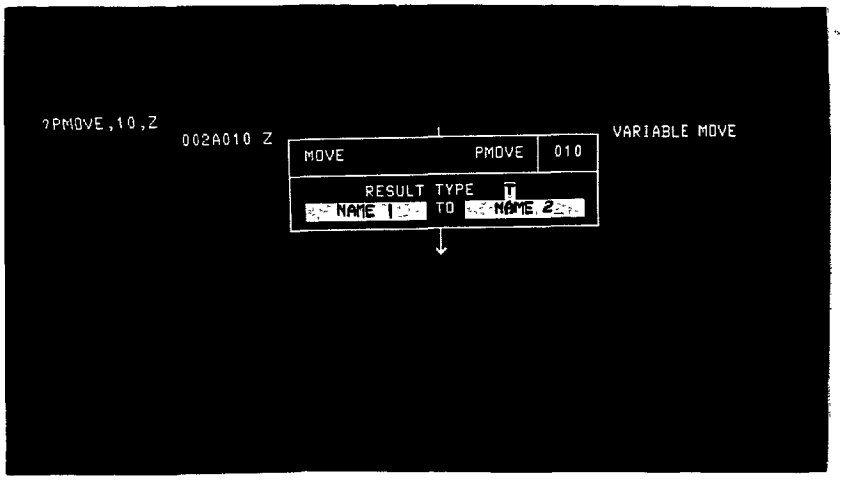
	M	BLOCK SIZE		1000
	OPERATION CODE			
	-1			
1	9	B	9	1
2	POINTER TO **2 (POSITION 4)			
3	POINTER TO REQUEST CODE (POS 5)			
4	0			
5	6			

5. ADDITIONAL INFORMATION:

None

PMOVE

MOVE VARIABLE



1. FUNCTION:

Moves value of NAME1 to NAME2.
Type of value is specified as:

A - Analog
C - Count
L - Logic
T - Text

2. PARAMETERS:

INPUT	FORMS	VARIABLE TYPE	DEFAULT
NAME1	SN,N,S,V,C,G	ANALOG/COUNT LOGIC/TEXT	①
DESCRIPTOR			
T	OC	-	①
RESULT			
NAME2	S,V	ANALOG/COUNT LOGIC/TEXT	①

3. SPECIAL RESTRICTIONS:

Input and output signals must be same type. NAME1 and NAME2 must be type-specified by T.

① Required input

PMOVE

4. CBL MACRO INSTRUCTION FORMAT:

Type PROCEDURAL Op Code 8A66 Name MOVE VARIABLE

POS					
	M	BLOCK SIZE		BLOCK SEQUENCE NUMBER	
	OPERATION CODE				
	POINTER TO NEXT CBL OP. CODE				
1	0	0	0	TYPE	
2	POINTER TO SOURCE "NAME1"				
3	POINTER TO DESTINATION "NAME2"				

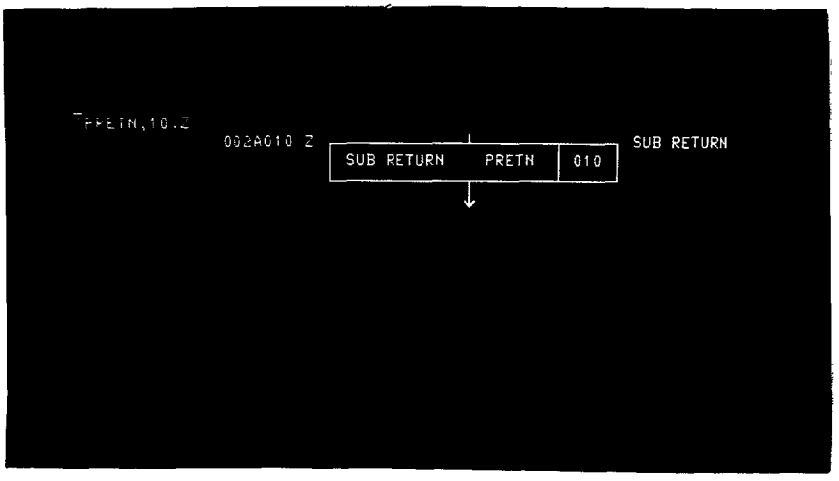
TYPE
- 0 - Analog
- 1 - Count
- 2 - Logic
- 3 - Text

5. ADDITIONAL INFORMATION:

None

PRETN

RETURN FROM SUBMODULE



1. FUNCTION:

This block identifies the end of a submodule and causes return to calling module.

2. PARAMETERS:

None

3. SPECIAL RESTRICTIONS:

None

4. CBL MACRO INSTRUCTION FORMAT:

Type PROCEDURAL Op Code 8A78 Name RETURN FROM SUBMODULE

POS	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER	
	OPERATION CODE			
	POINTER TO NEXT CBL OP. CODE			
1	1	POINTER TO "PSUBM" LINK		*

* This is an indirect pointer to the return address linkage pointer within the PSUBM block. Refer to description of PSUBM for detail.

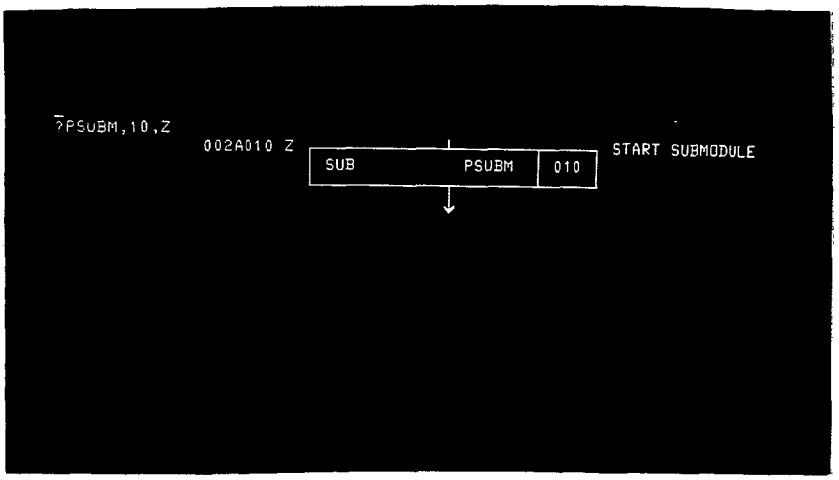
5. ADDITIONAL INFORMATION:

None

NOTES

PSUBM

BEGINNING OF SUBMODULE



1. FUNCTION:

No action. This block only identifies the beginning of the submodule.

2. PARAMETERS:

None

3. SPECIAL RESTRICTIONS:

None

4. CBL MACRO INSTRUCTION FORMAT:

Type PROCEDURAL Op Code 8A79 Name BEGINNING OF SUBMODULE

POS			
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER
	OPERATION CODE		
	POINTER TO NEXT CBL OP. CODE		
1	POINTER TO SUBMODULE RETURN ADDRESS		*

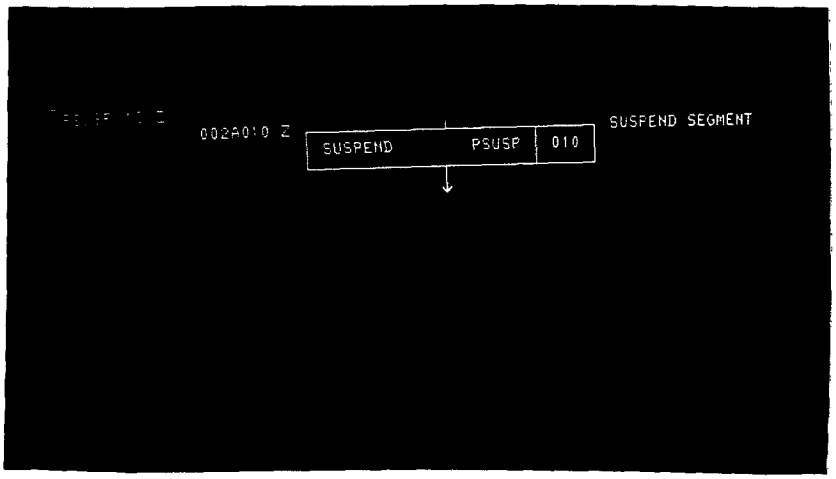
* This is the pointer to the address of the next CBL macro to be executed upon exit from this submodule. The return address is stored at run time by execution of the submodule call - PCALL.

5. ADDITIONAL INFORMATION:

A flow chart appears in Appendix D.

PSUSP

SUSPEND SEGMENT



1. FUNCTION: Execution of this segment is suspended until activated by another segment. Restart will be to next block.

NOTE
A segment may only suspend itself.

2. PARAMETERS:

None

3. SPECIAL RESTRICTIONS:

None

PSUSP

4. CBL MACRO INSTRUCTION FORMAT:

Type PROCEDURAL OP Code 8A74 Name SUSPEND SEGMENT

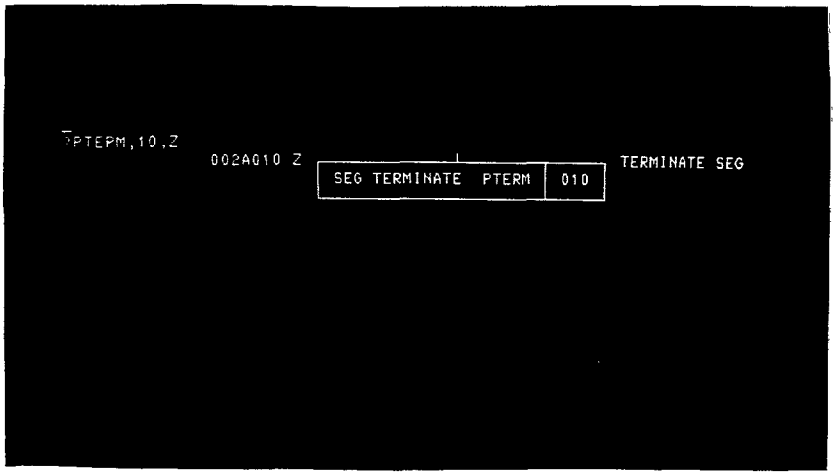
POS				
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER	
	OPERATION CODE			
	POINTER TO NEXT CBL OP. CODE			
1	9	B	9	1
2	POINTER TO * + 4 (POSITION 6)			
3	POINTER TO REQUEST CODE (POSITION 8)			
4	POINTER TO SEGMENT NAME (POSITION 9)			
5	POINTER TO TERMINATION TYPE (POSITION 10)			
6	8	A	7	C
7	F	F	F	8
8	0	0	0	6
9	0	0	0	0
10	0	0	0	1

5. ADDITIONAL INFORMATION:

None

PTERM

TERMINATE SEGMENT



1. FUNCTION:

This block allows segment termination within a submodule. The effect is same as PMEND block.

2. PARAMETERS:

None

3. SPECIAL RESTRICTIONS:

None

4. CBL MACRO INSTRUCTION FORMAT:

Type PROCEDURAL Op Code 8A75 Name TERMINATE SEGMENT

POS	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER	
	OPERATION CODE			
	POINTER TO NEXT CBL OP. CODE			
1	POINTER TO "PMEND" OP. CODE			*

* This is pointer to special PMEND that is not accessible to user.

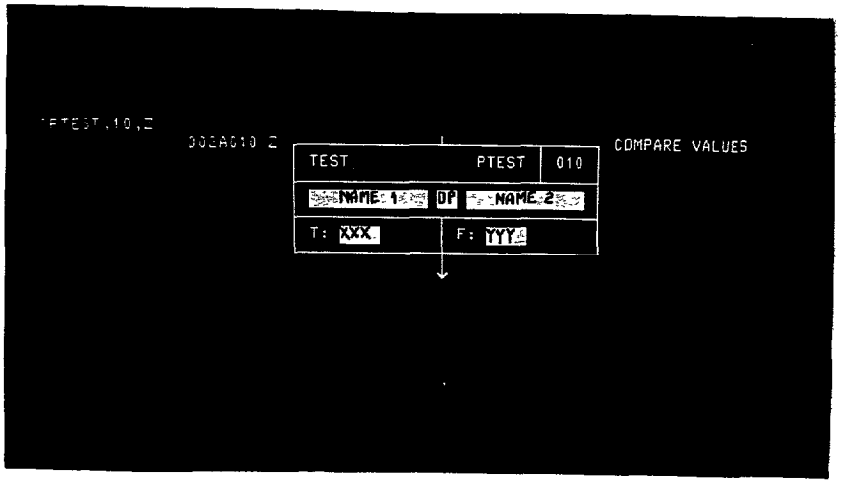
5. ADDITIONAL INFORMATION:

None

NOTES

PTEST

COMPARE VALUES



1. FUNCTION:

The value of NAME1 is compared to the value of NAME2. NAME1 and 2 are the same type - analog, count, logic or text. If the result of the comparison is true, submodule XXX is executed. If false, submodule YYY is executed.

OP: = Equal
<> Not equal
< Less than
> Greater than
>= Equal or greater
<= Equal or less

For text signal comparisons, "\" (blanks) are ignored. For logic types, only "=" comparisons are permitted.

2. PARAMETERS:

INPUTS	FORMS	VARIABLE TYPE	DEFAULT
NAME1	SN,N,S,V,C,G	ANALOG/COUNT LOGIC /TEXT	①
NAME2	SN,N,S,V,C,G	ANALOG/COUNT LOGIC /TEXT	①

DESCRIPTOR	FORMS	VARIABLE TYPES	DEFAULT
OP	OC	-	①
XXX	BN	-	Next block
YYY	BN	-	Next block

RESULTS
Module XXX or YYY will be executed

3. SPECIAL RESTRICTIONS:

NAME1 and NAME2 must be same type.

① Required input

4. CBL MACRO INSTRUCTION FORMAT:

Type PROCEDURAL Op Code 8A6A Name COMPARE VALUES

POS					
	M	BLOCK SIZE		BLOCK SEQUENCE NUMBER	
	OPERATION CODE				
	POINTER TO NEXT CBL OP. CODE				
1	0	0	CODE	TYPE	
2	POINTER TO 1st ARGUMENT "NAME1"				
3	POINTER TO 2nd ARGUMENT "NAME2"				
4	POINTER TO SUBMODULE IF TRUE "XXX"				*
5	POINTER TO SUBMODULE IF FALSE "YYY"				*

TYPE	CODE	COMPARISON
0 - Analog	0 -	=
1 - Count	1 -	<>
2 - Logic	2 -	<
3 - Text	3 -	>
	4 -	>=
	5 -	<=

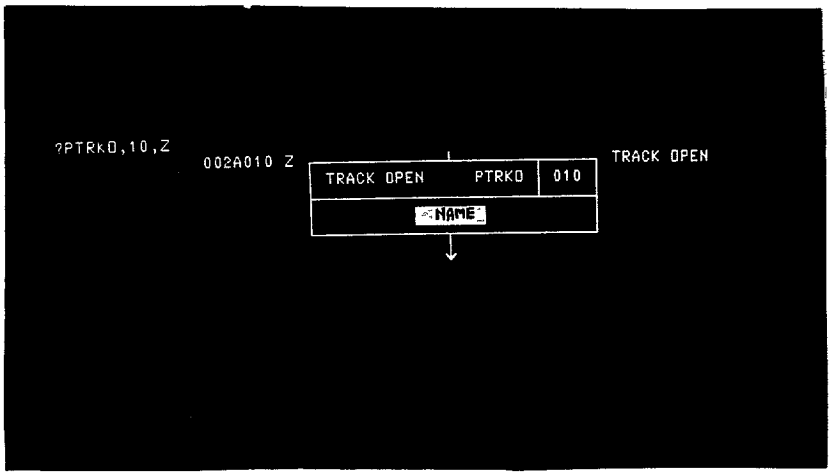
* Refer to description of PSUBM for detail of submodule address linkage.

5. ADDITIONAL INFORMATION:

A flow chart appears in Appendix D.

PTRKO

TRACK OPEN



1. FUNCTION:

The entire contents of table NAME are shifted down by one element. The element at the bottom of the table is lost. The element at the top of the table is replaced with zero if the table is analog or count-type. The element at the top of the table is replaced with blanks if the table type is text. The table is defined by a DATV, PTATV or INZV block.

2. PARAMETERS:

INPUT	FORM	VARIABLE TYPE	DEFAULT
NAME	AN	ANALOG/COUNT TEXT	①

DESCRIPTOR
None

RESULTS
The table NAME is modified

3. SPECIAL RESTRICTIONS:

The table specified by name must be defined by a DATV, PDATV or INZV block.

① Required input

PTRKO

4. CBL MACRO INSTRUCTION FORMAT:

Type PROCEDURAL Op Code 8A67 Name TRACK OPEN

POS					
	M	BLOCK SIZE		BLOCK SEQUENCE NUMBER	
	OPERATION CODE				
	POINTER TO NEXT CBL OP. CODE				
1	0	0	0	TYPE	
2	POINTER TO TABLE "NAME"				

TYPE
0 - Analog
1 - Count
3 - Text

5. ADDITIONAL INFORMATION:

BEFORE PTRKO

	SIZE
	6
	8
	4
	3
	2
	1
	9

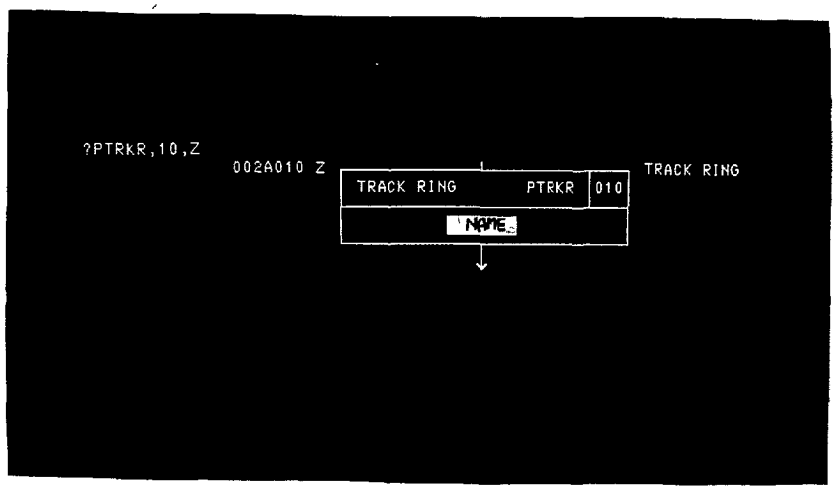
AFTER PTRKO

	SIZE
	0
	6
	8
	4
	3
	2
	1

DATV defined by
name (count, size = 7)

PTRKR

TRACK RING



1. FUNCTION:

The table labelled NAME is cyclical and is shifted down by one element. The bottom element reappears at the top of the table.

2. PARAMETERS:

INPUT	FORM	VARIABLE TYPE	DEFAULT
NAME	AN	ANALOG, COUNT TEXT	①

DESCRIPTOR
None

RESULT
The table NAME is modified

3. SPECIAL RESTRICTIONS:

The table specified by NAME must be defined by DATV, PDATV or INZV block.

① Required input

4. CBL MACRO INSTRUCTION FORMAT:

Type PROCEDURAL Op Code 8A68 Name TRACK RING

POS				
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER	
	OPERATION CODE			
	POINTER TO NEXT CBL OP. CODE			
1	0	0	0	TYPE
2	POINTER TO TABLE "NAME"			

TYPE
0 - Analog
1 - Count
3 - Text

5. ADDITIONAL INFORMATION:

BEFORE PTRKR

	SIZE
	6
	8
	4
	3
	2
	1
	9

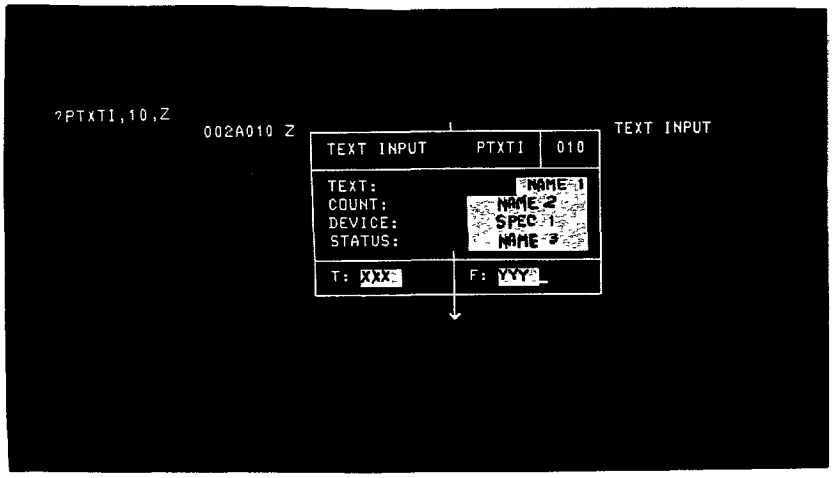
AFTER PTRKR

	SIZE
	9
	6
	8
	4
	3
	2
	1

DATV defined by
NAME (Count, size = 7)

PTXTI

TEXT INPUT



1. FUNCTION:

Text data is read from the input device designated by SPEC 1. The text data from the input buffer is filtered and moved to the array that is specified by NAME1. There shall be one input buffer assigned to each procedural segment that performs text input.

Following the transfer of data, then submodule XXX is performed before continuing to the next CBL macro. However, if the input fails to terminate successfully, submodule YYY is performed before continuing to the next CBL macro.

The maximum amount of text input does not exceed eighty characters. There may be from one to ten text signals. Each text signal may be from one to eight characters followed by a comma. The input spring will be terminated by "CR". NAME2 will contain a count variable which reflects the actual number of text signals transferred.

If the input fails, NAME3 will contain a code reflecting the type of error.

2. PARAMETERS:

INPUTS	FORMS	VARIABLE TYPE	DEFAULT
SPEC1	SN,N,S,V,C,G	COUNT	①
NAME2	SN,N,S,V,C,G	COUNT	①
XXX/YYY	BN	-	Next block

DESCRIPTOR
None

RESULTS			
NAME1	AN	TEXT	①
NAME3	S,V	COUNT	①

3. SPECIAL RESTRICTIONS:

XXX and YYY must be numbers of a PSUDM block within the same segment.

① Required input

PTXTI

4. CBL MACRO INSTRUCTION FORMAT:

Type PROCEDURAL Op. Code 8A6E Name Text INPUT

POS				
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER	
	OPERATION CODE			
1	9	B	9	1
2	POINTER TO * + 5 (POSITION 7)			
3	POINTER TO REQUEST CODE (POSITION 13)			
4	POINTER TO CONTROL WORD "SPEC1"			
5	POINTER TO INPUT BUFFER			*
6	POINTER TO LENGTH (POSITION 14)			
7	8	A	6	F
8	POINTER TO USERS' BUFFER "NAME1"			
9	POINTER TO NR. TEXT SIGNALS "NAME2"			
10	POINTER TO ERROR STATUS "NAME3"			
11	POINTER TO SUBMODULE XXX (NORMAL)			
12	POINTER TO SUBMODULE YYY (ERROR)			
13	1			
14	46			

* Input buffer is an intermediate array 46 words long.

5. ADDITIONAL INFORMATION:

Example of text input:

```
Keyboard inputs:
   AAAA, BB, CCCCC, D, EEEEEEE, FFFF (CR).


PTXTI INPUT
  NAME 1 (TEXT) - Array
  NAME 2 (COUNT) - 6
  SPEC 1 (DEVICE) - 7
```

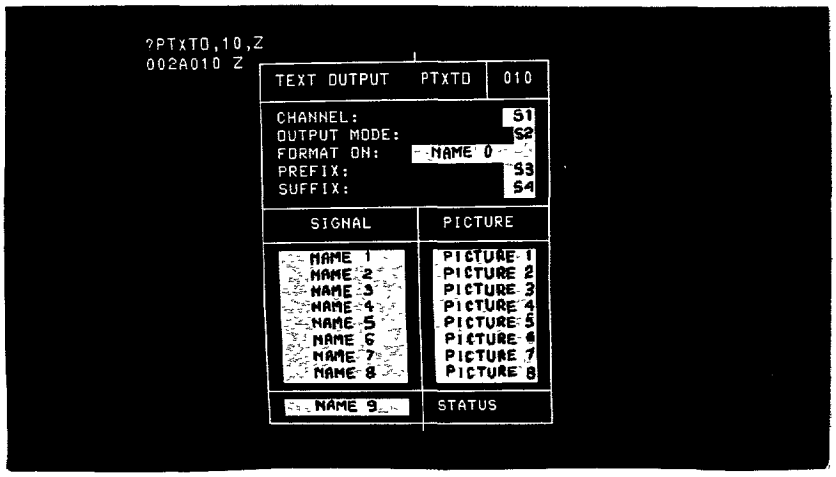
RESULTS

ARRAY	(1)	AA	AA	⍽⍽	⍽⍽
	(2)	BB	⍽⍽	⍽⍽	⍽⍽
	(3)	CC	CC	C⍽	⍽⍽
	(4)	Db	⍽⍽	⍽⍽	⍽⍽
	(5)	EE	EE	EE	EE
	(6)	FF	FF	⍽⍽	⍽⍽
	(7)				
	(8)				
	(9)				
	(10)				

A flow chart appears in Appendix D.

PTXTO

TEXT OUTPUT



1. FUNCTION:

This block causes an output request to be made. If the request is accepted by the output routine, the data and format pointers will be placed in the proper output queue and NAME9 is set true. If request is not accepted, NAME9 is set false. For accepted request, the values of the signals specified by NAME1 up to NAME8 will be output in the format specified by the corresponding picture.

2. PARAMETERS:

INPUTS	FORMS	VARIABLE TYPE	DEFAULT
NAME0	SN,N,S,V,C,G	LOGIC	FALSE
NAME1-8	SN,N,S,V,C,G	LOGIC, TEXT ANALOG, COUNT	①
DESCRIPTOR			
SPEC1 (S1)	C	COUNT	①
SPEC2 (S2)	OC	-	○
SPEC3 (S3)	OC	-	○
SPEC4 (S4)	OC	-	○
PICTURE 1-8	SPECIAL	TEXT	
RESULTS			
NAME9	S,V	-	-

3. SPECIAL RESTRICTIONS:

① Required input

4. CBL MACRO INSTRUCTION FORMAT:

Type PROCEDURAL Op Code 8A6B Name TEXT OUTPUT

POS				
	M	BLOCK SIZE *	BLOCK SEQUENCE NUMBER	
	OPERATION CODE			
	POINTER TO NEXT CBL OP. CODE			
1	POINTER TO ACCUMULATION BUFFER			
2	0	0	0	SPEC1
3	0	SPEC4	SPEC3	SPEC2
4	POINTER TO "NAME0"			
5	POINTER TO FORMAT TABLE			
6	POINTER TO SSB			
7	POINTER TO "NAME9"			
8	OP4	OP3	OP2	OP1
9	POINTER TO "NAME1"			
10	POINTER TO "NAME2"			
11	POINTER TO "NAME3"			
12	POINTER TO "NAME4"			
13	OP8	OP7	OP6	OP5
14	POINTER TO "NAME5"			
15	POINTER TO "NAME6"			
16	POINTER TO "NAME7"			
17	POINTER TO "NAME8"			
18	9	B	9	7
19	8	A	6	C

OP.8 and C thru 3 not allowed.
OP.F is used as no operation to permit alignment of pointer position with signal input.
* BLOCK SIZE = (No. of words in block + 1)/2

PTXTO

5. ADDITIONAL INFORMATION:

$SPEC_1$ - Specified device number (0-63) of the output device to which the message is directed.

$SPEC_2$ - Format defining mode:

0 - Formatted sequence of ASCII characters (default)	8 } Reschedule
1 - Unformatted data	9 } Reschedule
2 - Format only	10 } Reschedule

$NAME_0$ - Format control enable logic type signal.
- If true - Format control information sent out to formatter hardware.
- If false - No format control (default).
- Format control is not applicable if defining mode is zero; $NAME_0$ will be ignored.
- Format control is always sent out on first request for defining mode 1.

$SPEC_3$ - Prefix code; preceeds data string.
00 - No Action
01 - Single Space (CR)
02 - Double Space (CR/LF)
03 - 15 - No action

$SPEC_4$ - Suffix code; follows data string.
00 - Normal (CR/LF)
01 - No action (CR/LF suppressed)
02 - 15 No action

OP_{1-8} - Data Output Specification

Code	Internal Representation of Data
00	Last (No more)
01	Analog, 2 words
02	Count, 1 word
03	Logical, 1 bit
04	Text, 4 words (7 or 8 characters)
05	Hexadecimal, 1 word
06	Octal, 1 word
07	Time, 2 words/system time counts
08	Error
09	Text - 1 word (1 or 2 characters)
10	Text - 2 words (3 or 9 characters)
11	Text - 3 words (5 or 6 characters)
12 - 14	Error
15	NOP

NAME$_1$ TO NAME$_8$ - Analog, count logic or text type signals in any mixture. There can be any number of signals from one to eight. The signal type must match picture (explained in next paragraph). The value of the named signal will be output according to the format specified by the corresponding picture.

PICTURE$_1$ TO PICTURE$_8$ - Format control for conversion and output: Each signal listed for output has one to ten-character picture field associated with it. The picture specifies the output field width, format and, if applicable, fixed characters to be output. There may be only one format picture per named signal and, correspondingly, each picture may specify format for no more than one signal. Only fixed characters are allowed in a picture field associated with a "NAME" to which no signal is connected. If the picture field is terminated by a @, a carriage return is forced. Leading or trailing blanks are ignored. "/" must be used to fill out picture if wider column spacing is required. The "/" may be leading or trailing or both.

PICTURE	FORMAT DEFINITION
#####	- Defined field width for analog and count-type signals. - Field width must include space for sign if number will ever be negative. Negative signs will be output; positive signs will not. - For analog signals, one of the "#" characters may be a decimal point. If a decimal point is included in the picture, rounding will take place on the last significant digit in the field. The decimal point will be output, even if no fractional part is specified. - Lack of a decimal point in the picture implies whole number output. If an analog signal has a fractional part, it will be truncated. - Leading zeros are replaced with blanks for count and analog values.
(ANALOG TYPE) EEEEEE	- Allowable format for analog signals that will produce an exponential format for output. Quantity of E's will determine number of significant digits. Total field width is quantity of "E" plus 6.
(SPACING) &:SNNA	- Special picture with no input signal allowed. Will generate quantity "N: of character "A". A will default to blanks for generating spaces.

PTXTO

LLLLLL - Defines field for logic type signals. Output as number of L - 1 leading blanks followed by T or F.
- True state output as T.
- False state output as F.

TTTTTTTT - Defines field width for text type signals.
- Field starts with the "T" character and extends to the right-most "T" (e.g., TTT is a 3-character field width).
- The number of characters (maximum = 8) specified by the picture are output starting with the first character of the text signal. Leading blanks are generated for T excess of 8.
- Any characters remaining in the text signal (in excess of the field width specified by the picture) are ignored.

HH:MM:SS - Defines the field width and format for output of time. Count type signals will be assumed to contain time in hours/minutes/seconds. If no signals is entered, the current system clock value is used.
- Field starts with the H character and extends to the right-most character S.
- The colon(s), ":" will be output.

XXXXXXXXX - Defines field width and conversion of count signals to hexadecimal output format.
- The number of characters specified by the picture are output with 4 hex digits right-justified. If a field width of more than 4 characters is specified, leading blanks are output to fill the field. For 1, 2 or 3 X, 4 hex digits are always output.

All characters (except "\", "@") encountered in a picture field without related signal names are considered to be literals and will be output directly. A picture field with signal name input may contain only the format picture for that signal.

EXAMPLE

To print the message

TEMPERATURE = _ _ _ DEGREES
AT _ _: _ _: _ _ TIME

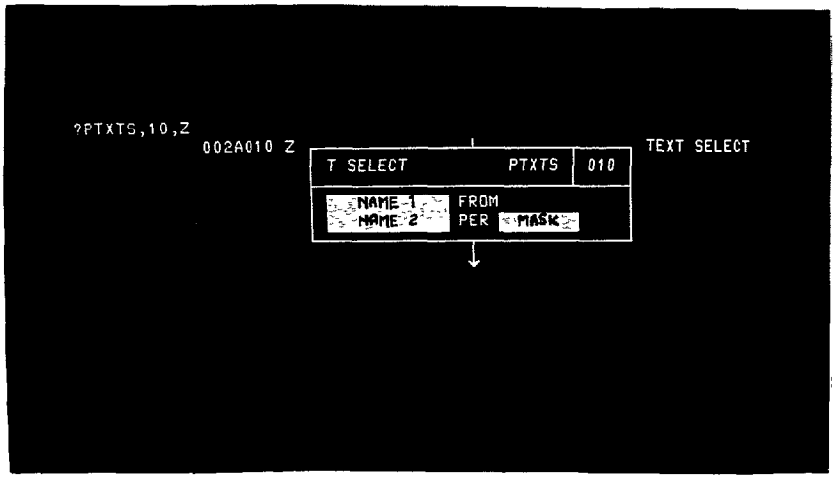
on a printer, a PTXTO block to do so would look like the following:

TEXT OUTPUT	PTXTP	001
CHANNEL:	6	(LU # OF PRINTER)
OUTPUT MODE:	0	(FORMAT MODE)
FORMAT ON:	8T	(FORMAT CONTROL ON)
PREFIX:	02	(CR/LF PRECEEDS)
SUFFIX:	00	(CR/LF FOLLOWS)

SIGNAL	PICTURE
	TEMPERATURE \=\
TEMP	### \DEGREES @ AT\
TIME	HH:MM:SS \TIME

PTXTS

TEXT SELECT



1. FUNCTION:

Characters from NAME2 are selected or deleted according to MASK and stored in NAME1. MASK is made up of "S" (SAVE) and "\" (DELETE) characters.

EXAMPLE: NAME2 ABCDE*GH

MASK \S\S\SS\\

NAME1 A*C***

Where: "*" denotes delete code (3F).

2. PARAMETERS:

INPUTS	FORMS	VARIABLE TYPE	DEFAULT
NAME2	SN,N,S,V,C,G	TEXT	①
MASK	SPECIAL	TEXT	①

DESCRIPTOR
None

RESULT			
NAME1	S,V	TEXT	—

3. SPECIAL RESTRICTIONS:

① Required input

4. CBL MACRO INSTRUCTION FORMAT:

Type PROCEDURAL Op Code 8A6D Name TEXT SELECT

POS	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER	
	OPERATION CODE			
	POINTER TO NEXT CBL OP. CODE			
1	POINTER TO INPUT VALUE "NAME2"			
2	POINTER TO "MASK"			*
3	POINTER TO OUTPUT VALUE "NAME1"			

* MASK is 8 codes in one word (16 bits)

C1	C2	C3	C4	C5	C6	C7	C8

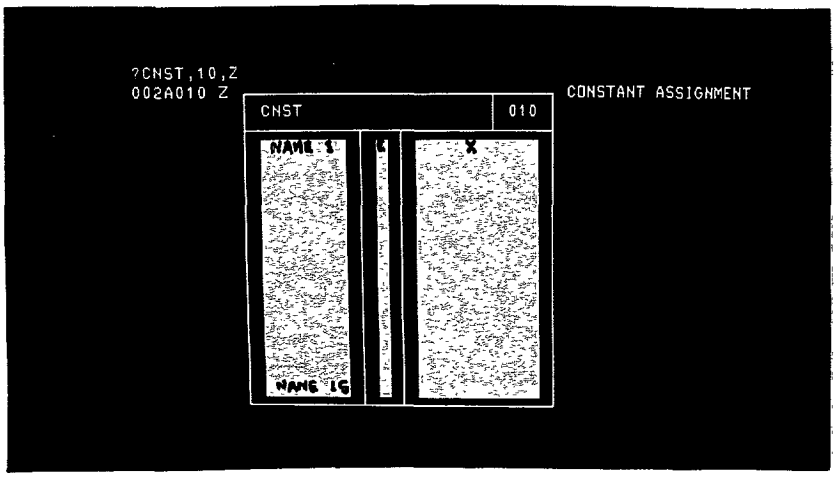
CODES ARE: 00 - DELETE "\"
01 - SAVE "S"
10 - FUTURE
11 - FUTURE

5. ADDITIONAL INFORMATION:

None

CNST

CONSTANT ASSIGNMENT



1. FUNCTION:

This block assigns constant values (X) according to code E, to names 1 thru 15 (maximum). Analog values may be entered in standard decimal format (±XXX.X) or in scientific notation (±XX.XXE±XX).

CODE E VALUES:
A - Analog
C - Count
L - Logic
T - Text
O - Octal

Name 1-15 and X values are preceded by a # sign. These names may only appear as inputs to blocks.

2. PARAMETERS:

INPUTS	FORMS	VARIABLE TYPE	DEFAULT
NAME1-15	S	① ANALOG/COUNT/LOGIC/TEXT	②
X (VALUES)	C	① ANALOG/COUNT/LOGIC/TEXT	②
DESCRIPTOR			
E (TYPE CODE)	OC	-	-

3. SPECIAL RESTRICTIONS:

① Type determined by code E

② Required input (only 1 set required)

Permissible value for count type is -32768 to +32767.

Permissible value for analog type is ±10E-38 to ±10E+38.

Permissible value for text type is 8 ASCII characters.

Permissible value for logical type is T or F.

4. CBL MACRO INSTRUCTION FORMAT:

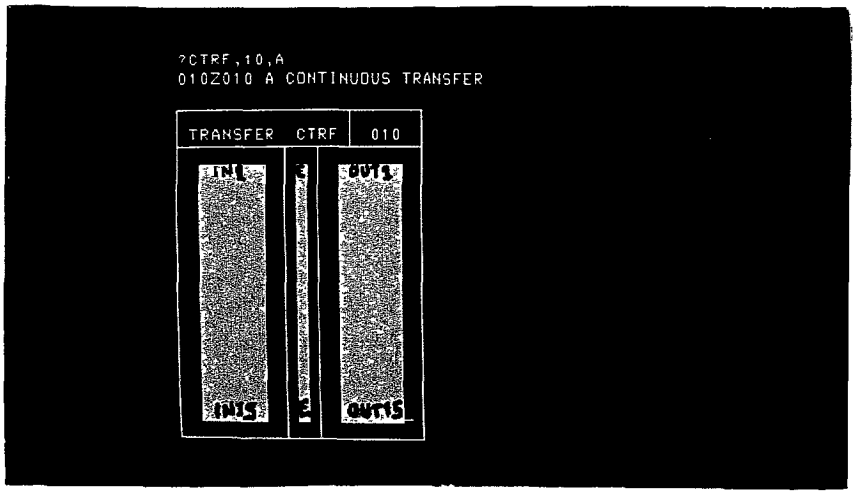
None

5. ADDITIONAL INFORMATION:

This control block is in the "special" category.

CTRF

CONTINUOUS TRANSFER



1. FUNCTION:

This block causes the PMC to schedule the inputs IN1 through IN15 before running the segment. The data that was input is scheduled as outputs to the CSH addresses OUT1 through OUT15 after running the segment. The effect is that of a continuous transfer of data from IN1 to OUT1, IN2 to OUT2, etc. The type field E specifies the length of the word to be transferred.

A - Analog Data, 32 bit transfer
C - Count Data, 16 bit transfer

IN; and OUT; may only be CSH group addresses.

The control block macro does not in itself perform any action function other than transfer control to the next block.

All lines are optional.

IN; and OUT; share the same holding area.

CTRF

2. PARAMETERS:

INPUTS	FORMS	VARIABLE TYPE	DEFAULT
IN 1-15	A	-	①
DESCRIPTORS			
E (1-15) (TYPE CODE)	OC	-	①
OUTPUTS			
OUT 1-15	A	-	①

3. SPECIAL RESTRICTIONS:

① Required input. (Only one set required).

(a) IN1-IN15 and OUT1-OUT15 may only be CSH group addresses.

(b) E is limited to A (analog) and C (count). This does not type the input or output but rather determines the number of words to transfer.

(c) All entries are optional on a line basis. That is, if IN1 is present then E1 and OUT1 are required also.

(d) FORCE will not work with CTRF block as the block is not actively involved in the transfer process.

CTRF

4. CBL MACRO INSTRUCTION FORMAT:

Type SPECIAL Op Code 8B50 Name CONTINUOUS TRANSFER

POS			
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER
	OPERATION CODE		
	POINTER TO NEXT CBL OP. CODE		
1	ADDRESS OF HOLDING AREA FOR INPUT/OUTPUT		
2	"		
3	"		
4	"		
5	"		
6	"		
7	"		
8	"		
9	"		
10	"		
11	"		
12	"		
13	"		
14	"		
15	"		*

* Length is variable from 0 to 15 holding area addresses.

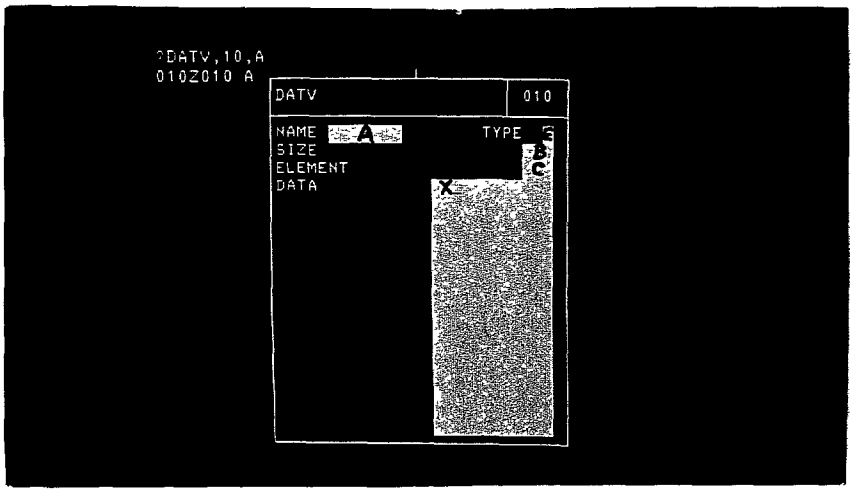
5. ADDITIONAL INFORMATION:

This block is of the "special" category.

NOTES

DATV

ARRAY ASSIGNMENT



1. FUNCTION:

One or more DATV blocks define the constants (E specifies type, e.g., analog, count, etc.) required by other blocks and supplies them in single width array or table form. Each DATV block provides space for 15 constants. Each additional DATV required to list more than 15 items has the same array name in field A, the same B length and same type E. The first DATV block has C = 1 for first 1-15 items. The 2nd DATV block has C = 16 for items 16 - 32, etc. f or maximum 6 DATV blocks.
CODE E:

A - analog
C - count
L - logic
T - text (ASCII)

DATV

2. PARAMETERS:

INPUTS	FORMS	VARIABLE TYPE	DEFAULT
A	AN	① ANALOG/COUNT/TEXT/LOGIC	②
X (VALUES)	C	① ANALOG/COUNT/TEXT/LOGIC	②
DESCRIPTOR			
E (Type code)	OC	COUNT	②
B (Table size)	C	COUNT	②
C	C		②

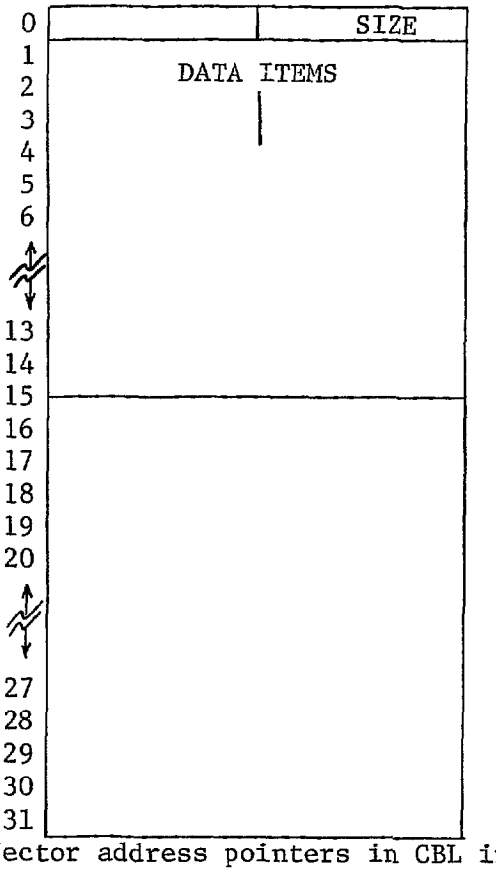
OUTPUTS
The values X are initially stored in Table A.

3. SPECIAL RESTRICTIONS:

① Type determined by Code E.

② Required Input

4. CBL MACRO INSTRUCTION FORMAT:



* Vector address pointers in CBL instruction are to word 0 in the DATV array. Word 0 contains a value which indicates total size of DATV array.

DATV

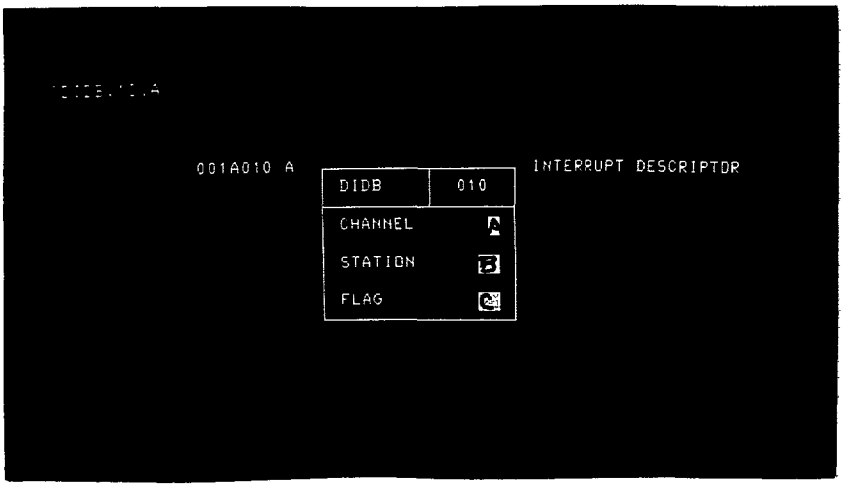
5. ADDITIONAL INFORMATION:

This control block is included in the "special" category.

NOTES

DIDB

DATA LINK INTERRUPT



1. FUNCTION:

The DIDB block provides a means of entering data into the CSH interrupt table which describes a status table from CSH slave station. When flag C at station B on channel A goes true, the interrupt segment containing this DIDB will be activated.

2. PARAMETERS:

INPUTS	FORMS	VARIABLE TYPE	DEFAULT
None			
DESCRIPTOR			
A (Channel)	OC	-	①
B (Station)	OC	-	①
C (Flag)			①
OUTPUTS			
None			

3. SPECIAL RESTRICTIONS:

This block may appear multiple times within a segment, but is allowed only in interrupt segments. The channel (J, K or L) must be a master, and the station (1-15) available, and the flag (0-7) valid.

① Required input

4. CBL MACRO INSTRUCTION FORMAT:

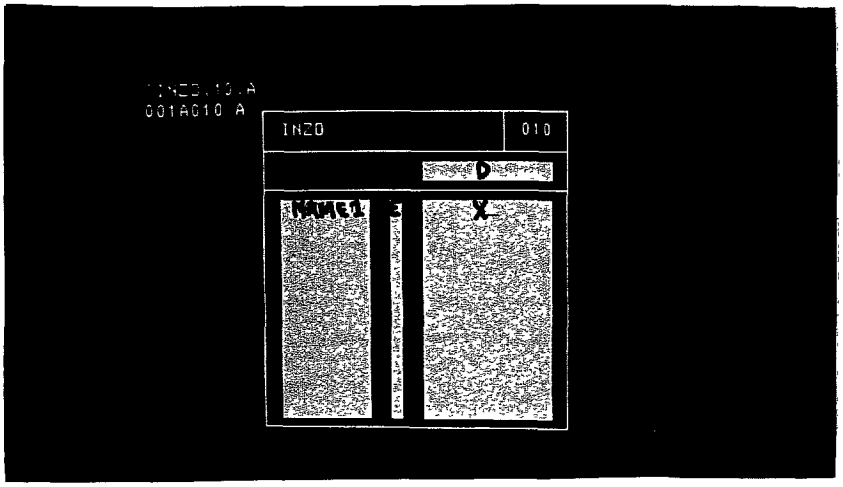
None

5. ADDITIONAL INFORMATION:

This block is included in the "special" category.

INZO

INITIALIZE OUTPUTS



1. FUNCTION:

This block initializes block outputs NAME1 (maximum of 12 per table) whenever this block is executed. The E field specifies the type of signal -

A - Analog
C - Count
L - Logic
O - Octal

Logic signals have T or F in field F.
Analog values may be in scientific notation.

2. PARAMETERS:

INPUTS	FORMS	VARIABLE TYPE	DEFAULT
NAME1 - NAME12	SN,N	① ANALOG/COUNT LOGIC/TEXT	②
X (VALUE)		① ANALOG/COUNT LOGIC/TEXT	②
DESCRIPTOR			
E (Type Code)	OC	-	②
D	SN,N,S,V,C,G,A	LOGIC	ENABLED ③

OUTPUTS
The X values are initially stored into the outputs, NAME1 - NAME 15, and whenever block is executed.

3. SPECIAL RESTRICTIONS:

① Data type must be specified by code E.

② Required input (only 1 set required).

③ Optional Input

4. CBL MACRO INSTRUCTION FORMAT:

Type SPECIAL Op Code 8B4B Name INITIALIZE OUTPUT

POS					
	M	**BLOCK SIZE	BLOCK SEQUENCE NUMBER		
	OPERATION CODE				
	POINTER TO NEXT CBL OP. CODE				
1	0	0	0	DATA CODE "E"	
2	POINTER TO INITIAL VALUE "X"				
3	POINTER TO DATA "C"				
4	0	0	0	DATA CODE	*
5	POINTER TO INITIAL VALUE				*
6	POINTER TO DATA				
7	0	0	0	F	

* These entires are repeated for each value to be initialized. The length of the block depends on the number of inputs.

DATA CODE E
- 0 - No operation
- 1 - Analog (2 words)
- 2 - Count (1 word)
- 3 - Logic (1 bit)
- 4 - Octal (1 word)
- F - End of list (terminate block)

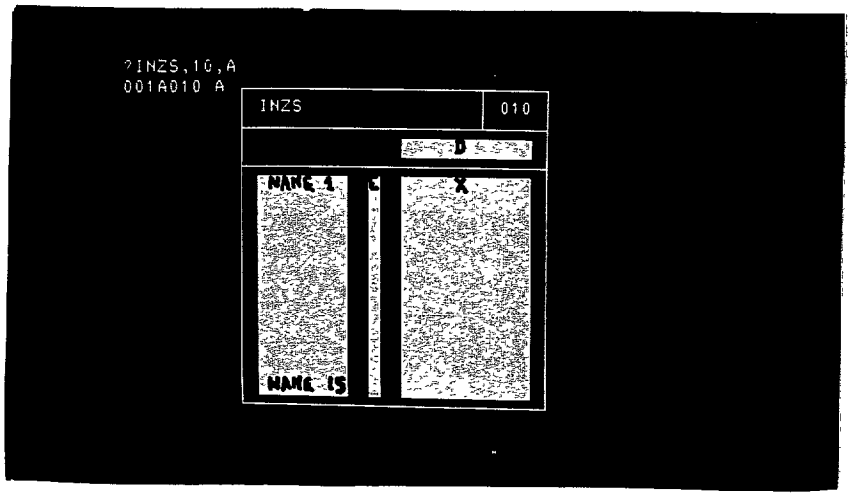
** Block size = (Number of words in block +1)/2

5. ADDITIONAL INFORMATION:

This block is included in the "special" category.

INZS

INITIALIZE DATA



1. FUNCTION:

This block established initial values X according to type code E for named variables NAME 1. Each time this block is executed, the named signals are set to the prescribed values.

Type code E is:

A - Analog
C - Count
O - Octal
L - Logical
T - Text

Logical values entered in the X field will be entered as T or F. Analog values may be in scientific notation. 15 values may be in each INZS block.

2. PARAMETERS:

INPUTS	FORMS	VARIABLE TYPE	DEFAULT
NAME 1-15	S	① ANALOG/COUNT LOGIC/TEST	②
X (Value)	C	① ANALOG/COUNT LOGIC/TEST	②

DESCRIPTOR	FORMS	VARIABLE TYPE	DEFAULT
E (Type Code)	OC	-	②
D	SN,N,S,V,C,G,A	LOGIC	ENABLED ③

OUTPUTS
The X values are initially stored into the outputs, NAME 1 - NAME 15, and whenever block is executed.

3. SPECIAL RESTRICTIONS:

① Type determined by code E.

② Required input

③ Optional input

4. CBL MACRO INSTRUCTION FORMAT:

Type SPECIAL Op Code 8B4B Name INITIALIZE DATA

POS					
	M	**BLOCK SIZE	BLOCK SEQUENCE NUMBER		
	OPERATION CODE				
	POINTER TO NEXT CBL OP. CODE				
1	0	0	0	DATA CODE "E"	
2	POINTER TO INITIAL VALUE "X"				
3	POINTER TO DATA "C"				
4	0	0	0	DATA CODE	*
5	POINTER TO INITAL VALUE				*
6	POINTER TO DATA				
7	0	0	0	F	

* These entires repeated for each value to be initialized. The length of the block depends on the number of inputs.

DATA CODE E:
0 - No Operation
1 - Analog (2 words)
2 - Count (1 word)
3 - Logic (1 bit)
4 - Octal (1 word)
5 - Text (4 words)
F - End of list (terminate block)

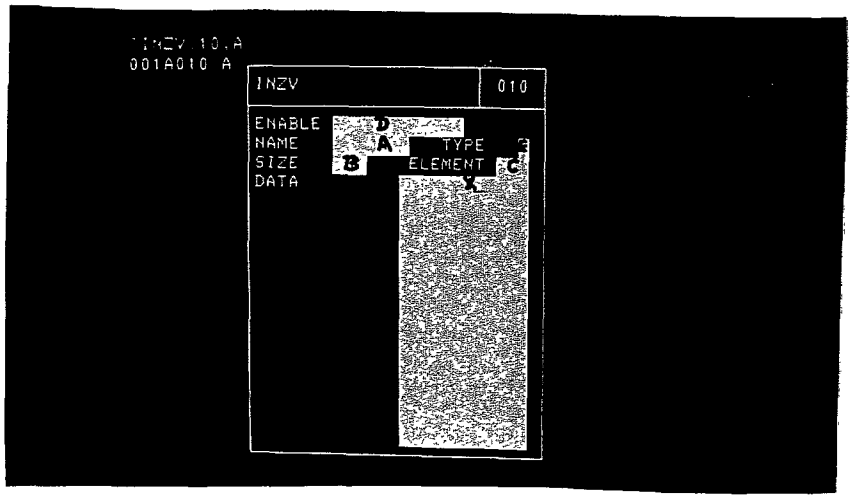
** Block Size = (Number of words in block +1)/2

5. ADDITIONAL INFORMATION:

This block is included in the "special" category.

INZV

INITIALIZE ARRAY



1. FUNCTION:

This block established and lists initial values (X) for a single width array defined in the NAME field A. The type of initial value is defined by code E.

A - Analog	O - Octal
C - Count	T - Text
L - Logic	

Up to 6 INZV blocks (15 items, each) may be utilized to provide an N element table. B-field would then become N in each INZV block with the same NAME. The C-fields define the starting element number in each, e.g., 1, 16, 31, 46, 61, 76. Each time INZV executes, the originally defined values are restored.

2. PARAMETERS:

INPUTS	FORMS	VARIABLE TYPE	DEFAULT
A	AN	① ANALOG/COUNT/LOGIC/TEXT	②
X (Value)	C	① ANALOG/COUNT/LOGIC/TEXT	②

DESCRIPTOR	FORMS	VARIABLE TYPE	DEFAULT
E (Type code)	OC	-	②
B (Table size)	C	COUNT	②
C (Starting element)	C	COUNT	②
D	SN,N,S,V,C,G,A	LOGIC	ENABLED 3

OUTPUTS
The X values are initially stored into the A table and whenever the block is initialized.

3. SPECIAL RESTRICTIONS:

① Type determined by code E.

② Required input

③ Optional input

4. CBL MACRO INSTRUCTION FORMAT:

Type SPECIAL Op Code 8B4F Name INITIALIZE VECTOR

POS				
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER	
	OPERATION CODE			
	POINTER TO NEXT CBL OP. CODE			
1	SIZE	0	DATA CODE "E"	*
2	POINTER TO INITIAL DATA VECTOR "X"			
3	POINTER TO VECTOR "A"			

DATA CODE E:
0 - No operation
1 - Analog (2 words)
2 - Count (1 word)
3 - Logic (1 bit)
4 - Octal (1 word)
5 - Text (4 words)

* Size entry is 00 for all types except logic. For logic vector, size contains integer value defining number of bits in vector.

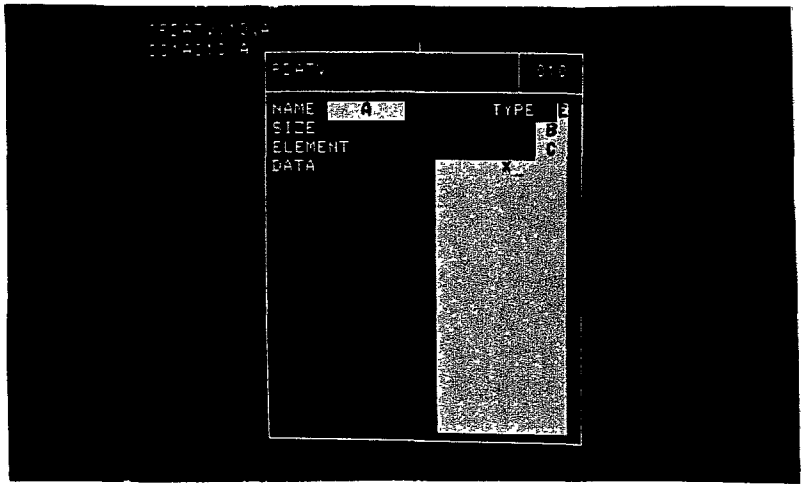
5. ADDITIONAL INFORMATION:

Up to 6 INZV blocks may generate data for a table A, however, only one control block is generated, (i.e., INZV block with element ONE).

This block is included in the "special" category.

PDATV

ARRAY ASSIGNMENT



1. FUNCTION:

One or more PDATV blocks define the constants (E specifies TYPE - Analog Count, etc.) required by other blocks and supplies them in single width array or table form. Each PDATV block provides space for 15 constants. Each additional PDATV required to list more than 15 items has the same array name in field A, same B length and same TYPE E. The first PDATV block has C-1 for first 1-15 items. The second PDATV block has C = 16 for items 16-30, etc. for a maximum of 6 PDATV blocks.

CODE E: A - Analog O - Octal
C - Count T - Text (ASCII)

2. PARAMETERS:

INPUTS	FORMS	VARIABLE TYPE	DEFAULT
A	AN	(1) ANALOG/COUNT/TEXT	(2)
X (Value)	C	(1) ANALOG/COUNT/TEXT	(2)
DESCRIPTOR			
E (Type code)	OC	-	(2)
B (Table size)	C	COUNT	(2)
C (Starting element)	C	COUNT	(2)

RESULTS

The X values are initially stored into table A.

3. SPECIAL RESTRICTIONS:

① Type determined by code E. (Logic not permitted).

② Required input

4. CBL MACRO INSTRUCTION FORMAT:

None

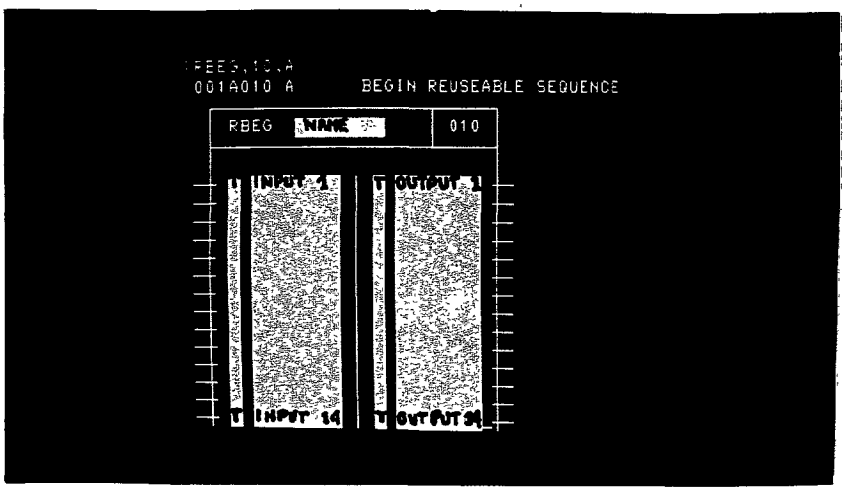
5. ADDITIONAL INFORMATION:

The PDATV block may be used only in a procedural segment. Data defined by PDATV is accessible only by the defining segment.

This block is included in the "special" category.

RBEG

BEGIN REUSABLE SEQUENCE



1. FUNCTION:

This block identifies the beginning of a reusable sequence of control blocks. It also defines the name, input and output parameters of the reusable sequence. T defines the type of parameter.

A - Analog
C - Count
L - Logical

2. PARAMETERS:

IDENTIFICATION	FORMS	VARIABLE TYPE	DEFAULT
NAME	Reusable Sequence Name	-	①
INPUTS			
Input 1-14	S	A, C, L	①
OUTPUTS			
Output 1-14	N	A, C, L	①
DESCRIPTOR			
T (Type code)	OC	-	①

3. SPECIAL RESTRICTIONS:

① Required input. (Only 1 set of inputs and outputs required).

RNAME should only be referenced by RCALL blocks.

4. CBL MACRO INSTRUCTION FORMAT:

Type SPECIAL Op Code 8B2D Name BEGIN REUSABLE SEQUENCE

POS			
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER
	"RBEG" OPERATION CODE		
	POINTER TO NEXT CBL OP. CODE		
1	POINTER TO ENABLE VARIABLE IN SSB		
2	POINTER TO REND OP. CODE		

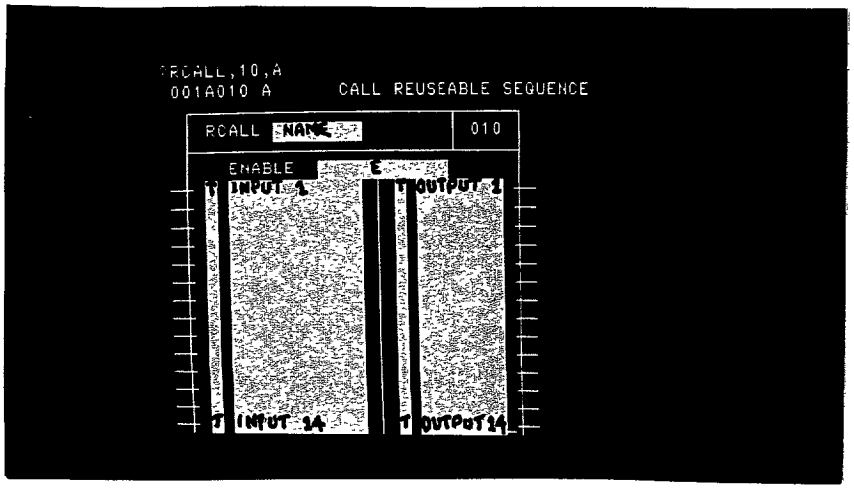
5. ADDITIONAL INFORMATION:

Refer to Appendix B for instructions in the use of RBEG in a reusable sequence.

This block is included in the "special" category.

RCALL

CALL REUSABLE SEQUENCE



1. FUNCTION:

RCALL causes the PMC to perform a reusable sequence of control blocks. It also defines the specific input and output signals required for this specific usage of the reusable sequence. T defines the type of signal.

A - Analog
C - Count
L - Logical

2. PARAMETERS:

IDENTIFICATION	FORM	VARIABLE TYPE	DEFAULT
RNAME	Reusable Sequence Name	-	①
INPUTS			
Input 1-14	SN,N,S,C,G	A, C, L	①
OUTPUTS			
Outputs 1-14	S	A, C, L	①
DESCRIPTOR			
T (Type code)	OC	-	①

RCALL

3. SPECIAL RESTRICTIONS:

The number and type of inputs and outputs must correspond exactly to the number and type of parameters defined in the RBEG block that begins the reusable sequence.

① Required input. (Only 1st set of inputs and outputs required).

4. CBL MACRO INSTRUCTION FORMAT:

Type SPECIAL Op Code 8A2F Name CALL REUSABLE SEQUENCE

POS			
	M	BLOCK SIZE	BLOCK SEQUENCE NUMBER
	"CALL" OPERATION CODE		
	POINTER TO NEXT CBL OP. CODE		
1	POINTER TO INTERMEDIATE TABLE		
2	POINTER TO RBEG CBL OP. CODE		
3	POINTER TO FIRST TIME FLAG IN SSB		
4	POINTER TO SEGMENT INTERVAL IN SSB (ANALOG)		
5	POINTER TO SEGMENT INTERVAL IN SSB (COUNT)		
6	POINTER TO INPUT 1		
7	POINTER TO OUTPUT 1		
	(FURTHER INPUT & OUTPUT POINTERS AS REQUIRED)		

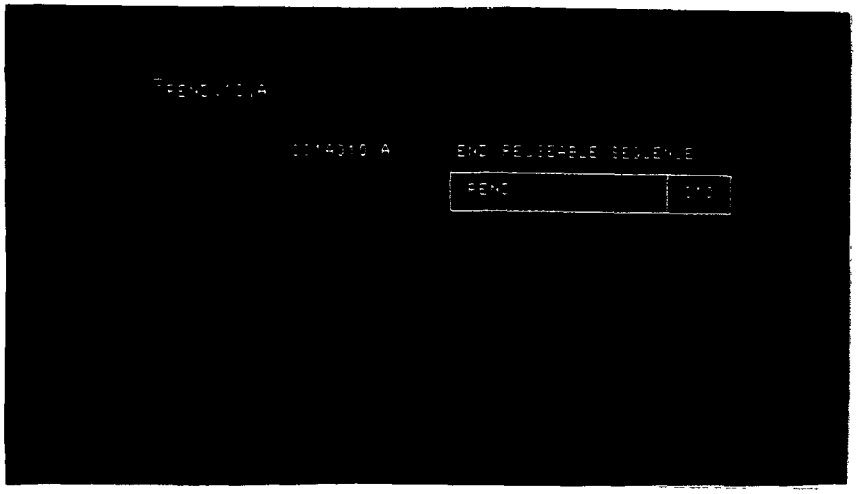
5. ADDITIONAL INFORMATION:

Refer to Appendix B for instructions in the use of RCALL in the reusable sequence.

This block is included in the "special" category.

REND

END REUSABLE SEQUENCE



1. FUNCTION:

This block identifies the end of a reusable sequence of control blocks.

2. PARAMETERS:

None

3. SPECIAL RESTRICTIONS:

None

4. CBL MACRO INSTRUCTION FORMAT:

Type SPECIAL Op Code 8B2E Name END REUSABLE SEQUENCE

M	Block Size	Block Sequence Number
"REND" Operation Code		
Pointer to Next CBL		

5. ADDITIONAL INFORMATION:

Refer to Appendix B for instructions in the use of REND in the reusable sequence.

This block is included in the "special" category.

NOTES

APPENDIX A

CBL PERFORMANCE DATA

The following tables list the operating speed and memory requirements (words) of each type CBL block. This data allows the calculation of execution time requirements for segments containing numerous types of blocks. It also shows the magnitude of memory required by individual blocks and segments.

To calculate the minimum execution time of a segment, add all the LO run times for each type of block within a given segment. Multiply the resulting sum times the number of occurrences of that type segment. The maximum execution time can be determined similiarly to the minimum execution time except that H1 run time is used. Use the AVG column to obtain average execution time. Note that performance figures do not appear in some of the data columns. In most of these cases, the blocks function only to supply information to the FSS translator. They do not occupy memory space and no CBL macro is produced.

LOGIC BLOCKS

NAME		RUN TIME (SEC X 10^{-6})			MEMORY REQUIRED (WORDS)		
BLOCK	PAGE NO	LO	HI	AVG	MIN	MAX	AVG
LTDL		20	23	22	8	8	8
LSHT		21	21	21	6	6	6
LFSR		15	22	19	6	6	6
LFFD		10	22	16	6	9	6
LCMP		15	15	15	5	8	5
LAND*		22	53	38	6	15	7
LORI*		22	53	38	6	15	7
LANO*		22	53	38	6	15	7
LORO*		22	53	38	6	15	7
LGIC**		30	238	134	8	62	35
LXIT		3	9	6	5	5	5
LRCT		63	63	63	12	12	12
LRLY***		30	369	200	5	54	30

* Low time is for one input. Total time = 22 + 8n where n is number of additional inputs.

** Low time is for two terms. Total time = 30 + 8n + 4s where n is number of additional terms and s is number of intermediate results.

*** Low time is for two contacts on one rung. Total time = 30 + 8n + 4s where n is additional number of contacts and s is number of stack operations.

SPECIAL BLOCKS

NAME		RUN TIME (SEC X 10^{-6})			MEMORY REQUIRED (WORDS)		
BLOCK	PAGE NO.	LO	HI	AVG	MIN	MAX	AVG
CNST		-	-	-	-	-	-
DATV		-	-	-	-	-	-
INZO		9	187	98	3	49	26
INZS		9	187	98	3	49	26
INZV		10	729	370	6	6	6
DIDB		-	-	-	-	-	-
PDATV		-	-	-	-	-	-
CTRF		3	3	3	5	18	12
RBEG		5	6	6	5	5	5
RCALL		5	5	5	10	36	23
REND		4	4	4	3	3	3

NOTE: A dash (-) in RUN TIME column indicates the block is not executable.

COUNT BLOCKS

	RUN TIME SEC X 10^{-6}			MEMORY REQUIRED (WORDS)		
NAME	LO	HI	AVG	MIN	MAX	AVG
CADD	7	7	7	6	6	6
CCMP	14	14	14	7	7	7
CCNV	14	24	19	6	6	6
CCTL	13	13	13	6	6	6
CICB	7	20	14	6	6	6
CLTC	11	11	11	6	6	6
CNTR	12	18	15	7	7	7
CSHR	13	16	15	7	7	7
CSUB	7	7	7	6	6	6
CTMR	12	12	12	6	6	6
CPAK	22	32	2	7	10	8
COCB	7	26	17	6	6	6

PROCEDURAL BLOCKS

NAME		RUN TIME SEC X 10^{-6}			MEMORY REQUIRED (WORDS)		
		LO	HI	AVG	MIN	MAX	AVG
PACLP	4-117			31.3	7	7	7
PACTV	4-119			*300	19	25	25
PAFGT	4-123			*300	8	8	8
PCALC	4-125			161	7	13	10
PCALL	4-127			4.56	4	4	4
PCONV	4-129			18.7	6	6	6
PDELY	4-131			*300	17	31	31
PFIND	4-139			*300	9	9	9
PFOFX	4-141			*300	10	10	10
PLGIC	4-145			10.95	7	7	7
PMAIN	4-147			3.81	3	3	3
PMEND	4-149			*300	8	8	8
PMOVE	4-151			7.46	6	6	6
PRETN	4-153			4.56	4	4	4
PSUBM	4-155			4.56	4	4	4
PSUSP	4-157			*300	13	13	13
PTERM	4-159			**2.86	4	4	4
PTRKO	4-163			20.92	5	5	5
PTRKR	4-166			19.36	5	5	5
PTXTI	4-167			*300	17	17	17
PTXTO	4-171			*300	23	23	23
PTXTS	4-177			34.32	6	6	6
PDOTL	4-135			26.05	9	9	9
PDOWL	4-137			11.39	7	7	7
PLCHK	4-143			58.34	9	9	9
PTEST	4-161			42.34	8	8	8

** ADD TIME FOR PMEND

* ESTIMATED

ANALOG BLOCKS

NAME		RUN TIME (SEC X 10^{-6})			MEMORY REQUIRED (WORDS)		
BLOCK	PAGE	LO	HI	AVG	MIN	MAX	AVG
AABS		10	12	11	5	5	5
ACIN		131	153	141	12	12	12
ACLP		31	37	34	7	7	7
ACMP		18	36	27	7	7	7
ACNV		18	31	25	6	6	6
ADIF		107	209	158	11	11	11
ADTL ①		16	168	92	7	7	7
ADVD ②		61	61	61	7	7	7
AFGS ③		58	100	79	6	6	6
AFGT		21	281	151	8	8	8
AGAI		53	53	53	7	7	7
AHLD		9	15	12	6	6	6
AICB		63	90	77	8	8	8
ALLG		284	303	294	11	11	11
AMAX		28	38	38	7	7	7
AMIN		23	23	23	7	7	7
AMPY		53	53	53	7	7	7
ANSW		14	24	19	8	8	8
AOCB		54	77	62	8	8	8
APIN		192	214	203	13	13	13
ASQR		142	142	142	5	5	5
ASUM ④		59	94	77	7	11	9
AVRG		16	168	92	7	7	7

NOTES:

① Run time dependent upon length of list and the initial or other execution.

② Run time dependent on which of seven linear equations required.

③ Run time dependent upon vector size and interpolation requirements.

④ Run time dependent upon length of list and the initial or orther execution.

APPENDIX B

REUSABLE SEQUENCE

1. PURPOSE

In many systems employing DIRECTO-MATIC ② Plus equipment, the same control functions must be provided for numerous different types of drives. The intent of a reusable sequence is to allow a sequence of control blocks to be defined only once and then subsequently be reused for several different drives.

2. APPROACH

A reusable sequence is defined and used through the use of 3 control blocks; RBEG, REND and RCALL. The RBEG and REND are used to identify the beginning and end of a reusable sequence of control blocks. An important additional function of the RBEG block is to define the name, input and output parameters of the reusable sequence.

The RCALL is used to cause the PMC to perform the reusable sqeuence. The name of the reusable sequence and specific input and output signals required for a particular drive are defined in the RCALL block. When the PMC controller encounters the RCALL macro, it performs the reusable sequence of control blocks once for the specific drive. (See reusable sequence diagram on following page).

3. RESTRICTIONS

The following restrictions are inherent in the use of reusable sequences:

a. A reusable sequence must be allocated its own separate segment. This restriction allows the FSS Interaction Module to control usage of the reusable sequence.

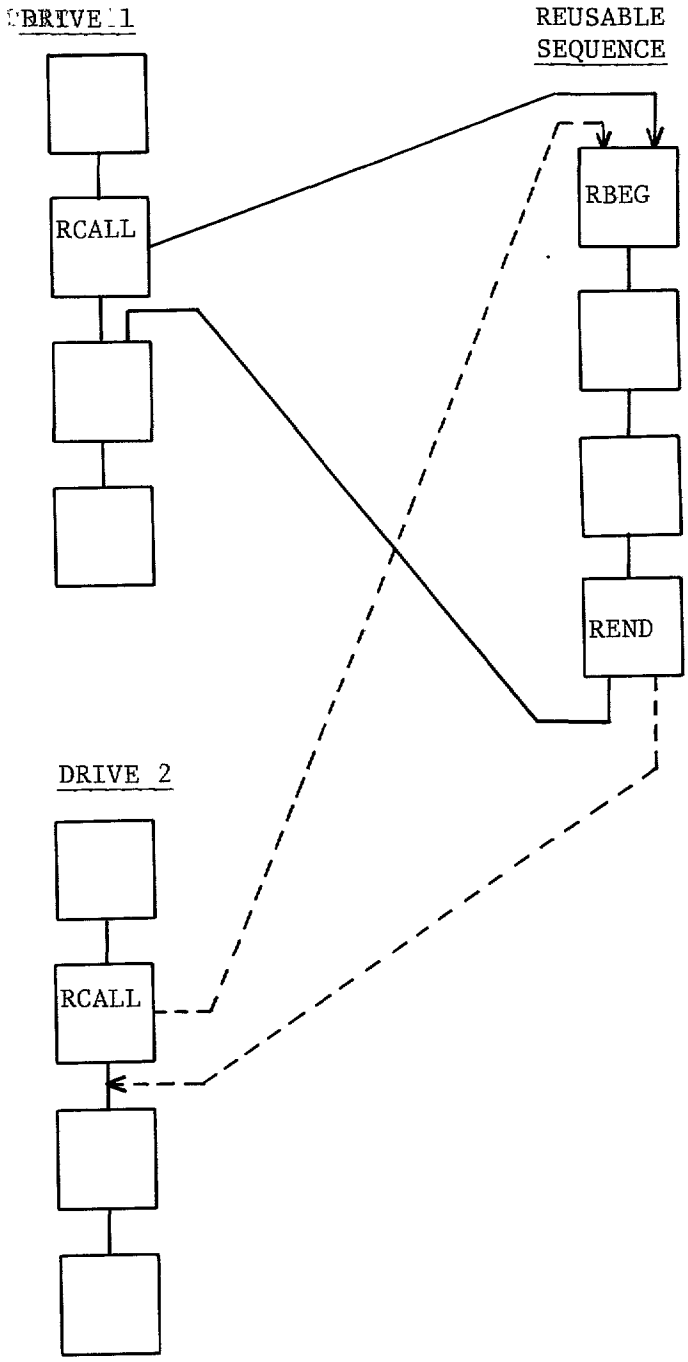
b. The drive segment number of the segment that contains the reusable sequence must be lower than that of any segment that RCALLs the reusable sequence*. This restriction allows the FSS translator to validate the RCALL parameters against the requirements defined in the RBEG.

c. Array names, vectors or CSH addresses are not permitted as input parameters in the RCALL block.

d. Use of CSH addresses are not permitted within a reusable sequence.

e. The RCALL block is not permitted within a reusable sequence.

f. Signals within a reusable sequence cannot be initialized by use of the INZO block.


DRIVE 1
REUSABLE SEQUENCE
RCALL
RBEG
REND
DRIVE 2
RCALL

*NOTE: A drive segment name consists of the combination of the drive number and segment designator. For example, drive 2 and segment B would be designated 2B. Drive segment designators are ordered by the lowest drive number and segment name. EXAMPLE: Segment name 2B has a RCALL to a reusable sequence whose number must be lower than 2B, e.g., 2A, 1Z, 1M, 1A, etc.

4. NOTES

The following features of reusable sequencer should be noted:

a. The use of array names and vectors are permitted within a reusable sequence. An input parameter may be used as a subscript variable.

b. The use of an LXIT block within a reusable sequence will bypass all further processing of blocks in the RCALL segment, as well as all further processing of blocks in the reusable sequence.

APPENDIX C

CBL MACRO INSTRUCTION FORMATS AND ADDRESSING

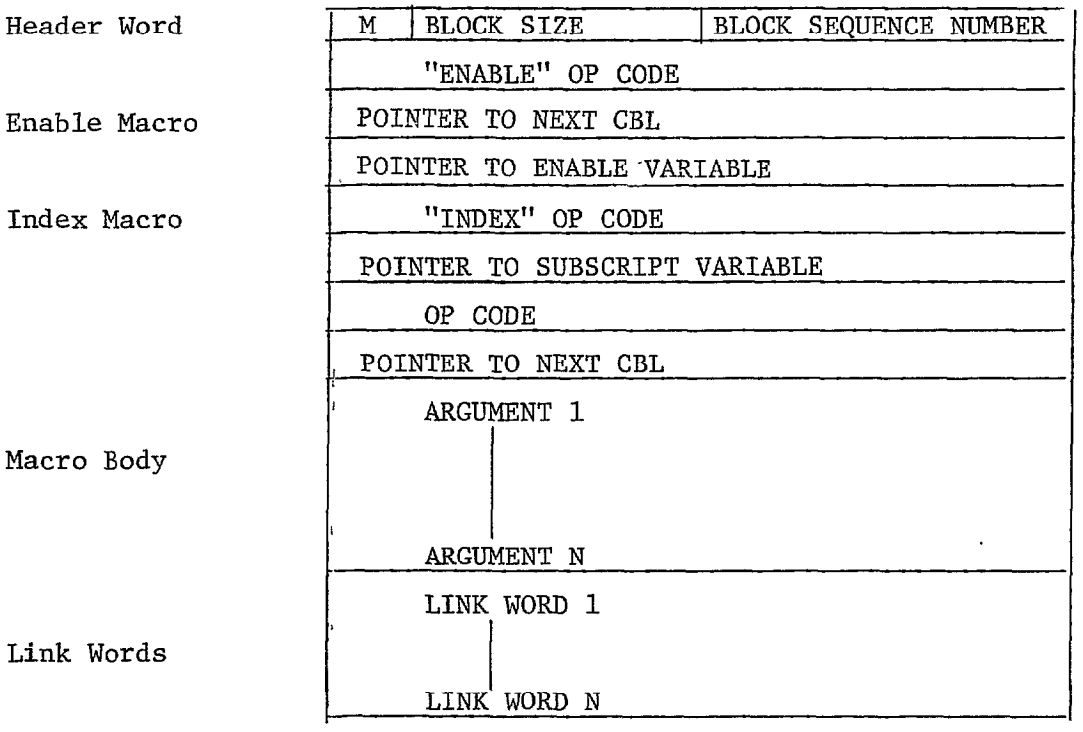
1. INTRODUCTION

The CBL macro instruction formats defined in the body of this publication show the simplest form of macro generated.

These formats do not consider enable variables, subscripted variables and the different forms of variable addressing. These factors are explained in this appendix.

2. OVERALL MACRO FORMAT

A CBL macro diagram is shown below and its major parts are described in subsequent paragraphs.



a. Header Word

M - Modify flag indicates that the macro in the PMC has been changed from its original FSS translated form.

BLOCK SIZE - The number of words in the macro.

BLOCK SEQUENCE NUMBER - The identifying number of the block in the segment/drive.

OPERATION CODE - Indicates the PMC operation that must be accomplished to execute this block. (Unique for each type of block).

b. Enable Macro

The enable macro is optional and is only included if an Enable Variable is defined for the control block. The format of an Enable Macro is as follows:

"ENABLE" OP CODE
POINTER TO NEXT CBL
POINTER TO ENABLE VARIABLE

(105516 octal)

The POINTER TO ENABLE VARIABLE contains the address of the enable variable, which will always be type logical. If the enable variable is true, the control block is executed. If the enable variable is false, the control block is bypassed (i.e., the PMC will proceed to execute the control block defined in POINTER TO NEXT CBL).

c. Index Macro

The index macro is optional and is only included if the control block refers to an array element using a variable subscript. The format of an index macro is as follows:

INDEX OP CODE
POINTER TO SUBSCRIPT VARIABLE

(105514 octal)

The POINTER TO SUBSCRIPT VARIABLE contains the address of the subscript variable which is always a type count. the PMC saves the subscript variable in an internal hardware register (called the Index Register). This data is later used to resolve references to array elements within the macro body. This process is described

c. Index Macro (Continued)

in detail in paragraph 5,a.(Indexed Addressing). An index macro is not required if the control block refers to an array element using a Constant Subscript.

d. Link Words

Link words are optional and are only included if the control block refers to an array element using a variable subscript. They are described in detail in paragraph 5,a.

3. VARIABLE ADDRESSING

The PMC uses several different methods of variable addressing. The method used depends on the type of variable and how it is used (addressing mode).

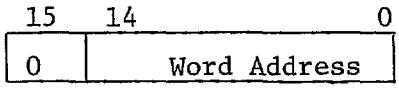
4. VARIABLE TYPES

Although the user recognizes many types of variables (e.g., signals, scalars, constants, temporary variables, etc.) as far as the variable addressing is concerned, the PMC only recognizes 3 different types; i.e., Count, Analog or Text, Logical and Logical with Op Code.

a. Count, Analog or Text

The count, analog or text types of variable address references a word (or the first of several words) in the PMC memory that contains a count, analog or text variable.

Considering the normal address mode for this type of variable, the format of the argument word in macro body is as follows:



The word address is an absolute address of the variable in the PMC memory.

b. Logic

The logic type variable address references a bit on the PMC memory that contains a logical variable.

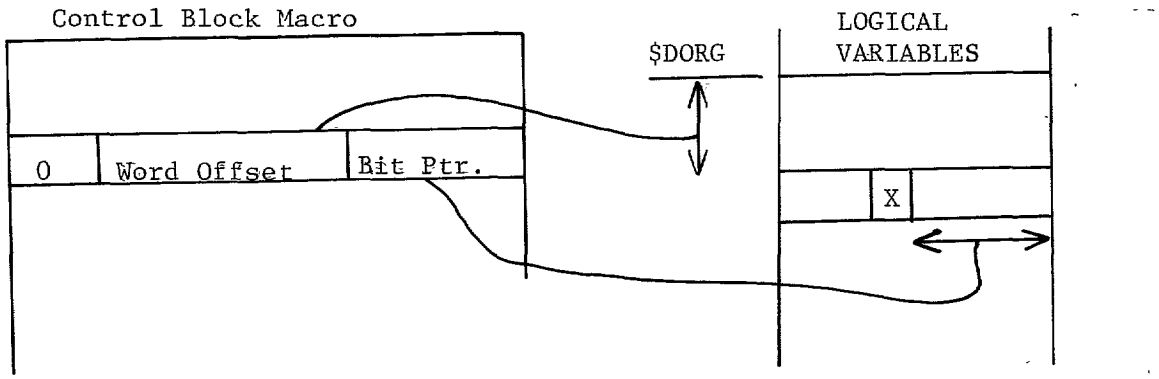
b. Logic (Continued)

Considering the normal address mode for this type of variable, the format of the argument word in the macro body is as follows:

15	14 ... 4	3 ... 0
0	Word Offset	Bit Pointer

All logical variables are grouped together in one area of PMC memory. The starting address of this area is called $DORG.

The PMC adds the word offset to $DORG to locate the required word. The PMC then uses the required word. The PMC then uses the bit pointer to locate the required bit within the word as diagrammed below:



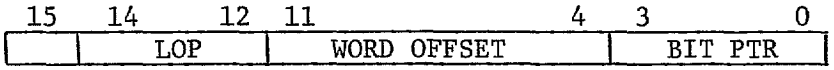
X = Required Bit

c. Logic with Op Code

The logic with op code type of variable address references a bit in PMC memory in the same way as the logic-type of variable address. The only difference is the format of the argument in the macro body which, in addition to the address information, also contains an op code. The op code defines the logical function to be performed with the associated variable; e.g., AND, OR.

This type of address is used with those macros that require this additional information, e.g., LAND, LORO, LGIC.

Considering the normal address mode for this type of variable, the format of argument word in the body of the macro is as follows:

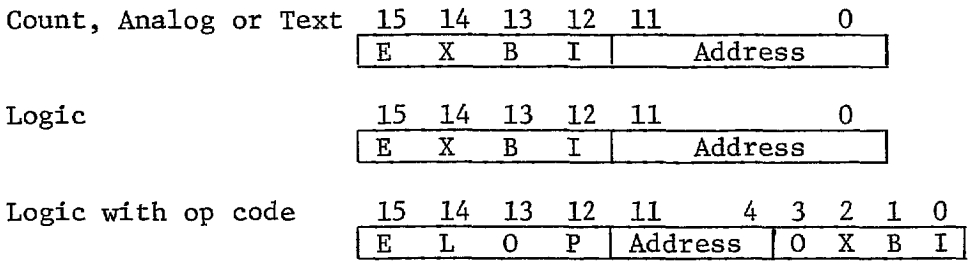


LOP is the logical op code.

The address resolution is the same as for normal logic address.

5. ADDRESSING MODES

There are three addressing modes used for special purposes. They are called extended addressing modes and are described in sub-paragraphs a thru c below. The general format of the argument word in the body of the macro is as follows:



E is the extended address bit. This will be zero for the normal address mode and 1 for any type of extended address. The X, B and I bits are used to specify the extended address mode. The contents of the address field depend on the extended address mode.

a. Indexed Addressing

This mode is used for array elements which are referenced using a variable subscript. This address mode requires that the subscript value has been loaded into the index register by a previous index macro. This mode of addressing always uses a Link word. The Link word contains the address of the array.

The format of the argument word in the body of the macro is as follows:

(1) The E bit is set to indicate extended addressing.

(2) The X bit is set to indicate the indexed mode of addressing.

(3) The address area specifies the relative offset of the link word containing the array address.

e.g.

Count, Analog, Text

E	X	B	I	Address
1	1	0	T	T* LINK Offset

* (T = Variable Type: 1 - Count, 2 - Analog, 3 - Text)

Logical

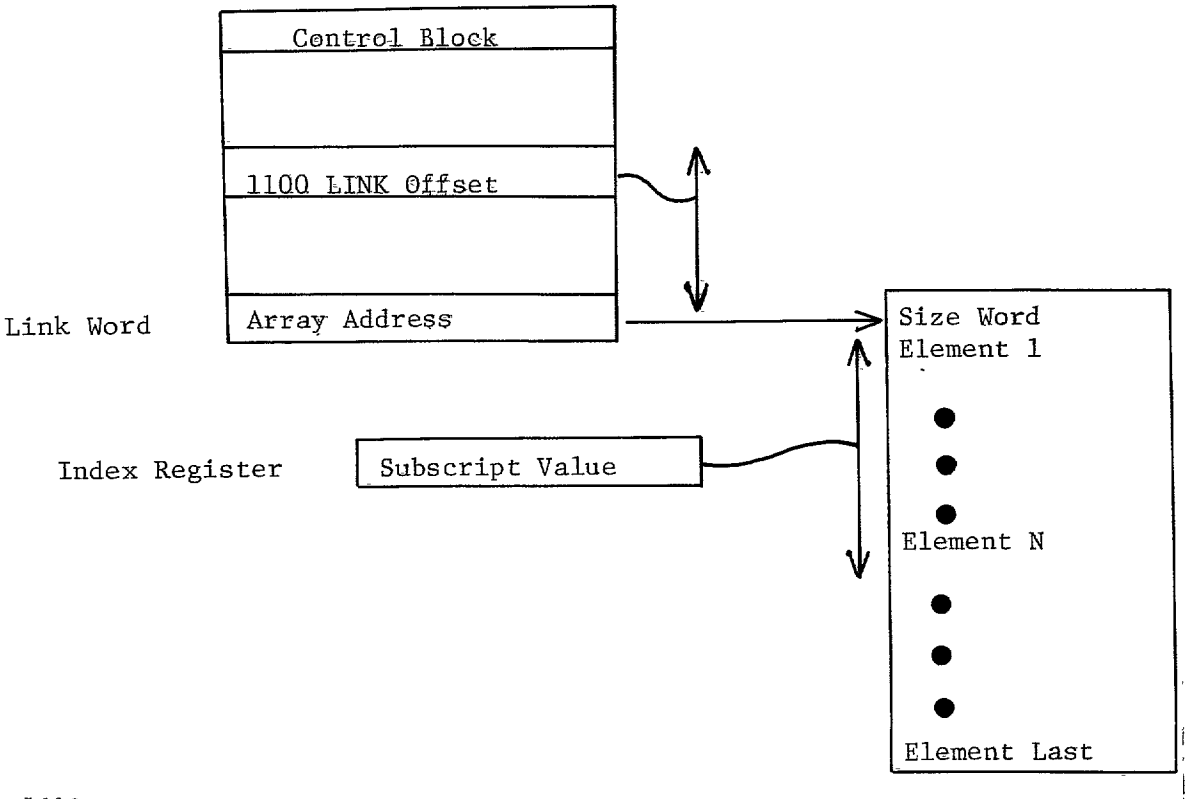
E	X	B	I	Address
1	1	0	0	LINK Offset

Logical with Op Code

E	L	O	P	Address	X	B	I
1	L	0	P	LINK Offset	0	1	0	0

The PMC uses the LINK Offset to obtain the address of the array from the associated link word. The PMC then uses the subscript value in the index register to relocate the required array element.

e.g.



b. Based Addressing

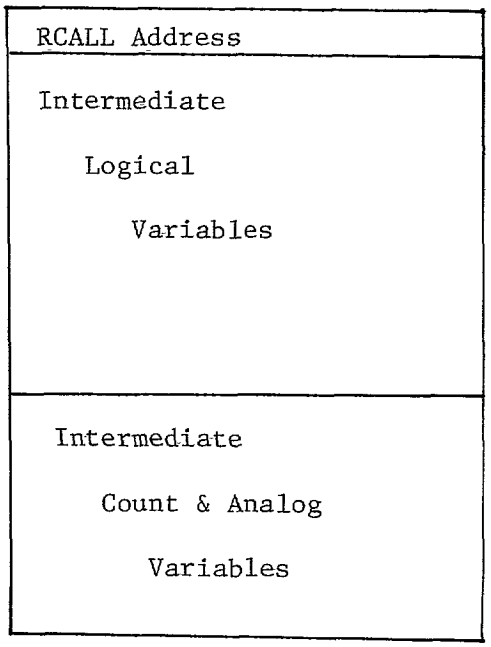
The signals and temporary variables within a reusable sequence are called Intermediate Variables. The based address mode is used for all references to Intermediate Variables.

The PMC maintains a separate set of intermediate variables for each user of the reusable sequence. This is done for two reasons:

(1) To eliminate the possibility of interference between different users of the reusable sequence.

(2) To permit diagnostic analysis of what the reusable sequence is doeing for one specific user.

All the intermediate variables for one user are grouped together in a structure called the Intermediate Table. The format of the intermediate table is as follows:



Each user of a reusable sequence will have his own copy of the intermediate table, and hence his own unique set of intermediate variables.

One of the parameters in the RCALL macro is the address of the intermediate table. One of the functions of the RCALL macro is to load this address into an internal hardware register (called the Base Register) before transferring control to the reusable sequence.

Within a reusable sequence, all references to intermediate variables use the based mode of addressing. The format of the argument word is as follows:

(1) The E bit is set to indicated Extended Addressing.

(2) The B bit is set to indicate the Based mode of addressing.

(3) The address area specifies the offset of the variable within the Intermediate Table.

e.g.

Count, Analog

E X B I	Address
1 0 1 0	I.T. Offset

Logical

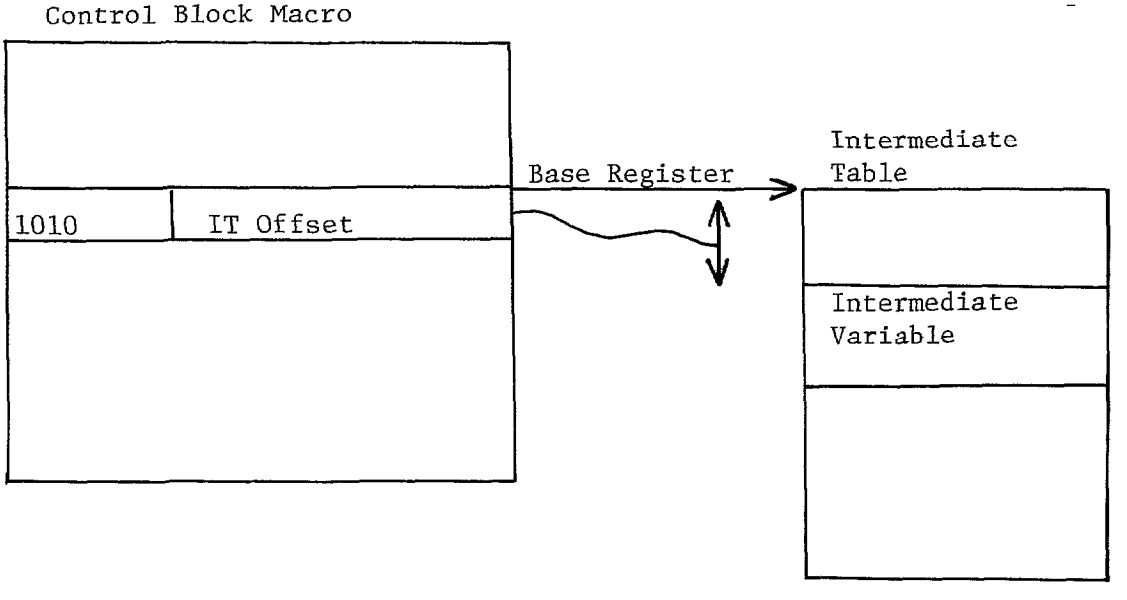
E X B I	Address	
1 0 1 0	I.T. Offset	Bit Ptr.

Logical with Op Code

E L O P	Address		O X B I
1 L O P	I.T. Offset	Bit Ptr	0 0 1 0

The PMC adds the contents of the Base Register to I.T. offset to resolve the address.

e.g.



c. Indirect Addressing

The RBEG block defines those signals that are the input and output parameters of the reusable sequence. The indirect addressing mode is used for all references to input/output parameters.

The RCALL block defines the particular input and output signals required for one specific user of the reusable sequences. These will always be in the form of normal mode variable addresses.

(1) The E bit is set to indicate the Indirect mode of addressing.

(2) The B and I bits are set to indicate the Indirect mode of addressing.

(3) The address area specifies the offset in the RCALL macro where the variable address resides.

e.g.

Count, Analog

E	X	B	I	Address
1	0	1	1	RCALL Offset

Logical

E	X	B	I	Address
1	0	1	1	RCALL Offset

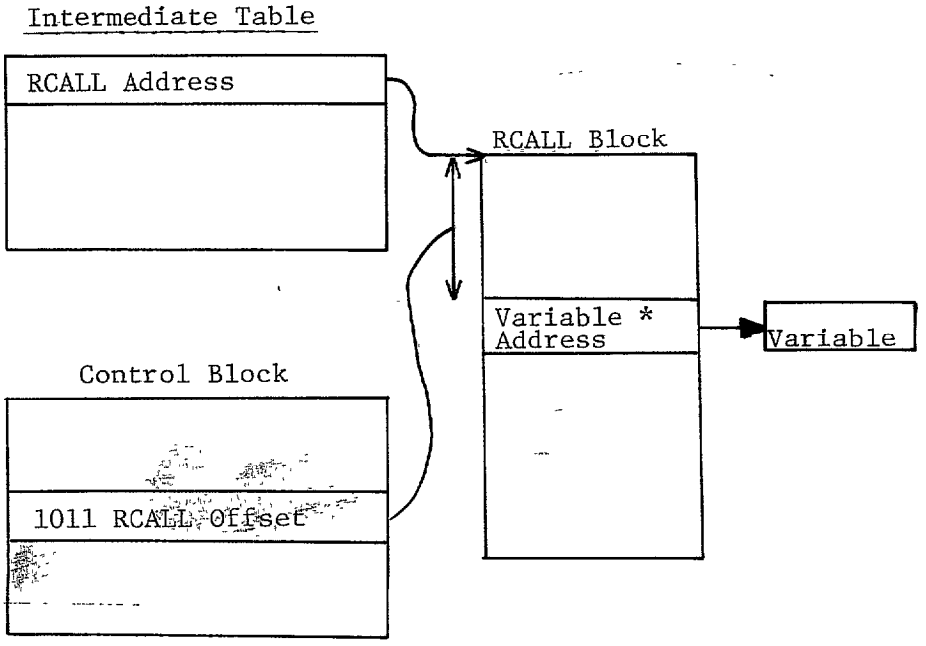
Logical with Op Code

E	L	O	P	Address	O	X	B	I
1	L	O	P	RCALL Offset	0	0	1	1

The PMC uses the Base Register to obtain the address of the RCALL macro from the Intermediate Table. The PMC adds the RCALL offset to locate the required variable address within the RCALL macro. The PMC then resolves the variable address in the normal way.

e.g.

Base Register



* Note: Logical Variables will contain Word Offset and Bit Pointer.

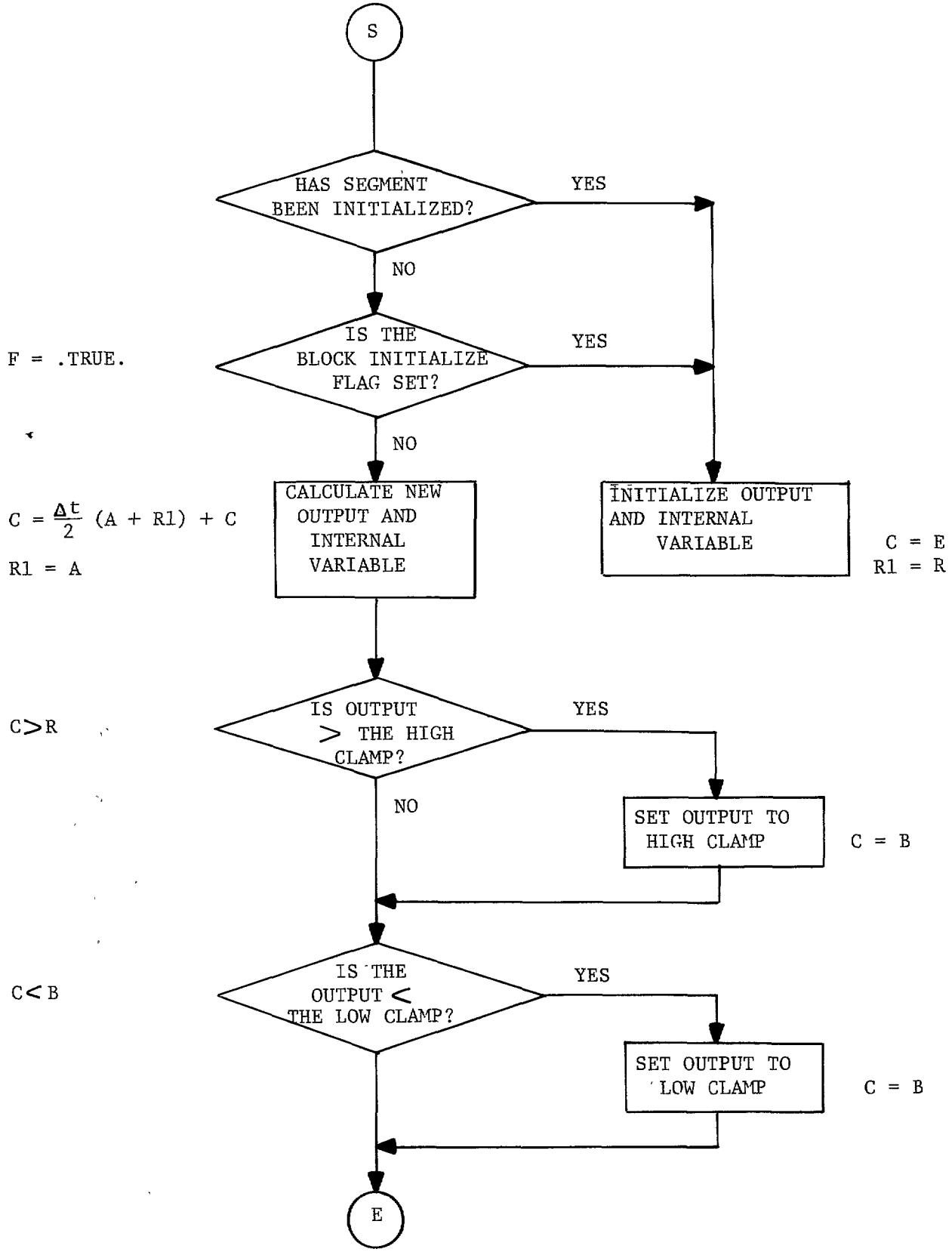
APPENDIX D

CBL FUNCTION FLOW CHARTS

The following flow charts are an aid to understanding the operational function and sequencing for some of the control blocks found in Section 4.

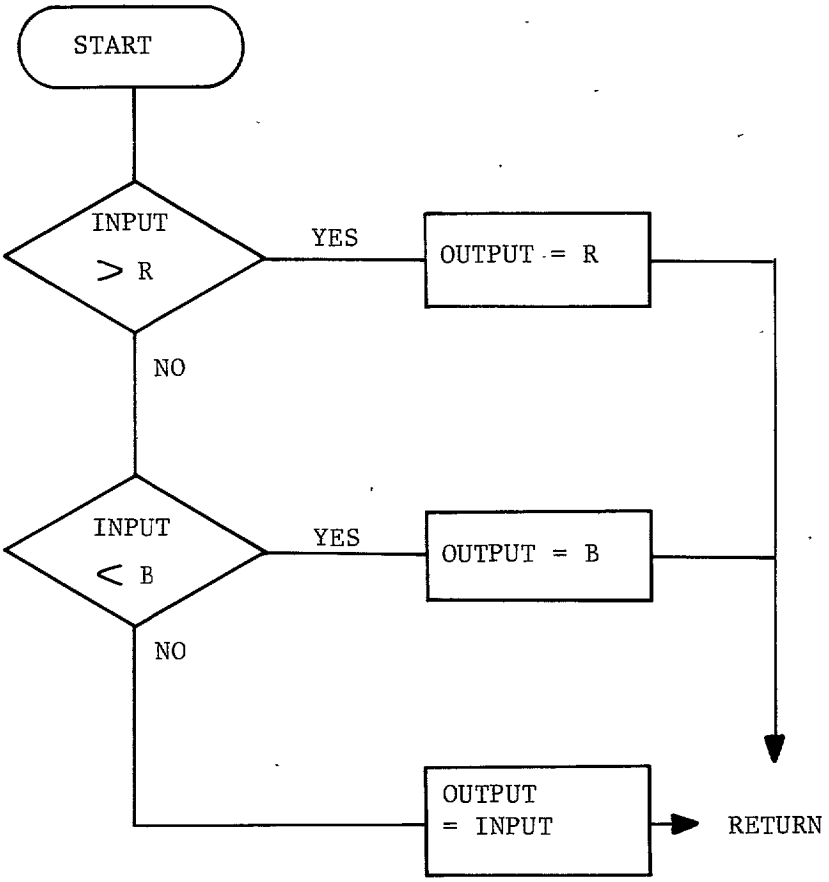
FLOW CHART
CLAMPED INTEGRATOR

ACIN



FLOW CHART

CLAMP



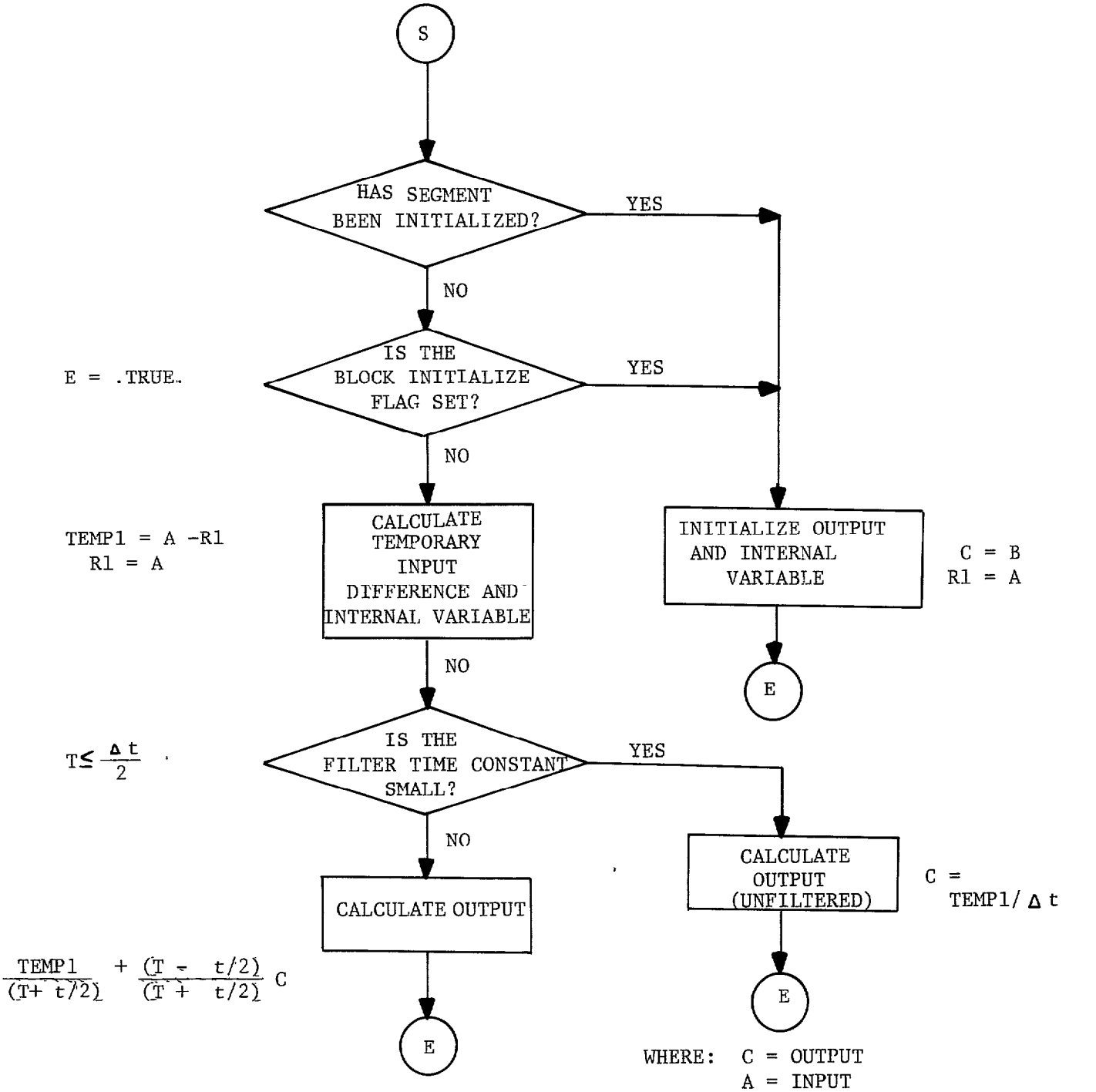
R = HIGH LIMIT

B = LOW LIMIT

ADIF

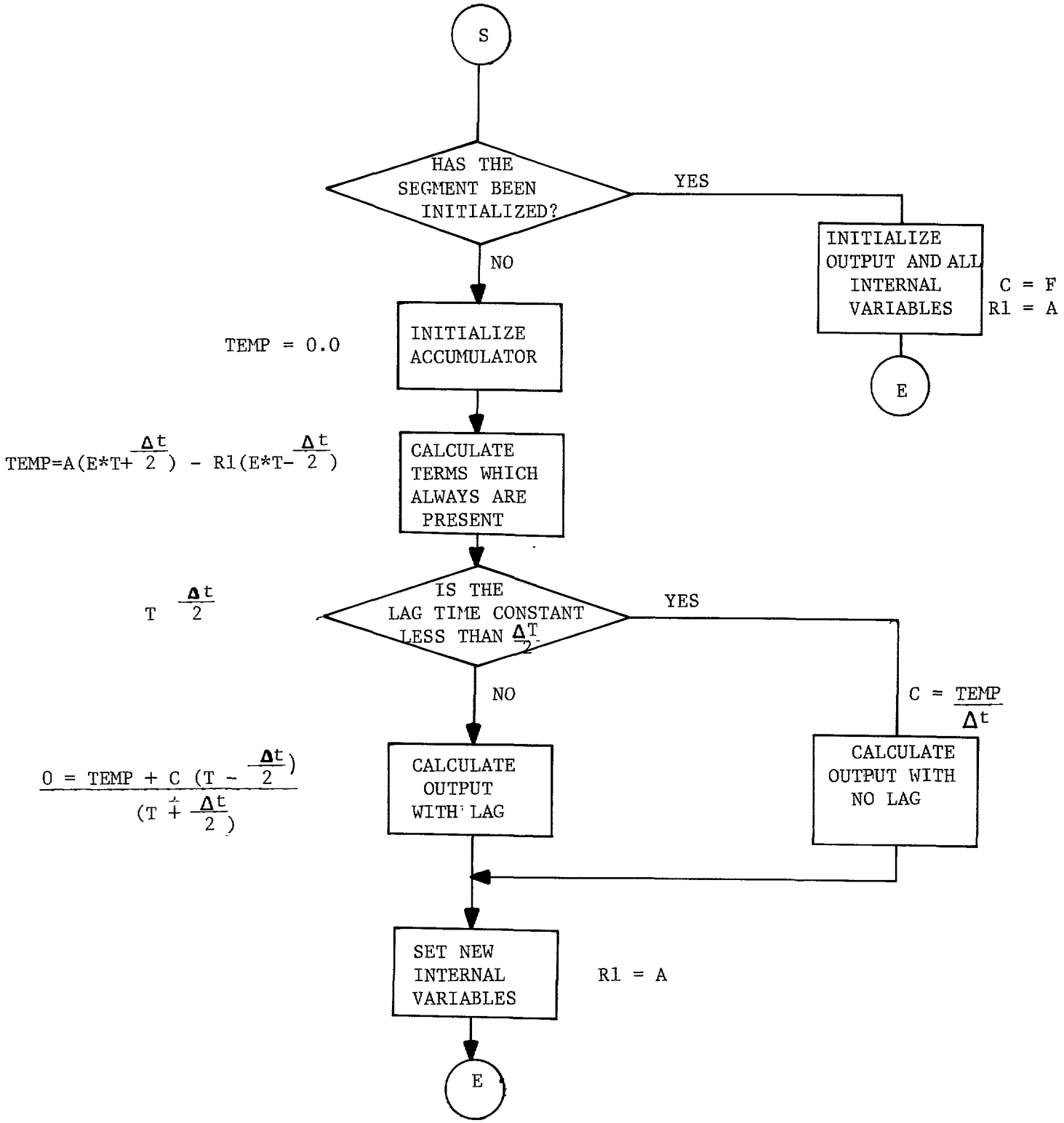
FLOW CHART

DIFFERENTIATOR



WHERE: C = OUTPUT
A = INPUT
B = INITIALIZE VARIABLE
R1 = INTERMEDIATE VALUE
Δt = SEGMENT INTERVAL
T = LAG FILTER TIME CONSTANT
E = BLOCK INITIALIZE FLAG

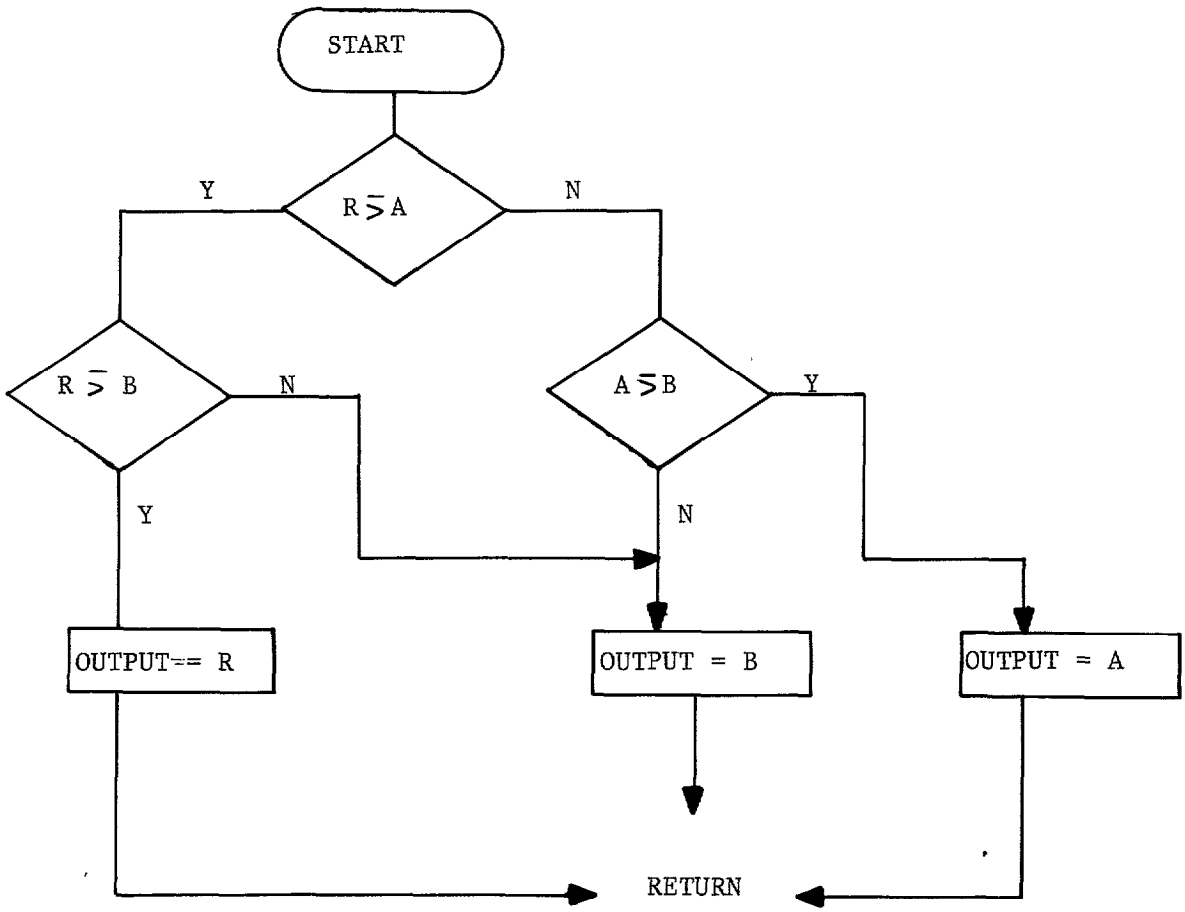
LEAD/LAG



T = Segment Interval
R1 = Intermediate Value

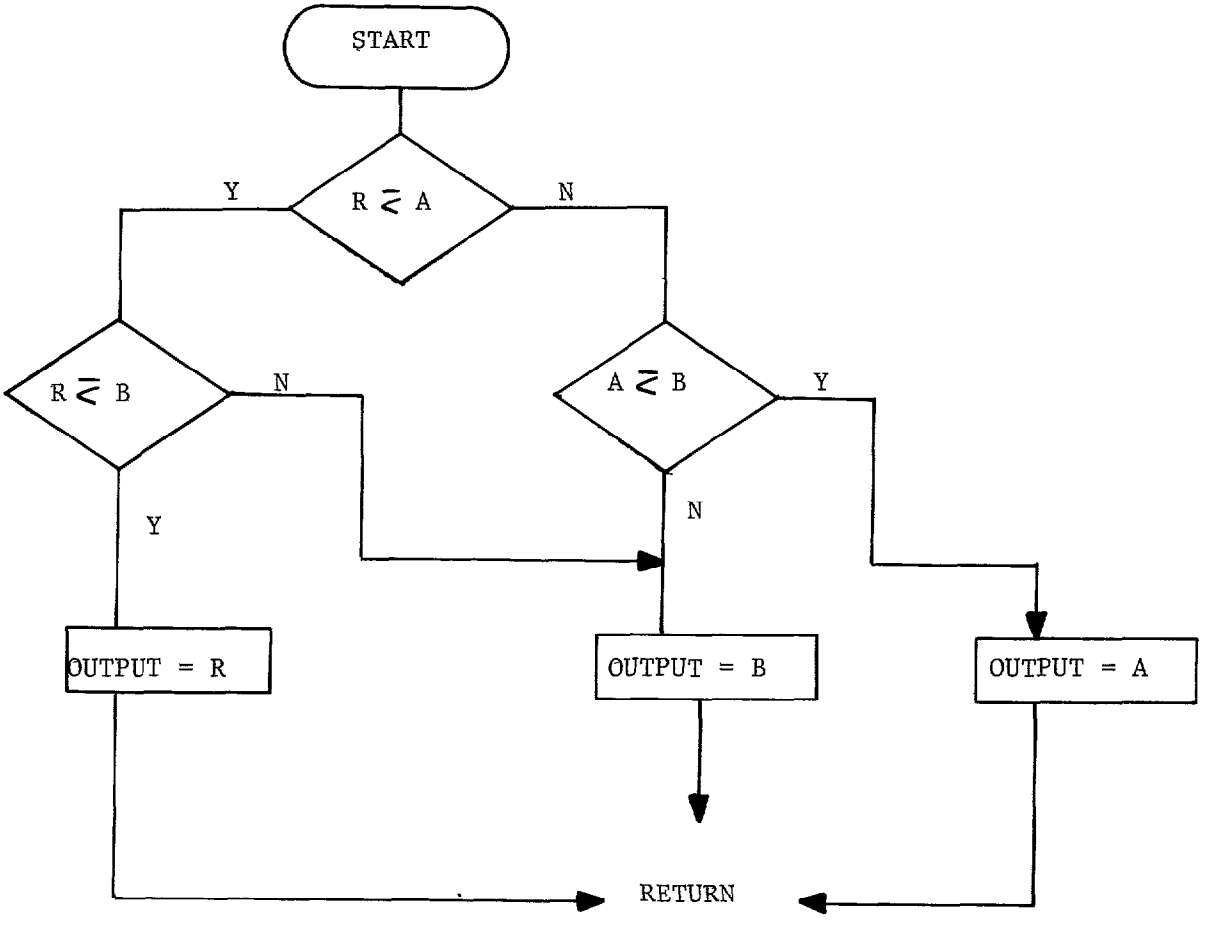
FLOWCHART

ANALOG MAXIMUM



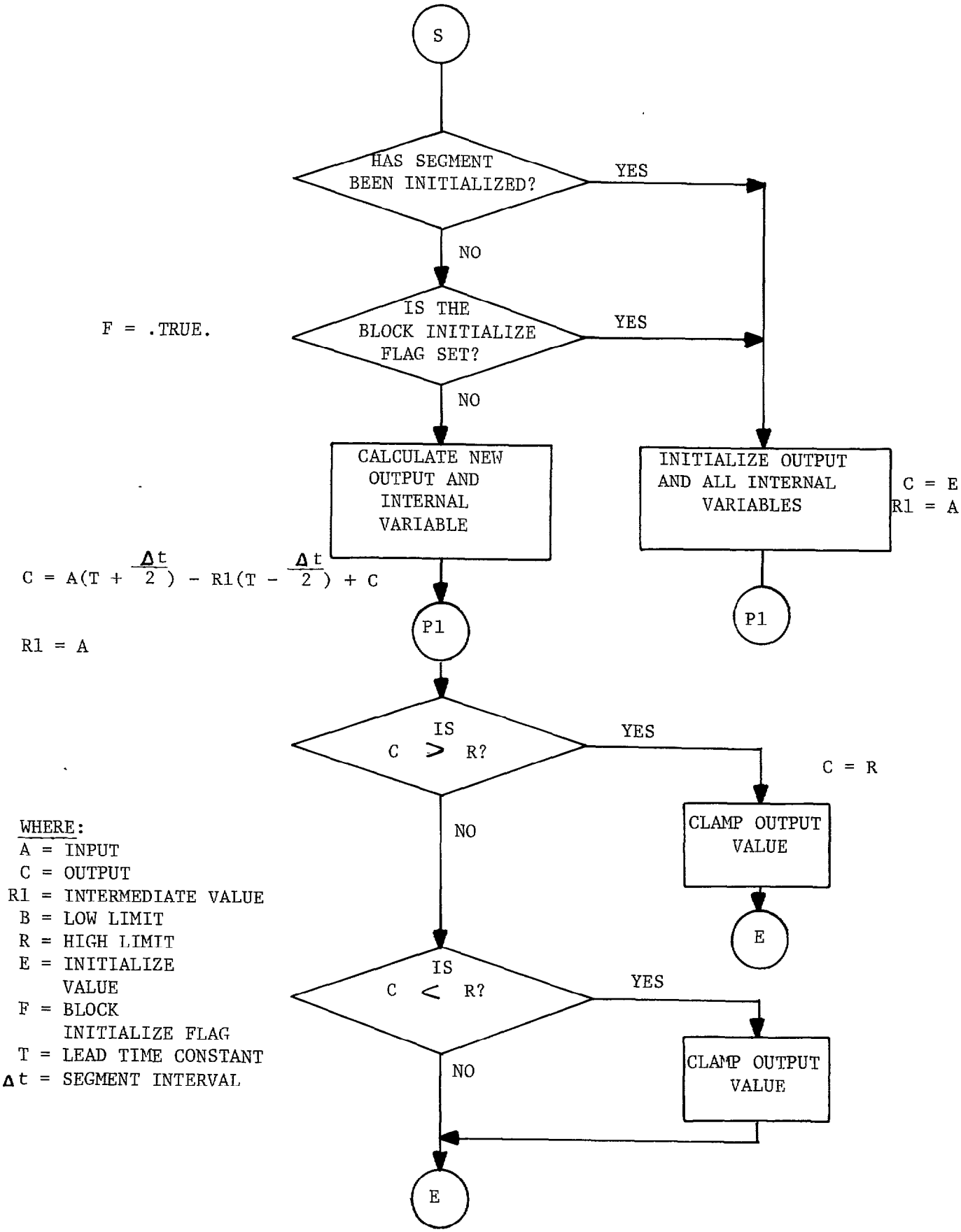
FLOWCHART

ANALOG MINIMUM



FLOWCHART

INTEGRATOR WITH LEAD



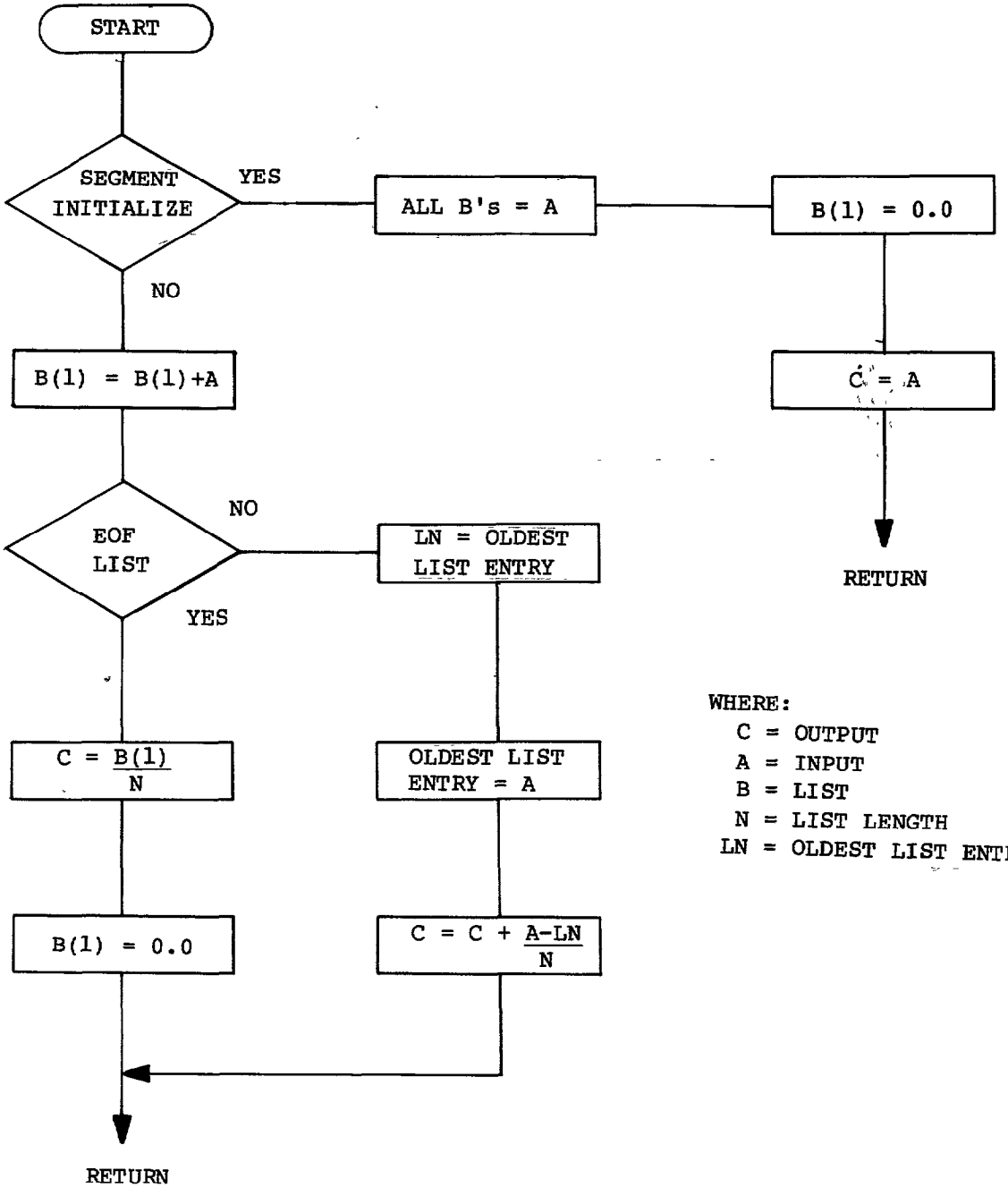
WHERE:

A = INPUT
C = OUTPUT
R1 = INTERMEDIATE VALUE
B = LOW LIMIT
R = HIGH LIMIT
E = INITIALIZE VALUE
F = BLOCK INITIALIZE FLAG
T = LEAD TIME CONSTANT
Δt = SEGMENT INTERVAL

AVRG

FLOWCHART

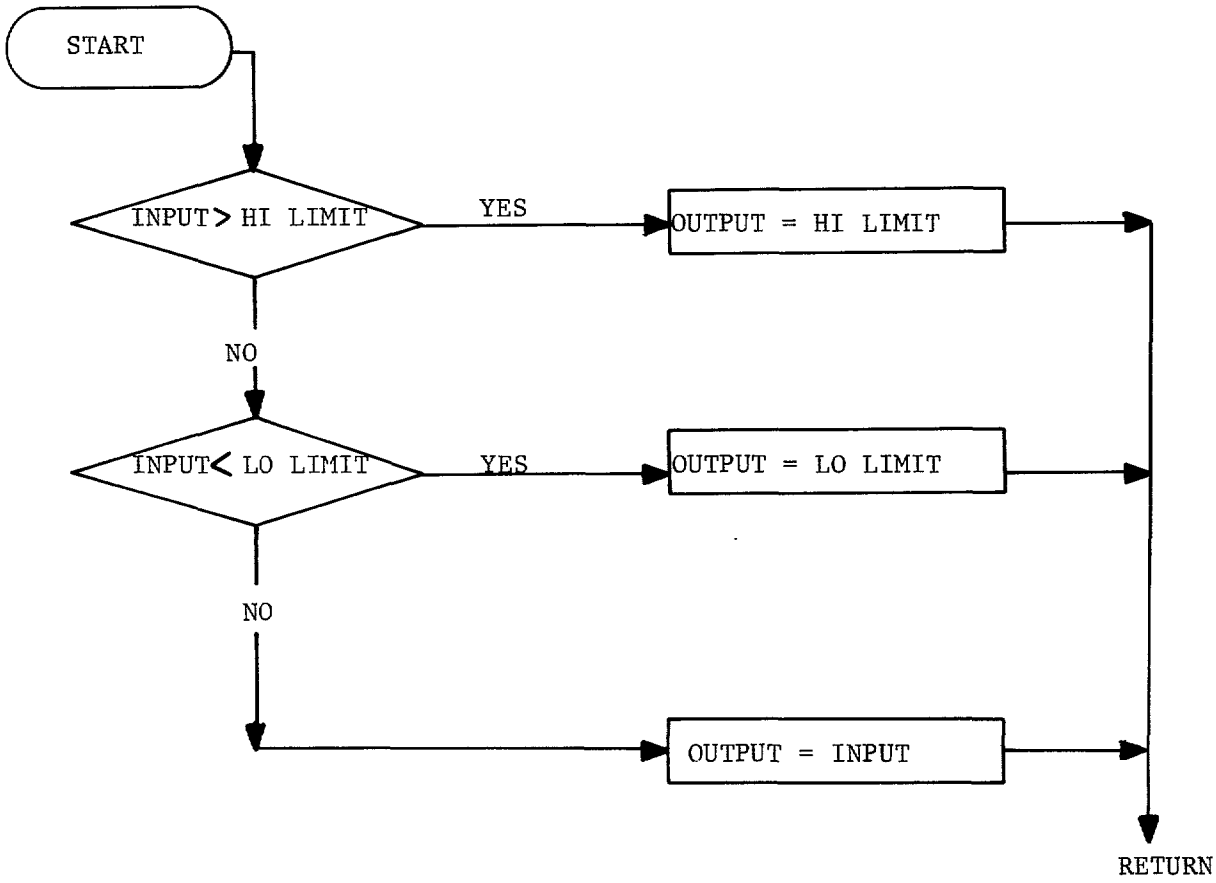
ROLLING AVERAGE



PACLP

FLOWCHART

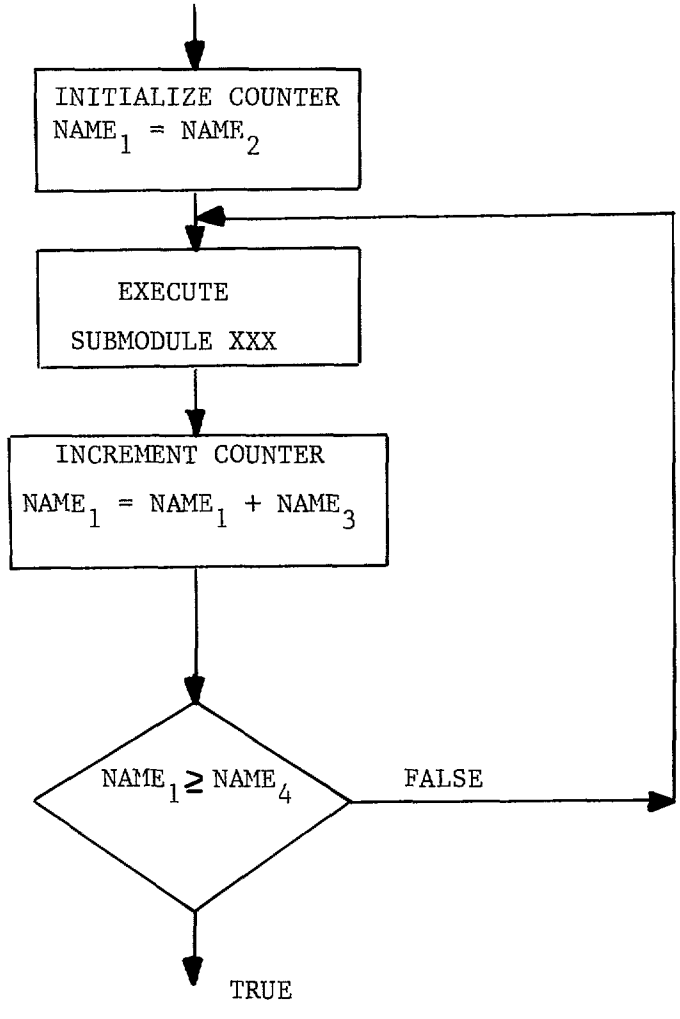
CLAMP



PDOTL

FLOWCHART

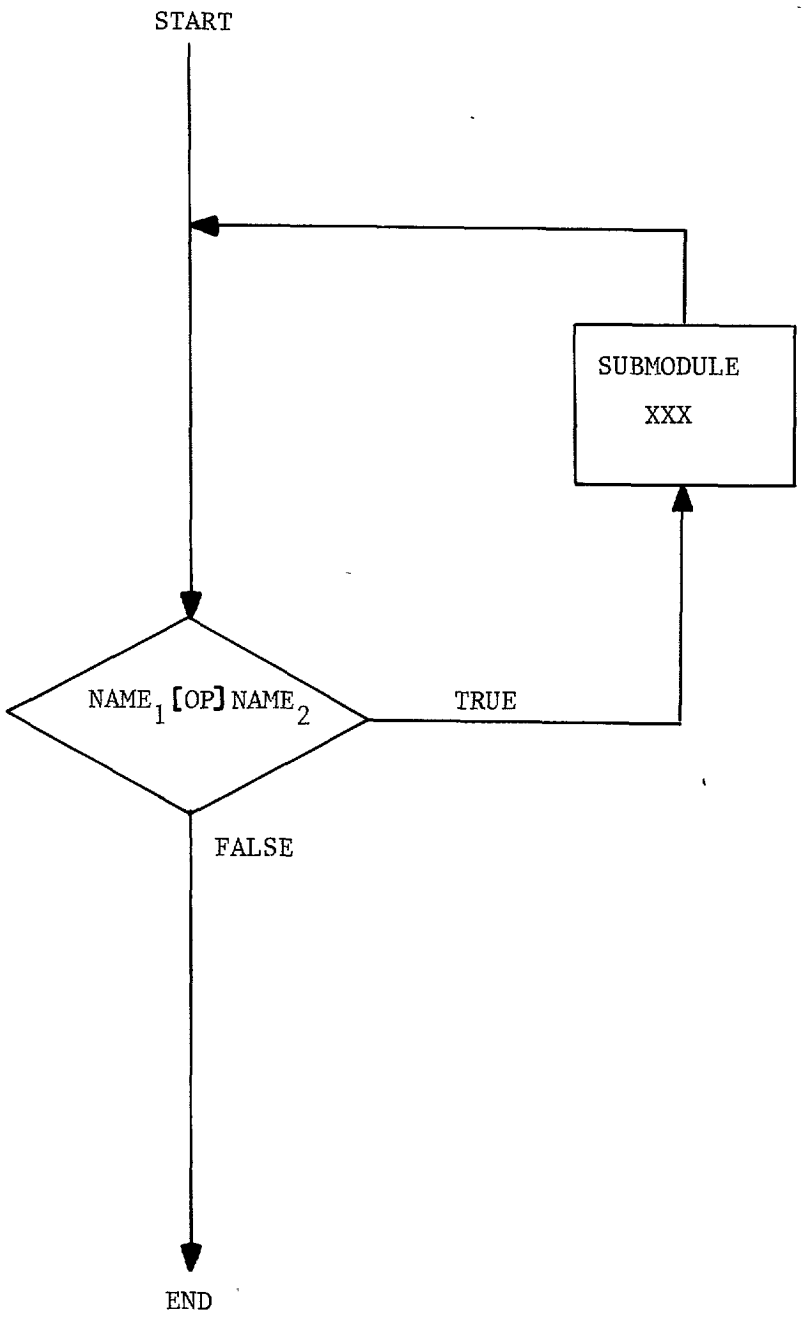
DO UNTIL TRUE



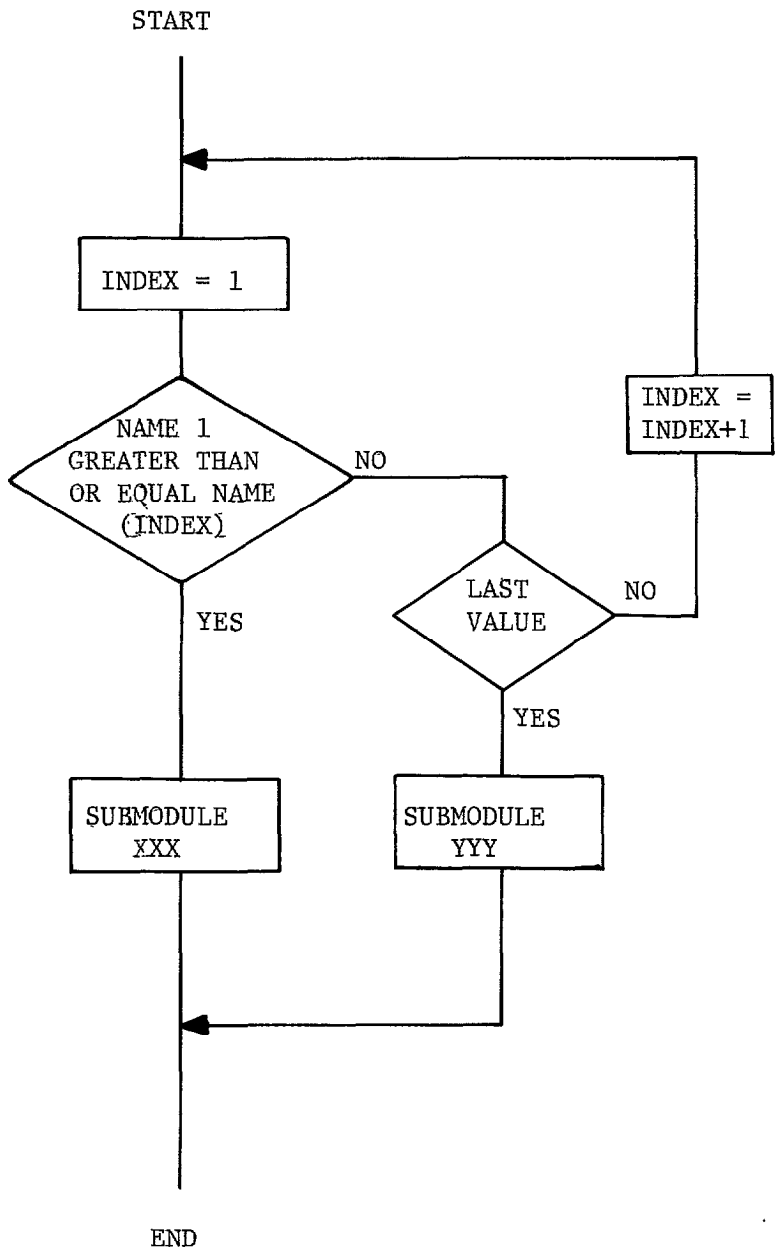
PDOWL

FLOWCHART

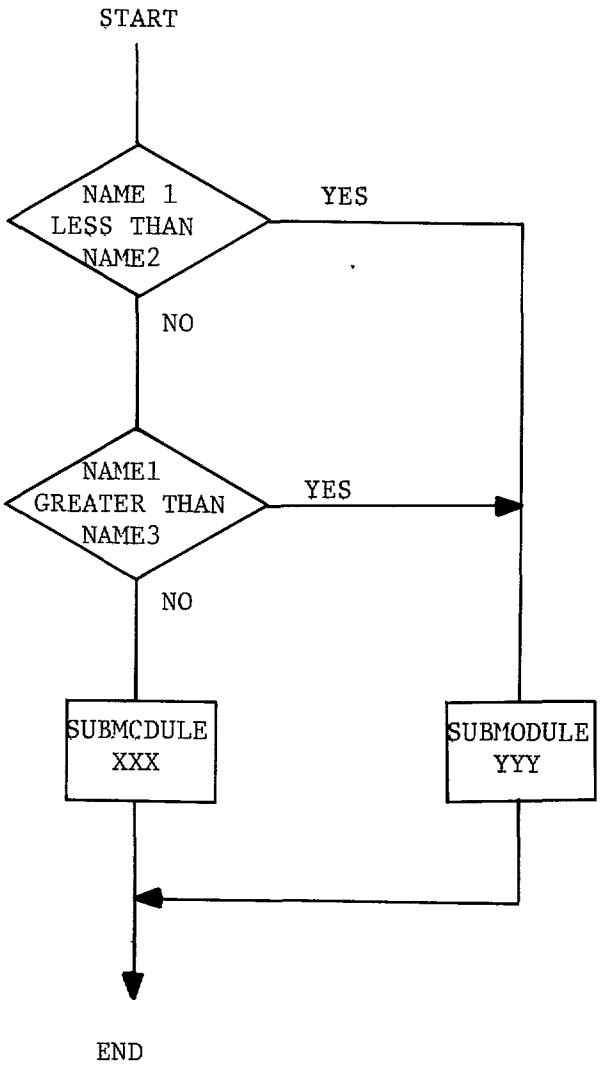
DO WHILE TRUE



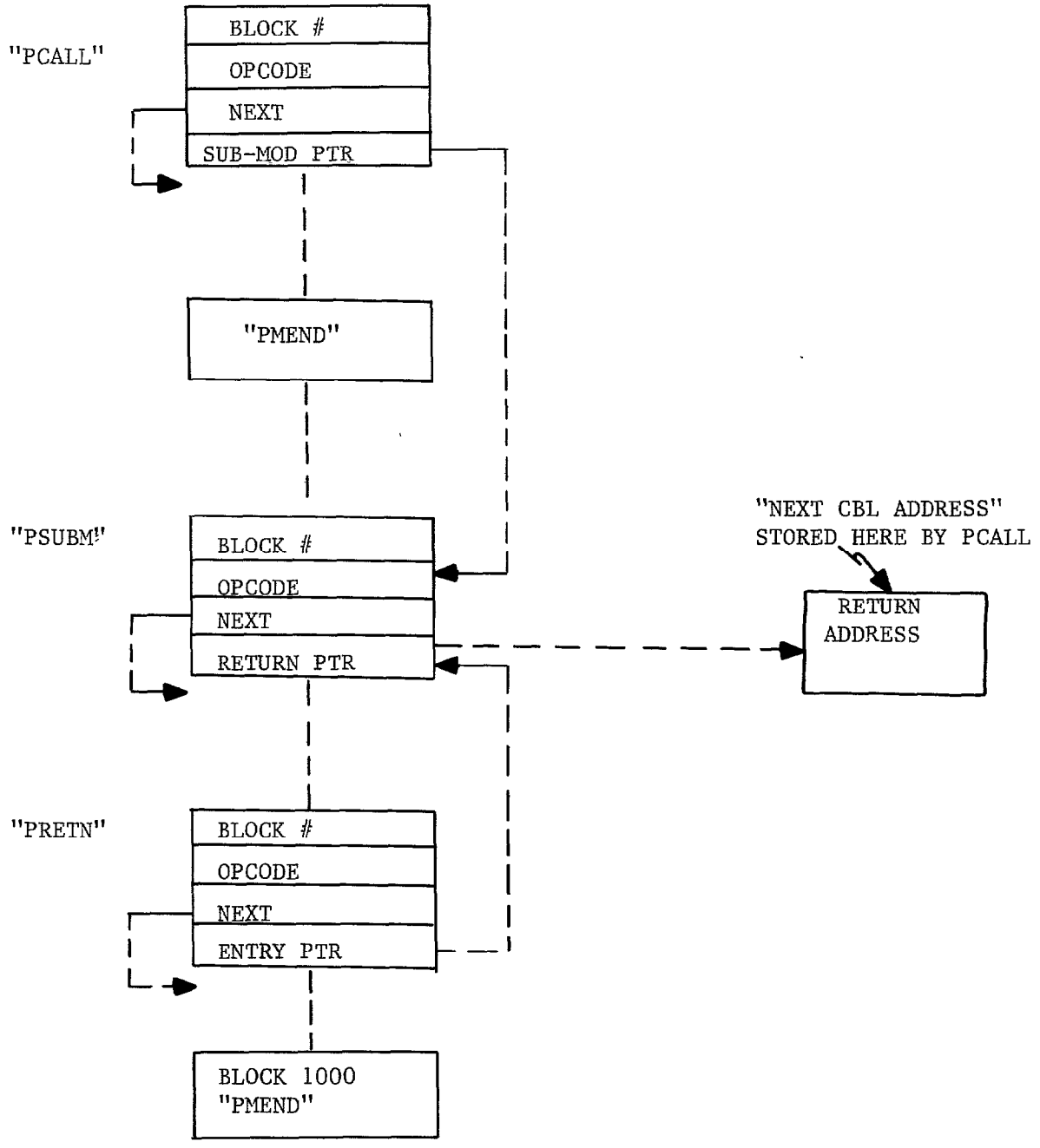
PFIND


START
INDEX = 1
NAME 1 GREATER THAN OR EQUAL NAME (INDEX)
NO
YES
INDEX = INDEX+1
LAST VALUE
NO
YES
SUBMODULE XXX
SUBMODULE YYY
END

PLCHK

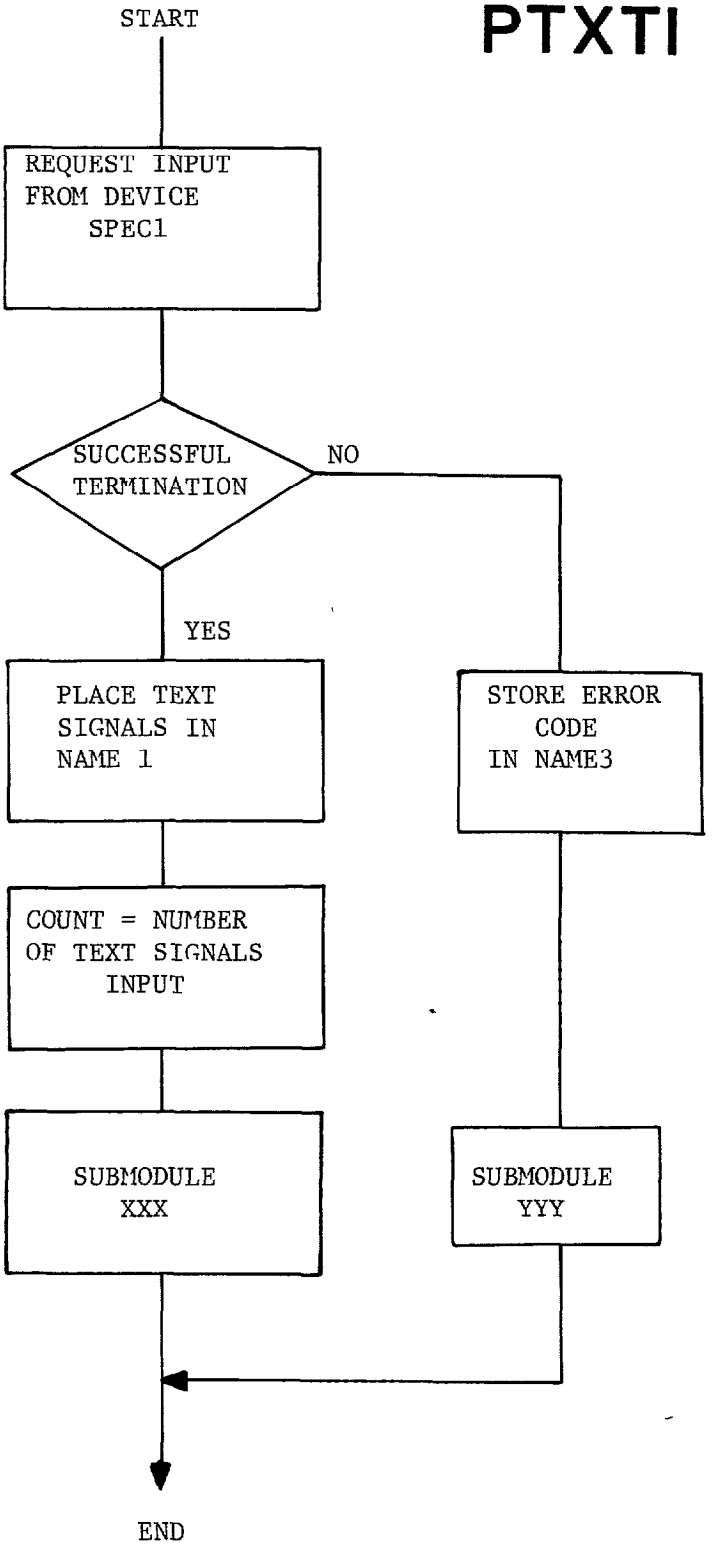

START
NAME 1
LESS THAN
NAME2
YES
NO
NAME1
GREATER THAN
NAME3
YES
NO
SUBMODULE
XXX
SUBMODULE
YYY
END

EXAMPLE OF PROCEDURE SUB-MODULE LINKAGE

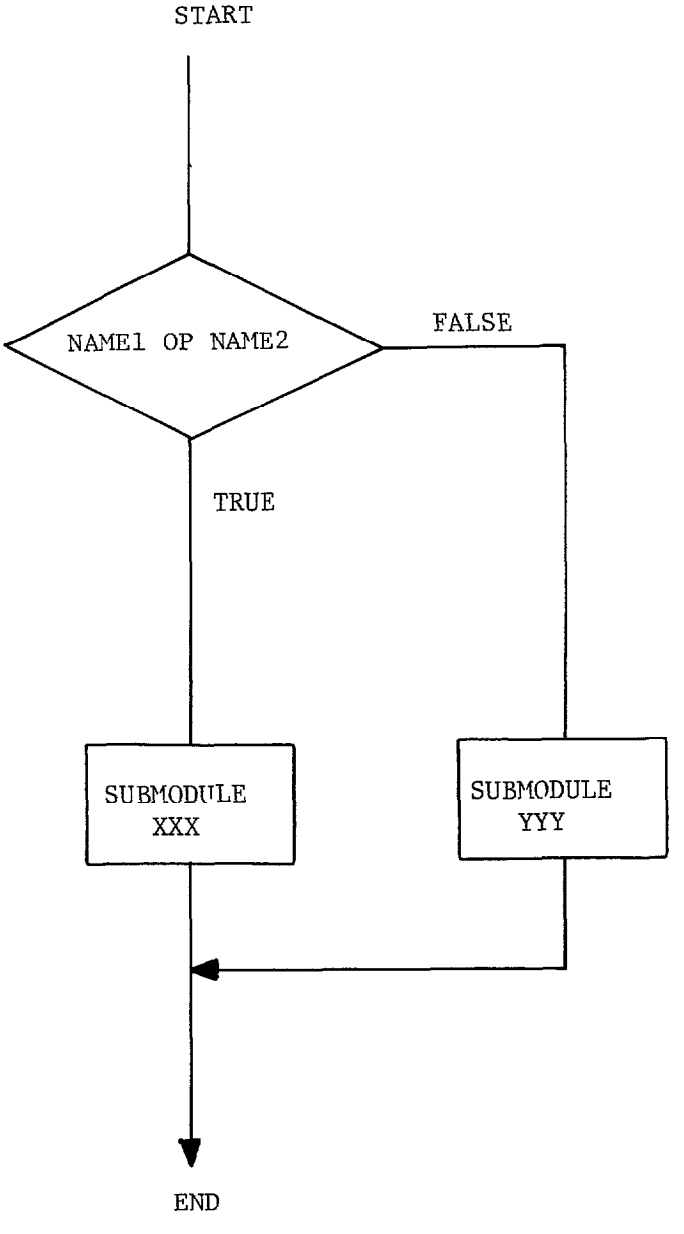


NOTE: ALL "NEXT" POINTERS ARE DIRECTED TO THE NEXT HIGHER CBL NUMBER.

PTXTI


START
REQUEST INPUT FROM DEVICE SPEC1
SUCCESSFUL TERMINATION
NO
YES
PLACE TEXT SIGNALS IN NAME 1
STORE ERROR CODE IN NAME3
COUNT = NUMBER OF TEXT SIGNALS INPUT
SUBMODULE XXX
SUBMODULE YYY
END

PTEST


START
NAME1 OP NAME2
FALSE
TRUE
SUBMODULE XXX
SUBMODULE YYY
END

GENERAL ELECTRIC COMPANY • DRIVE SYSTEMS DEPARTMENT
SALEM, VA. 24153

GENERAL ELECTRIC